

AUDITORÍA GENERAL RAG MUNBOT

Diagnóstico resumido

- Falla observada:** El bot no responde preguntas sobre trámites/documentos (p. ej. “*permiso de aterrizaje*”) con información disponible, sino que devuelve un mensaje genérico de no entender. En los logs se ve que dichas consultas se clasifican con intención `n/a` y activan el fallback genérico en vez del flujo RAG [GitHub](#).
- Flujo esperado:** Consultas de trámites o info municipal deberían clasificarse en la intención de búsqueda documental (RAG) – ya sea “*documento*”, “*tramite*” o “*faq*” según el caso – para luego extraer respuesta desde la base de documentos (colección Qdrant). El router *config* lista varias colecciones RAG (incluyendo `_tramites` donde aparece “permiso de aterrizaje”) [GitHub](#), y el mapa de intenciones (INTENTMAP) traduce “documento”/“tramite”_ a la acción `ask_document` del orquestador [GitHub](#). Por tanto, el sistema está preparado para manejar esas consultas vía RAG.
- Causa raíz probable:** El clasificador de intención basado en LLM no está reconociendo correctamente estas consultas informativas. En el caso de “permiso de aterrizaje”, la clasificación resultó `intent: "n/a"` (ni *documento* ni *tramite*), lo que desencadena el fallback sin llamar al servicio RAG [GitHub](#). Esto puede deberse a que el IntentEngine devolvió un puntaje de similitud bajo o a que la respuesta del LLM clasificador no coincidió exactamente con las etiquetas esperadas, quedando bajo el umbral y marcándose como `n/a` [GitHub](#).
- Confirmación en código:** Actualmente, si la intención queda como *no entendida*, el orquestador simplemente responde con disculpa y no intenta búsqueda documental [GitHub](#). No hay lógica de *fallback* hacia RAG: la función `handle_fallback` retorna directamente “*Lo siento, no he entendido tu consulta...*” sin consultar el servicio de documentos.
- Impacto:** Todas las preguntas que el clasificador no identifique con confianza caen en este vacío. Consultas válidas sobre trámites (que sí están en la base de conocimiento) terminan sin respuesta útil. El bot parece “no entender” incluso cuando la información existe, afectando la experiencia del usuario.

Hipótesis priorizadas

- Formato de salida del clasificador LLM (alta probabilidad ~70%):** El modelo de clasificación podría no estar devolviendo exactamente las etiquetas esperadas. Por ejemplo, si respondió “*trámite*” con tilde o con texto extra, el código lo marcará como no válido y asignará `n/a` [GitHub](#). *Dato a validar:* revisar la salida cruda del endpoint `doc-classify_intent_llm` para consultas de trámites; si contiene acentos o frases adicionales, confirmaría esta hipótesis.
- Umbral de similitud del IntentEngine muy estricto (~50%):** Aunque “permiso de aterrizaje” está en los datos, es posible que el cálculo de similitud no superó el umbral dinámico (0.12 para ~3 palabras) y el engine devolvió `intent: n/a` [GitHub](#). El fallback LLM tampoco corrigió esto, quizás por contexto insuficiente. *Cómo verificar:* bajar temporalmente `UMBRAL_SCORE` (ej. a 0.1) o inspeccionar el `confidence` que retorna `classify_intent_payload` para esa frase y ver si estaba ligeramente bajo 0.12. Si con umbral menor clasifica como “*tramite*”, esta sería la causa.
- Intenciones mal mapeadas o ausentes (baja probabilidad ~10%):** Se consideró que quizá el clasificador (u otra parte) usa nombres distintos, ej. `doc-buscar_fragmento_documento` o `info-respuesta_faq`, que no coinciden con el INTENTMAP. *Sin embargo, el INTENT_MAP vigente ya contempla las categorías “faq”, “documento” y “tramite” mapeadas a ask_document* [GitHub](#), y no se encontraron referencias a esos nombres legacy en el flujo actual. Por tanto, es poco probable que el fallo provenga de un mapeo faltante. (Suposición descartada con evidencia del código)._

Acciones inmediatas (Comandos)

A continuación, se proponen comandos para **reproducir el problema** y verificar el entorno, así como para **aplicar el parche** y confirmar la solución:

```
## 1. Verificar que la colección de documentos en Qdrant tiene puntos (debe devolver count ~199)
curl -s http://qdrant:6333/collections/munbot_docs/points/count \
  -H 'Content-Type: application/json' -d '{}' | jq .

## 2. Reproducir la consulta problemática al orquestador (antes del parche)
## Esperado actualmente: la respuesta es la frase de no entendimiento (fallback genérico).
curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"pregunta": "Necesito información sobre el permiso de aterrizaje"}' | jq .

## 3. (Opcional) Probar directamente el clasificador de intención LLM_docs para ver qué produce
## (Debería devolver intent "n/a" actualmente para la frase de ejemplo, indicando el fallo de clasificación)
curl -s -X POST http://llm_docs-mcp:8000/tools/call -H 'Content-Type: application/json' \
  -d '{"tool": "doc-classify_intent_llm", "params": {"texto": "Necesito informacion sobre el permiso de aterrizaje"}}' | jq .

## --- APLICAR PARCHES: editar el archivo mcp-core/orchestrator.py ---
## Abrir el archivo en un editor y modificar la función handle_fallback según el diff propuesto abajo.

## 4. Reiniciar el contenedor MCP-Core para cargar los cambios (ejemplo con Docker Compose)
docker compose restart mcp-core
```

```
## 5. Repetir la consulta de trámite tras el parche
## Ahora debería obtener una respuesta extraída de los documentos (o al menos "No se encontró respuesta" en vez
de la disculpa).
curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"pregunta": "Necesito información sobre el permiso de aterrizaje"}' | jq .

## 6. Probar un caso fuera de base (ejemplo texto aleatorio)
## para confirmar que si RAG no halla nada, el bot aún devuelve la disculpa genérica.
curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"pregunta": "asdjkhqwue ejemplo"}' | jq .
```

Parche propuesto (DIFF)

A continuación se muestra el *diff* unificado para implementar la solución en `mcp-core/orchestrator.py`. El parche modifica la función de fallback para invocar una búsqueda RAG cuando la intención es `n/a`, antes de rendirse. Si la búsqueda no arroja resultados relevantes (o el servicio no tiene respuesta), entonces mantiene el mensaje de “no entendido” como último recurso:

```
--- a/mcp-core/orchestrator.py
+++ b/mcp-core/orchestrator.py
@@
-def handle_fallback(session_id: str, user_input: str) -> Dict[str, Any]:
-    ## Lógica de fallback, escalamiento a humano, etc.
-    return {"respuesta": "Lo siento, no he entendido tu consulta. ¿Podrías reformularla?", "session_id":
session_id}
+def handle_fallback(session_id: str, user_input: str) -> Dict[str, Any]:
+    ## SUPOSICIÓN: intentar RAG como último recurso cuando la intención es 'n/a'
+    rag_result = handle_document_query(
+        session_id,
+        user_input,
+        {"tema_especifico": None, "tramite": None, "departamento": None},
+        [],
+    )
+    respuesta = (rag_result or {}).get("respuesta", "") if isinstance(rag_result, dict) else ""
+    if respuesta.strip().lower().startswith("no se encontr"):
+        respuesta = "Lo siento, no he entendido tu consulta. ¿Podrías reformularla?"
+    return {"respuesta": respuesta, "session_id": session_id}
```

Explicación: Ahora, ante una intención no clara, el orquestador llamará a `handle_document_query` para buscar en la base de conocimientos (pasando la pregunta tal cual, sin entidades especiales). Si el servicio de documentos devuelve un texto de respuesta, se entrega directamente al usuario. En caso de que esa respuesta indique “*No se encontró una respuesta.*” (lo que implica que ningún fragmento relevante fue encontrado) o si por alguna razón viene vacía, entonces se recurre al mensaje genérico de no comprensión original. **Esta solución es una mejora incremental:** asegura que se intente una respuesta basada en RAG para preguntas potencialmente válidas, sin perder el fallback seguro en caso de no hallar nada.

Pruebas/Checks de verificación

Después de aplicar el parche, se recomiendan las siguientes pruebas para validar que el comportamiento ha mejorado:

- Consulta de trámite antes/después:** Repetir la consulta de ejemplo (“*Necesito información sobre el permiso de aterrizaje*”).
 - Antes del parche:** El JSON de respuesta contiene `"respuesta": "Lo siento, no he entendido tu consulta..."` [GitHub](#), evidenciando que no se usó RAG.
 - Después del parche:** El bot debe devolver una respuesta relevante o al menos distinta. Por ejemplo, debería mencionar requisitos o vigencia del permiso de aterrizaje (según los datos). Si la información existe, la respuesta podría ser un párrafo explicativo; si no hay coincidencias, podría verse `"respuesta": "No se encontró una respuesta."`. En ningún caso debería ya verse la frase de disculpa en la primera respuesta a esa pregunta.
- Pregunta fuera de base:** Probar con una entrada sin sentido o fuera del alcance (ej: “*asdf qwerty*”).
 - Se espera que el sistema **siga devolviendo el fallback genérico**. Internamente, ahora también intentará RAG (probablemente sin encontrar nada útil), y al detectar “no se encontró respuesta”, caerá al mensaje “*Lo siento, no he entendido...*”. Esto confirma que el cambio no rompe el manejo de consultas realmente inválidas.
- Pequeñas charlas y flows existentes:** Verificar que intenciones ya manejadas (saludos, agenda, reclamos) no se vean afectadas. Por ejemplo, enviar “*Hola*” debe seguir clasificando como *saludo* y respondiendo con la variante definida, sin activar el nuevo fallback RAG (lo cual sucederá, ya que el flujo de smalltalk se ejecuta antes que fallback [GitHub](#)).
- Logs de seguimiento:** Revisar los logs de `mcp-core` y `llm_docs-mcp` tras las pruebas. Deben apreciarse nuevas entradas indicando el uso del servicio de documentos durante el fallback. En particular, deberían aparecer líneas de routing como `intent=doc-generar_respuesta_llm` con respuesta 200 OK, incluso para la consulta que antes era `n/a`. Esto confirma que el orquestador ahora sí está invocando la búsqueda RAG en esas situaciones.

Riesgos y rollback

- Riesgos:** La modificación propuesta es segura en términos de estabilidad del sistema (no introduce dependencias nuevas, solo usa un flujo ya existente). El principal riesgo es **funcional**: que para preguntas realmente fuera de contexto, el bot intente dar alguna

respuesta de la base de conocimientos que pueda ser irrelevante. En el peor de los casos, el usuario obtendría un “*No se encontró una respuesta.*” en lugar de “*No te entendí*”. Esto puede ser menos claro para el usuario, pero sigue indicando que la pregunta no obtuvo información. Además, cada consulta n/a ahora conlleva una llamada extra al servicio RAG, lo que **aumenta ligeramente la latencia** en estos casos y carga al servicio de vector search. Se deberá monitorear el impacto, aunque al ser consultas que ya antes eran problemáticas, este overhead es acotado.

- Rollback:** Si el comportamiento no es el esperado o introduce confusiones, revertir el cambio es sencillo. Se puede restaurar el archivo original `orchestrator.py` (quitando las líneas añadidas en `handle_fallback` y dejando solo la respuesta fija). Esto podría hacerse con un control de versiones (`git revert` del commit correspondiente) o manualmente. Tras revertir, reimplantar con un `docker compose up -d --build mcp-core` para que el contenedor cargue la versión anterior. El sistema volvería inmediatamente al comportamiento previo (ignorando RAG en intenciones n/a). Es importante sopesar el rollback vs. la mejora aportada, ya que la solución propuesta apunta a una mejor experiencia en la mayoría de consultas informativas.

Siguientes pasos

- Mejorar clasificador de intención:** Ajustar el pipeline de clasificación para reducir la probabilidad de falsos `n/a`. En concreto, normalizar acentos y caracteres especiales en la salida del LLM clasificador (por ejemplo, tratar “trámite” y “tramite” como equivalentes) para evitar pérdidas por formato. También considerar entrenar unos pocos ejemplos adicionales o *fine-tuning* ligero para que el modelo reconozca mejor términos municipales específicos.
- Ajustar umbrales y recall:** Revisar el umbral adaptativo del IntentEngine. Una opción es disminuir ligeramente `UMBRAL_SCORE` (ej. de 0.18 a 0.15) para aumentar recall en detección de intenciones de documento, siempre que no provoque muchos clasificadores incorrectos. Complementariamente, implementar **MMR (Maximal Marginal Relevance)** o aumentar `top_k` en las búsquedas de Qdrant, para que el servicio RAG ofrezca más candidatos y así el IntentEngine tenga más chance de encontrar coincidencias y contextualizar la intención.
- Métricas y monitoreo:** Activar métricas que ya existen o añadir nuevas para seguimiento de este flujo. Por ejemplo, medir la tasa de consultas que terminan con `intent=n/a` y cuántas de esas ahora logran encontrar respuesta con el fallback RAG. Monitorear también la latencia añadida en estos casos. Se puede aprovechar Prometheus/Grafana (ya hay endpoints en el sistema) para crear alertas si la tasa de *fallbacks* supera cierto umbral, indicando potenciales problemas en la clasificación.
- Validación con dataset:** Preparar un listado de ~50 consultas de usuario representativas (incluyendo saludos, trámites comunes, preguntas de departamentos, consultas fuera de alcance) y probar el sistema actualizado. Esto permitirá evaluar cuántas reciben respuesta adecuada tras el cambio. Especial atención a que ninguna intención válida se degrade con el nuevo flujo. Los resultados de este test de regresión pueden alimentar mejoras posteriores (por ejemplo, si aún hay trámites que salen como “No se encontró respuesta”, incorporar sinónimos o alias en los datos).
- Mejoras en respuestas RAG:** Considerar pulir cómo se formulan las respuestas cuando no hay resultados claros. Actualmente se devuelve “*No se encontró una respuesta.*” cuando el RAG no halla nada [GitHub](#). Podría reemplazarse por algo más informativo al usuario, como “*No tengo información sobre ese tema.*”. Asimismo, evaluar si conviene enumerar fuentes o sugerir reformulación en esos casos.
- Escalamiento opcional a humano:** Si la estrategia de fallback RAG demuestra ser útil, un siguiente nivel podría ser: para las consultas que aún queden sin respuesta (ni intención ni RAG), registrar esos casos (quizá ya se almacenan en `missed_questions.csv`) y estudiar la posibilidad de derivarlas a un agente humano o de enriquecer la base de conocimientos con esas preguntas frecuentes no cubiertas. Esto cierra el ciclo de mejora continua del asistente.

En resumen, el parche aplicado aborda el problema inmediato asegurando que las consultas sobre **trámites/documentos no queden sin intentar respuesta**, aprovechando el sistema RAG existente. Con las afinaciones adicionales en el clasificador y monitoreo, MunBot podrá brindar respuestas más completas y precisas a los ciudadanos de Curoscant, reduciendo significativamente el porcentaje de “*no entendí*” en consultas donde sí hay información disponible.

AUDITORÍA CLASIFICADOR DE INTENCIÓN

Umbral de score actual y cálculo por longitud de consulta

El **UMBRAL_SCORE** actual está definido en `0.18` (es decir, 18%) [GitHub](#). Sin embargo, el sistema ajusta dinámicamente este umbral según el largo de la consulta usando la función `dynamic_threshold`. En concreto:

- Consultas muy cortas (≤ 2 palabras):** umbral reducido a 0.08 (8%).
- Consultas medianas (3–4 palabras):** umbral fijado en 0.12 (12%).
- Consultas más largas (≥ 5 palabras):** se aplica el umbral por defecto de **0.18** [GitHub](#).

Esto significa que a mayor número de tokens en la pregunta del usuario, más alto es el score mínimo exigido para considerar la intención como válida (se vuelve al 18% cuando la consulta tiene 5 palabras o más). La lógica detrás es no penalizar saludos o frases muy cortas (requieren menor score), pero exigir mayor certeza en frases largas.

Aplicación del umbral en la clasificación (intent_engine)

El **IntentEngine** aplica este umbral adaptativo al determinar la intención final. Después de calcular el mejor puntaje (`score`) para un candidato de conocimiento, compara dicho score contra el umbral dinámico calculado para el texto usuario. Si el score es menor que ese umbral, **la consulta se clasifica como intent “n/a”** (no identificada) en lugar de asignarle alguna intención [GitHub](#). En el código fuente, se observa que:


```
thr = dynamic_threshold(texto_usuario) if score < thr: return { "intent": "n/a", ... , "confidence": round(score,4), "needs_disambiguation": True }
```

Es decir, si el mejor candidato no supera el **umbral de confianza**, el motor devuelve explícitamente `"intent": "n/a"` junto con `needs_disambiguation: True` [GitHub](#). En otras palabras, el sistema opta por *no* decidir ninguna intención válida cuando la similitud es baja, forzando un fallback. Esto explica por qué consultas como *"permiso de aterrizaje"* acabaron en *n/a*: al no encontrar coincidencias suficientemente altas en la base de conocimiento, el score quedó bajo el umbral (para 7 palabras, $\text{thr} \approx 0.18$) y se activó este caso de no-intención.

Formato de salida esperado del clasificador LLM y posibles inconsistencias

El clasificador de intención basado en LLM (`doc-classify_intent_llm`) está diseñado para devolver **exactamente una de las etiquetas predefinidas**: `"faq"`, `"documento"`, `"tramite"`, `"agenda"`, `"reclamo"`, `"saludo"`, `"despedida"`, `"agradecimiento"` o `"n/a"` [GitHub](#). El prompt dado al modelo le instruye devolver **solo la etiqueta** sin explicaciones. Internamente, el código espera que la respuesta del LLM sea una cadena con la categoría, en minúsculas y sin adornos. De hecho, inmediatamente después de la generación se aplica `strip().lower()` a la respuesta del modelo [GitHub](#).

Importante: Actualmente **solo se normaliza a minúsculas y se quitan espacios extremos**, pero **no** se corrigen tildes ni otros caracteres. Esto puede producir inconsistencias sutiles: por ejemplo, si el LLM devolviera **"Trámite"** con tilde o `"Tramite."` con un punto final, el código lo convertiría a `"trámite"` (minúsculas, pero conservando la tilde) o `"tramite."` (con el punto) respectivamente. Ninguna de esas variantes coincide exactamente con las etiquetas válidas esperadas. El sistema comprueba la pertenencia estricta de la etiqueta a la lista `LLM_INTENT_LABELS` [GitHub](#), por lo que una discrepancia de tilde, mayúscula interna o carácter extra provocará que **no se reconozca como etiqueta válida**. En ese caso, el clasificador **forzará la salida a** `"n/a"` [GitHub](#).

En resumen, el resultado esperado del LLM es una palabra exacta igual a las categorías definidas (sin variaciones ortográficas). **Cualquier desviación** – por ejemplo `"tramite"` vs `"trámite"`, diferencias en espacios o acentos – **hará que el sistema no la considere válida** y la reemplace por *n/a*. Actualmente no existe una lógica de normalización más allá de minúsculas para comparar la salida del LLM con las etiquetas válidas, a diferencia del motor de conocimiento base que sí normaliza eliminando acentos y caracteres especiales para sus comparaciones [GitHub](#). Esta falta de normalización robusta en la respuesta del LLM es un factor clave en los errores observados.

Efecto en consultas como "permiso de aterrizaje" y "certificado de residencia"

En la práctica, consultas de tipo trámite/documento han evidenciado el problema descrito. Por ejemplo, el usuario ingresó: **"Necesito información sobre el permiso de aterrizaje"**, esperando que el sistema identifique un trámite o documento específico. Sin embargo, en los logs se observa que:

- El mensaje fue recibido correctamente,
- El clasificador de intención devolvió finalmente **intent: "n/a"** en ese caso, por lo que **no se activó el flujo RAG** (no se realizó búsqueda en documentos).

¿Por qué ocurrió? Analizando paso a paso:

- Motor de conocimiento (RAG estático):** No encontró un match fuerte para "permiso de aterrizaje". El único parecido podría ser "permiso de circulación" u otros permisos si existían en la base, pero la similitud fue baja. Con 7 tokens en la consulta, el umbral era 0.18; cualquier solapamiento parcial (ej. solo palabras comunes "permiso" y "de") dio un score inferior ($\approx 0.10\text{--}0.15$). Por debajo del umbral, el IntentEngine retornó `intent: "n/a"` [GitHub](#) marcando incertidumbre. Esto activó el **fallback al LLM**. (*En contraste, si el score hubiera superado 0.18, se habría clasificado quizás como "documento"; por ejemplo, en pruebas unitarias similares "_permiso de circulación" sí alcanza suficiente score y se clasifica como documento* [GitHub](#))._
- Clasificador LLM:** Al entrar en juego, debería haber categorizado la consulta como "documento" o "tramite" dada la naturaleza del “permiso”. Sin embargo, su respuesta no coincidió con las etiquetas válidas esperadas. Posiblemente el modelo devolvió **"n/a"** por no reconocer la categoría (tratándolo como caso no encajable), **o bien** devolvió *"tramite"* con tilde (`"trámite"`) u otro ligero desvío. En cualquiera de esos escenarios, el código interpretó la salida como no válida y la redujo a *n/a* [GitHub](#). El resultado combinado fue que la intención permaneció en *"n/a"*, tal como vimos en los registros.

Otro ejemplo, **"certificado de residencia"**, muy probablemente habría pasado por la misma situación. Este tipo de consulta debería clasificarse idealmente como un **"documento"** (es un certificado oficial); sin embargo, si no hay un alias exacto ni suficiente overlap en la base, el motor regresaría *n/a* y dependería del LLM. El LLM, de no identificar claramente la etiqueta, podría fallar por las mismas razones (tilde o incertidumbre). En consecuencia, antes del fix es muy posible que **"certificado de residencia" tampoco active el flujo** (quedando en *n/a*). En general, cualquier consulta de trámite/documento con palabras no vistas o que el LLM responda con ligeras variaciones enfrenta este riesgo. Esto confirma que el **clasificador no tenía la robustez adecuada** para entender consultas de trámites menos comunes, por la combinación de umbral alto y falta de normalización en la respuesta del LLM.

Entrega de resultados y acciones

- Causa raíz del *intent* clasificado como *n/a*:** La raíz del problema está en dos factores combinados: (a) el **umbral de score adaptativo elevado** para consultas largas hizo que queries como "permiso de aterrizaje" no logran suficiente puntuación de coincidencia en la base de conocimientos, retornando inicialmente *intent: n/a*. (b) Al entrar el **clasificador LLM de respaldo**, su respuesta **no coincidió exactamente con las etiquetas esperadas** (ya sea porque el modelo respondió `"n/a"` al no estar seguro, o porque incluyó una tilde/carácter extra en `"tramite/documento"`). El código no reconoce la intención en esos casos y la deja en

n/a, en lugar de asignarla a "tramite" o "documento" correspondientes [GitHub](#). En síntesis, un ligero desajuste en la respuesta del modelo (tildes, mayúsculas, formato) causó que el sistema no la valide como intención conocida, de allí la clasificación “*n/a*”.

2. **Reproducción y evidencia en logs:** Para reproducir el fallo, se puede usar la API de clasificación de intención con las consultas mencionadas. Por ejemplo, enviando la frase "permiso de aterrizaje" mediante la herramienta `doc-classify_intent_llm` (vía endpoint REST o el cliente Python) antes del fix, la respuesta resultante es *intent: n/a*. En los logs adjuntos se observa claramente esta secuencia: el usuario envía "*Necesito información sobre el permiso de aterrizaje*" y el sistema registra **Intención clasificada: n/a** para esa petición. Esto confirma que el flujo llegó hasta el LLM pero la etiqueta no fue reconocida. De forma similar, cualquier término de trámite poco común (ej. "*certificado de residencia*") puede probarse del mismo modo para verificar que, sin corrección, el clasificador devolvía *n/a*. No se halló referencia a "*certificado de residencia*" en los logs, lo que sugiere que el usuario no obtuvo respuesta útil (posiblemente porque también cayó en *n/a*).
3. **Parche mínimo propuesto para mayor robustez:** Sin necesidad de retocar el modelo LLM en sí, el fix consiste en **normalizar la salida textual del LLM antes de compararla con las etiquetas válidas**. En concreto, se debe eliminar la sensibilidad a tildes, espacios sobrantes o puntuación trivial. Por ejemplo, en el código de `intent_classifier.py`, justo después de convertir a minúsculas la etiqueta devuelta por el modelo, aplicar una normalización similar a `_norm` (como la usada en el IntentEngine estático) que remueva diacríticos. Esto podría lograrse con `unicodedata.normalize("NFKD", label).encode("ascii", "ignore").decode("ascii")` u otro método equivalente [GitHub](#), para convertir "trámite" en "tramite", **manteniendo** sin cambios casos especiales como "n/a". Asimismo, se pueden recortar comillas o puntos finales si existen. Tras normalizar así la variable `label`, la verificación `if label not in LLM_INTENT_LABELS` reconocerá correctamente "*tramite*" como válido (ya presente en la lista) en lugar de descartarlo. Este es un cambio mínimo (añadir una línea de procesamiento de la cadena) que **no modifica el modelo LLM subyacente**, sino solo cómo interpretamos su respuesta. Con este parche, pequeñas diferencias ortográficas ya no impedirán que se detecte la intención.
4. **Comandos para verificar la corrección:** Después de aplicar el parche, se recomienda ejecutar de nuevo las consultas problemáticas y algunos tests automatizados:

- **Consulta directa:** Invocar el endpoint REST de clasificación de intención con los mismos ejemplos. Por ejemplo:

```
curl -X POST http://<host>:8000/tools/doc-classify_intent_llm \
-H "X-API-KEY: <API_KEY>" \
-d '{"texto": "permiso de aterrizaje"}'
```

Debería responder con un JSON donde "intent" ya **no sea** "n/a", sino "documento" o "tramite" según lo que dictamine el modelo (lo esperable es "documento"). Igualmente probar con "certificado de residencia" y confirmar que entrega "documento" como intent.

- **Cliente Python / Unit tests:** Utilizar el cliente `LlmDocsClient.classify_intent` dentro de MCP-core o correr los tests existentes. Por ejemplo, correr `pytest tests/test_intent_classifier.py` para asegurarse de que todos los casos pasan. Se puede añadir un caso de prueba simulando una respuesta con tilde: p.ej. mockear que el LLM devuelve "Trámite" y verificar que el resultado final normalizado sea "tramite" en lugar de *n/a*.
 - **Verificar en logs:** Reiniciar el stack con el cambio y realizar una consulta real desde la interfaz (por ej., escribir "*permiso de aterrizaje*"). En los logs de `mcp-core` se debería ver ahora algo como `Intención clasificada: documento` (u otra intención válida) en lugar de *n/a*. También se esperaría que el flujo RAG se active (búsqueda de fragmentos en Qdrant y respuesta del LLM con información) para esas consultas, confirmando que ya **no quedan atascadas en n/a**.
5. **Riesgos y plan de reversión:** El ajuste propuesto es bastante seguro y de impacto acotado. **Riesgos:** Podría, en teoría, hacer que una salida del LLM ligeramente malformada coincida con una etiqueta cuando no era la intención del modelo. Sin embargo, dado que las categorías están bien definidas y son pocas, es difícil que la normalización introduzca falsos positivos (por ejemplo, remover tildes no convertirá una palabra distinta en una categoría válida por accidente, solo uniformiza *tramite*). Otro posible riesgo es que la normalización excesiva modifique el token "n/a", pero se ha tenido cuidado de preservar el slash y la estructura de "n/a" en la comparación. En general, el cambio solo amplía la tolerancia a variaciones ortográficas esperables, por lo que **no se anticipan regresiones** en clasificaciones previas (salvo mejora en los casos problemáticos).

En caso de algún comportamiento no deseado, la reversión es sencilla: se podría quitar o ajustar la línea de normalización añadida. Dado que es un cambio aislado en la post-proceso de la etiqueta, volver al estado anterior implicaría simplemente eliminar esa transformación adicional en el código. Alternativamente, si se quisiera desactivar rápidamente, podría introducirse una bandera de configuración para ignorar la normalización si fuera necesario. No obstante, considerando las pruebas, el parche mejora la robustez sin consecuencias negativas, así que es poco probable que sea necesario revertirlo.

Fuentes: Código fuente del proyecto Munbot-MCP y registros de ejecución del sistema proporcionados. En particular, se consultaron los módulos de clasificación de intención estático y con LLM [GitHub](#) [GitHub](#) [GitHub](#) [GitHub](#) [GitHub](#), las pruebas unitarias relevantes [GitHub](#), así como los archivos de log para evidenciar el comportamiento actual. Todas estas evidencias sustentan las conclusiones y la solución aquí propuestas.

AUDITORIA PIPELINE RAG

1. **Colección activa por defecto:** La colección de Qdrant utilizada por defecto es “**munbot_docs**”. Esto se establece en el código al definir la variable de entorno `QDRANT_COLLECTION` con valor predeterminado “munbotdocs” [GitHub](#). Por ejemplo, archivos como `qdrant_helpers.py` y los scripts de indexación confirman este valor por omisión [GitHub](#), de modo que si no se especifica otra colección, el sistema empleará `_munbot_docs`.

2. Tamaño del vector (dimensión de embeddings): La dimensionalidad del vector de embeddings es **384**. En la configuración (por ejemplo, en el *docker-compose* y archivos de entorno) se define `EMBEDDINGS_DIM=384` [GitHub](#). Esto concuerda con el modelo de embeddings utilizado (un modelo *MiniLM* multilingüe), que produce vectores de 384 dimensiones.

3. Valor *top_k* utilizado y *fallback*: La función central de búsqueda `obtener_fragmentos` tiene un parámetro `k` con valor por defecto 3 [GitHub](#), es decir, normalmente retornaría 3 fragmentos relevantes tras el reordenamiento (*reranking*). Sin embargo, en la práctica varios endpoints solicitan más resultados inicialmente. Por ejemplo, la búsqueda semántica invoca a Qdrant con `top_k=5` para obtener hasta 5 candidatos [GitHub](#), los cuales luego son reordenados y se toman los mejores *k* finales (por ej., los 3 mejores si *k*=3).

En cuanto al *fallback*, actualmente **no existe una estrategia sofisticada** más allá de responder que no hay información. Si la búsqueda en la base vectorial no arroja fragmentos relevantes (o la similitud es baja), el código simplemente devuelve una respuesta genérica. En el pipeline *RAG*, si `obtener_fragmentos` resulta en lista vacía, el sistema responde con un mensaje como “*No encontré información relevante para tu consulta.*” y sin fuentes [GitHub](#). De modo similar, en la pasarela (*gateway*) se verifica el puntaje del mejor resultado de Qdrant y, si no alcanza un umbral, se activa un *fallback* que devuelve “*No dispongo de esa información*” sin referencias [GitHub](#). En esencia, cuando no hay resultados, el sistema **no** envía contexto al LLM; en su lugar, informa al usuario que no se halló información útil.

AUDITORIA DEL ORQUESTADOR

Diagnóstico Resumido

- El orquestador recibe la consulta, llama a `llm_docs-mcp /tools/call` (200 OK) y **clasifica** `saludo` correctamente, pero en consultas de contenido (ej. “permiso de aterrizaje”) **clasifica** `n/a` y **responde el fallback genérico** “no entendí”, sin intentar RAG. Evidencia: múltiples `"[INTENT] ...: n/a"` seguidos de `POST /orchestrate 200` sin trazas de búsqueda/documentos.
- No hay “fallback RAG”** en el orquestador: ante `n/a` no se hace una sonda de recuperación (probe) contra `llm_docs-mcp`; se corta el flujo con disculpa.
- Conectividad OK** entre servicios (varios `HTTP/1.1 200 OK` en `llm_docs-mcp:8000/tools/call` y `/health`). El problema es de **ruteo lógico**, no de red.
- El **RAG está poblado** (199 chunks indexados en `munbot_docs`) y responde; simplemente **no es invocado** cuando la intención cae en `n/a`.
- Trazabilidad insuficiente:** no se logea qué ruta tomó el orquestador ni si se evaluó (o no) una sonda RAG.

Hipótesis Priorizadas

- H1 – Falta de “RAG-first fallback”** al caer en `n/a` (80%): el orquestador no intenta recuperación mínima antes de rendirse. Se valida si al forzar un probe a `llm_docs-mcp` aparecen fragmentos; si sí, confirma H1.
- H2 – Mapeo de intents a handlers incompleto** (45%): `documento/tramite/faq` podrían no estar cableados a handlers de documentos/FAQ o ese bloque no se ejecuta cuando el clasificador devuelve `n/a`. Se valida revisando el bloque `if intent == ...` y el enrutamiento.
- H3 – Falta de logging estructurado de ruteo** (40%): sin logs de “ruta seleccionada” es difícil auditar. Se valida añadiendo logs y observando decisiones.

Acciones Inmediatas (Comandos)

```
## 1) Confirmar que hay puntos en Qdrant (conteo > 0)
curl -s http://qdrant:6333/collections/munbot_docs/points/count \
  -H 'Content-Type: application/json' -d '{}' | jq .

## 2) Reproducir el caso problemático (antes del parche): hoy debería devolver el fallback genérico
curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"text":"Necesito información sobre el permiso de aterrizaje"}' | jq .

## 3) Ver rastro de decisiones (antes del parche): INTENT y respuesta
docker logs --since=10m mcp-core | egrep -i '\[INTENT\]|\[ROUTER\]|\[RAG-PROBE\]|orchestrate' | tail -n 120
```

Parche Propuesto (diff)

Notas:>

- Se añade un **probe RAG** cuando `intent == "n/a"` para intentar recuperar fragmentos antes de rendirse.
- Se añade **logging** de ruteo y del resultado del probe.
- Los **nombres de tool/argumentos** en `llm_docs-mcp` están marcados como **SUPOSICIÓN**: reemplázalos por los reales (ver tus `services/llm_docs-mcp/tool_schemas/*.json`).

A. orquestador: fallback con RAG y logs de ruteo

```
@@
-import logging
+import logging
```

```

+import httpx  ## para el probe RAG
@@
    logger = logging.getLogger("munbot")

+def _log_router(event: str, **kv):
+    info = {"event": event, **kv}
+    try:
+        logger.info("[ROUTER] %s", info)
+    except Exception:
+        logger.info("[ROUTER] %s", str(info))
+
+def try_rag_probe(user_text: str, top_k: int = 3, timeout_s: int = 20):
+    """
+    Intenta recuperar respuesta vía llm_docs-mcp. Devuelve (ok: bool, data: dict|None).
+    SUPOSICIÓN: tool/function y argumentos; sustituir por los reales.
+    """
+    payload = {
+        "messages": [
+            {"role": "system", "content": "Eres un asistente municipal. Devuelve respuesta breve y cites fuentes si existen."},
+            {"role": "user", "content": user_text}
+        ],
+        "tools": [
+            { "type": "function",
+              "function": {
+                  "name": "doc-generar_respuesta_llm",  ## SUPOSICIÓN
+                  "arguments": {"query": user_text, "top_k": top_k}  ## SUPOSICIÓN
+              }
+            }
+        ]
+    }
+    try:
+        r = httpx.post("http://llm_docs-mcp:8000/tools/call", json=payload, timeout=timeout_s)
+        r.raise_for_status()
+        data = r.json()
+        ## SUPOSICIÓN: presencia de fragments/sources indica hallazgos
+        sources = data.get("sources") or data.get("fragments") or []
+        ok = isinstance(sources, list) and len(sources) > 0
+        _log_router("rag_probe_done", ok=ok, hits=len(sources) if isinstance(sources, list) else 0)
+        return ok, data
+    except Exception as e:
+        _log_router("rag_probe_error", error=str(e))
+        return False, None
+
def orchestrate():
    ## ...
    user_text = payload.get("text") or payload.get("pregunta") or ""
@@
-    intent = classify_main_intent(user_text)
-    logger.info("[INTENT] Intención clasificada: %s", intent)
+    intent = classify_main_intent(user_text)
+    logger.info("[INTENT] Intención clasificada: %s", intent)
+    _log_router("intent_decided", intent=intent)
@@
-    if intent == "n/a":
-        return handle_fallback(session_id, user_text)
+    if intent == "n/a":
+        ## Antes de rendirse, intentemos RAG de bajo costo
+        _log_router("intent_na_try_rag")
+        ok, data = try_rag_probe(user_text, top_k=3)
+        if ok and isinstance(data, dict):
+            ## SUPOSICIÓN: formateador; en su defecto, devolvemos data "tal cual"
+            return {"respuesta": data.get("answer") or data.get("respuesta") or " ",
+                    "fuentes": data.get("sources") or data.get("fragments") or [],
+                    "session_id": session_id}
+        return handle_fallback(session_id, user_text)
@@
-    elif intent in ("documento", "tramite"):
-        return handle_document_query(session_id, user_text, entidades, contexto)
+    elif intent in ("documento", "tramite"):
+        _log_router("route_document", intent=intent)
+        return handle_document_query(session_id, user_text, entidades, contexto)
@@
-    elif intent == "faq":
-        return handle_faq_query(session_id, user_text, entidades, contexto)
+    elif intent == "faq":
+        _log_router("route_faq", intent=intent)
+        return handle_faq_query(session_id, user_text, entidades, contexto)
@@

```



```
def handle_fallback(session_id: str, user_input: str):
-     return {"respuesta": "Lo siento, no he entendido tu consulta. ¿Podrías reformularla?", "session_id":
session_id}
+     return {"respuesta": "Lo siento, no he entendido tu consulta. ¿Podrías reformularla?", "session_id":
session_id}
```

B. (Opcional) router_config.json – asegurar intents cableados

Si usas archivo de configuración:

```
## 1) Reconstruir y reiniciar mcp-core
docker compose build mcp-core && docker compose up -d mcp-core

## 2) Consultas que antes caían en n/a: ahora deben devolver respuesta o al menos fuentes vacías pero SIN “no
entendí” inmediato
curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"text":"Necesito información sobre el permiso de aterrizaje"}' | jq .

curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"text":"¿Dónde saco el certificado de residencia?"}' | jq .

## Esperado (post-parche):
## - HTTP 200
## - JSON con "respuesta" no vacía si hay fragmentos; incluir "fuentes" (lista) cuando existan.
## - Si no hay hallazgos, puede responder algo neutro (y sólo entonces el fallback genérico).

## 3) Logs de ruteo y probe (deben aparecer nuevas marcas):
docker logs --since=5m mcp-core | egrep -i '\[ROUTER\]|\[INTENT\]|rag_probe' | tail -n 200
## Debe verse:
##   [ROUTER] {'event': 'intent_decided', 'intent': 'n/a'}
##   [ROUTER] {'event': 'intent_na_try_rag'}
##   [ROUTER] {'event': 'rag_probe_done', 'ok': true, 'hits': N}    ## si recuperó
##   ó [ROUTER] {'event': 'rag_probe_error', ...}

## 4) Sanity de smalltalk y FAQ: no debe romperse
curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"text":"hola"}' | jq .

curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"text":"¿Cuál es el horario de la oficina de tránsito?"}' | jq .
```

Si estas llaves ya existen, no cambies nada. El parche A es suficiente.

Pruebas/Check de Verificación

```
## 1) Reconstruir y reiniciar mcp-core
docker compose build mcp-core && docker compose up -d mcp-core

## 2) Consultas que antes caían en n/a: ahora deben devolver respuesta o al menos fuentes vacías pero SIN “no
entendí” inmediato
curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"text":"Necesito información sobre el permiso de aterrizaje"}' | jq .

curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"text":"¿Dónde saco el certificado de residencia?"}' | jq .

## Esperado (post-parche):
## - HTTP 200
## - JSON con "respuesta" no vacía si hay fragmentos; incluir "fuentes" (lista) cuando existan.
## - Si no hay hallazgos, puede responder algo neutro (y sólo entonces el fallback genérico).

## 3) Logs de ruteo y probe (deben aparecer nuevas marcas):
docker logs --since=5m mcp-core | egrep -i '\[ROUTER\]|\[INTENT\]|rag_probe' | tail -n 200
## Debe verse:
##   [ROUTER] {'event': 'intent_decided', 'intent': 'n/a'}
##   [ROUTER] {'event': 'intent_na_try_rag'}
##   [ROUTER] {'event': 'rag_probe_done', 'ok': true, 'hits': N}    ## si recuperó
##   ó [ROUTER] {'event': 'rag_probe_error', ...}

## 4) Sanity de smalltalk y FAQ: no debe romperse
curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"text":"hola"}' | jq .

curl -s http://mcp-core:5000/orchestrate -H 'Content-Type: application/json' \
  -d '{"text":"¿Cuál es el horario de la oficina de tránsito?"}' | jq .
```


Riesgos y Rollback

- **Riesgos:** +latencia puntual ante `n/a` (una llamada extra a `llm_docs-mcp`); riesgo de respuesta “pobre” si el LLM genera sin buenas fuentes. Mitigado porque solo corre cuando ya ibas a fallar.
- **Rollback:** revertir cambios en `orchestrator.py`:

```
git checkout -- mcp-core/orchestrator.py docker compose up -d --build mcp-core
```

Siguientes Pasos

- **Contrato con `llm_docs-mcp`:** reemplaza las **SUPOSICIONES** por los **nombres/args reales** del tool (p. ej., `doc-generar_respuesta_llm_with_sources` vs `doc-generar_respuesta_llm`), y estandariza el campo de salida (`sources/fragments`).
- **Enriquecer logging:** añade `trace_id` entrante al payload y propágalo al probe; loguea `route_selected`, `tool_called`, `hits`, `latency_ms`.
- **Umbral inteligente:** combina este parche con la mejora del clasificador (normalización de tildes) y, si es necesario, reduce el umbral de `n/a` o añade una regla léxica (“permiso|certificado|trámite|oficina|horario”) para forzar **probe**.
- **Tests E2E:** crea 10–20 casos representativos (trámites/FAQ) y valida `hit-rate` y tasa de `n/a` < 20%. Automatiza con `pytest` contra `/orchestrate`.
- **MMR y reformulación:** si `hits` =0, reintentar con consulta expandida (sinónimos) y/o MMR en `llm_docs-mcp` antes de rendirse; loguea qué estrategia salvó el caso.

Con este parche, aunque el clasificador deje consultas en `n/a`, el orquestador **intentará RAG antes de fallar**, recuperando respuestas útiles cuando la información exista

REINGENERÍA RAG

SISTEMA HÍBRIDO

Visión General de la Arquitectura Actual

Actualmente, MunBot utiliza un enfoque híbrido que combina flujos transaccionales predefinidos (para agenda de citas, registro de reclamos, etc.) con generación de respuestas a preguntas informativas mediante un modelo LLM local. En resumen:

- **Clasificación de intención:** Cada consulta del usuario se clasifica en categorías (`agenda`, `reclamo`, `tramite`, `documento`, `faq`, `saludo/despedida`, etc.) usando una combinación de reglas semánticas (alias, palabras clave) y un modelo LLM local como respaldo [GitHub](#) [GitHub](#). Este paso determina si se debe activar un flujo transaccional específico o un flujo de respuesta de documentos (RAG).
 - **Ejemplo:** Si la intención detectada es `"agenda"` o `"reclamo"`, el orquestador inicia el flujo correspondiente; si es `"faq"`, `"tramite"` o `"documento"`, se usa el flujo de consulta de documentos (RAG) [GitHub](#) [GitHub](#).
- **Flujos transaccionales (Agenda, Reclamo):** Son manejados actualmente por lógica específica en el orquestador (`orchestrator.py`). Por ejemplo, intenciones `"init_scheduler"/"cont_scheduler"` disparan el flujo de agendamiento paso a paso [GitHub](#), mientras que `"init_complaint"/"cont_complaint"` manejan el flujo de reclamos [GitHub](#) [GitHub](#). Estas rutinas guían al usuario solicitando datos requeridos (RUT, nombre, etc.) en varios turnos, usando funciones auxiliares para extraer entidades de la respuesta del usuario y rellenar `slots` [GitHub](#) [GitHub](#). Los textos de cada pregunta en estos flujos están predefinidos en el orquestador (p. ej. `FIELD_QUESTIONS` para cada campo) [GitHub](#).
- **Flujo informativo (RAG):** Cuando la intención es de tipo pregunta frecuente o trámite/documento, el orquestador delega al microservicio `llm_docs-mcp` para obtener una respuesta usando *Retrieval-Augmented Generation*. El orquestador invoca el endpoint `doc-generar_respuesta_llm` del microservicio [GitHub](#) [GitHub](#), pasando la pregunta y metadatos (p. ej. `tema_especifico`, `tramite`, etc. si los hubiese).

En `llm_docs-mcp`, el flujo RAG realiza: (1) embeddings de la pregunta, (2) búsqueda de vectores en Qdrant con filtros según hints, (3) retorna fragmentos relevantes, y (4) genera una respuesta usando el modelo Llama local con esos fragmentos como contexto [GitHub](#) [GitHub](#). Si la similitud es baja o no hay fragmentos, responde con un fallback (“No dispongo de esa información”) [GitHub](#). El código actual construye el prompt para Llama con una plantilla fija donde inserta el **CONTEXTO** y la **PREGUNTA** del usuario, pidiendo una respuesta basada únicamente en la información proporcionada [GitHub](#).

- **Smalltalk y respuestas directas:** Saludos, despedidas y agradecimientos se detectan por patrones regex y se responden con frases aleatorias predefinidas sin usar el modelo [GitHub](#) [GitHub](#). Asimismo, si el clasificador basado en reglas encuentra una respuesta exacta en la base de conocimiento (por alias), es posible responder directamente sin invocar el LLM. El orquestador ya contempla esto: si `llm_docs` devuelve un ítem exacto (campo `item` o `candidates`), lo entrega directamente [GitHub](#) [GitHub](#). En general, el diseño actual intenta usar respuestas almacenadas cuando hay correspondencia exacta, y solo recurre al LLM cuando es necesario.

En resumen, el sistema actual es **minimalista** y modular: el orquestador maneja la lógica de alto nivel y contextos de conversación, mientras que el microservicio `llm_docs-mcp` se encarga de la búsqueda semántica y generación local de respuestas. No se usa un

framework de agentes tipo LangChain; en su lugar, se implementaron endpoints específicos y funciones utilitarias para clasificación, búsqueda y generación.

Propuesta de Sistema Híbrido (Agente + Flujos)

El “sistema híbrido” propuesto consiste en **integrar un agente LLM capaz de decidir y orquestar llamadas a herramientas (microservicios)** en tiempo real, combinando así la flexibilidad del modelo con la confiabilidad de los flujos existentes. En la práctica, esto significa que el modelo LLM local podría evaluar la consulta del usuario y tomar una de las siguientes acciones:

- **Responder directamente** con información (usando RAG para consultas de conocimiento general, como ya hace).
- **Invocar una herramienta de flujo transaccional** (por ejemplo, llamar a una función para agendar una cita o registrar un reclamo) cuando detecte que la intención del usuario es realizar ese trámite.
- **Pedir aclaraciones o más información al usuario** si la consulta es ambigua o faltan datos necesarios (algo que un agente puede hacer de forma más dinámica, comparado al flujo fijo actual).

En esencia, se busca que el **modelo actúe como orquestador inteligente**, en lugar de depender totalmente de reglas estáticas para decidir el camino. Esto permitiría aplicar la misma interfaz conversacional a *todas* las interacciones (“que se aplique a todos”), desde preguntas informativas hasta solicitudes de trámites, sin necesidad de separar explícitamente la lógica desde el inicio. Importante: **los flujos transaccionales NO se eliminarían**, sino que el agente los utilizaría como subrutinas (herramientas). De esta forma se preserva su lógica de negocio y validaciones tal como están configurados, asegurando que no se rompa nada de lo ya implementado.

¿Es posible no borrar los flujos transaccionales tal como están? Sí. Podemos exponer las funcionalidades de esos flujos como funciones (APIs internas) que el agente llama cuando corresponda, manteniendo su implementación actual. De hecho, gran parte de esa lógica ya está encapsulada en funciones (*handlers*) dentro del orquestador [GitHub](#) [GitHub](#), así que se trata de “envolverlas” para que el modelo pueda usarlas. Veamos cómo lograr esto paso a paso.

Cambios Requeridos en el Código

Para implementar este sistema híbrido basado en un agente LLM, habrá que modificar e incorporar varias partes del código:

1. Extender `llm_docs-mcp` para soportar agentes y herramientas

Actualmente el endpoint principal `/tools/call` de `llm_docs-mcp` espera un formato sencillo: un JSON con campos `"tool"` y `"params"` para invocar directamente una herramienta específica [GitHub](#). No maneja entradas conversacionales complejas ni una lista de posibles herramientas (de hecho, si no se proporciona `"tool"`, devuelve error [GitHub](#)). En el intento actual de *probe* en el orquestador (`try_rag_probe`), se envía un payload con estructura de mensajes y funciones al endpoint, pero ese formato no está siendo procesado por el código [GitHub](#) [GitHub](#). En otras palabras, **falta implementar la lógica de agente** que interprete la conversación y decida la herramienta.

Qué hacer: modificar `/tools/call` para añadir un modo de funcionamiento tipo agente. Cuando reciba un JSON que contenga la clave `"messages"` (ej. mensajes de sistema/usuario) y opcionalmente una lista `"tools"` disponibles, debería seguir un flujo como:

- 1. Interpretar la petición de agente:** Si `req` tiene `"messages"`, extraer la conversación y la lista de herramientas. Por ejemplo:

```
req = await request.json()
if "messages" in req:
    messages = req["messages"]
    tools = req.get("tools", [])
    ## ... lógica de agente ...
else:
    ## comportamiento existente (llamada directa a tool)
```
- 2. Preparar prompt para el modelo:** Combinar los mensajes suministrados (roles `"system"`, `"user"`, etc.) en un formato adecuado para el LLM local. También informar al modelo sobre las herramientas disponibles. Dado que no usamos LangChain, podemos codificar manualmente una convención: por ejemplo, incluir en el *prompt* de sistema instrucciones explícitas del tipo “*Tienes disponibles las siguientes funciones: X, Y. Si la pregunta del usuario requiere usarlas, debes indicar un llamado en formato <CALL herramienta(param1=..., param2=...)>*” (u otro marcador fácilmente parseable). El objetivo es guiar al modelo a **emitir una señal de invocación** de herramienta cuando corresponda. Esto puede requerir experimentar con *prompts* o incluso ejemplos de formato (few-shot) para que el modelo entienda cómo responder.
- 3. Generar respuesta del modelo:** Usar `llama.generate()` para obtener la salida del LLM dado el prompt construido. Aquí es importante definir un límite de tokens y quizás usar un `stop` token especial si se define un formato de llamada. (Por ejemplo, podríamos usar un tag especial `[FUNC]` para delimitar la respuesta de función).
- 4. Interpretar la salida del modelo:** Analizar el texto generado para determinar si:
 - **Es una respuesta final al usuario** (el modelo no necesita herramienta) – en ese caso, simplemente retornar esa respuesta.
 - **Contiene una invocación de herramienta** – por ejemplo, si detectamos una marca `<CALL ...>` o algún patrón acordado:
 - Extraer el nombre de la herramienta y sus parámetros de la salida.
 - Verificar que la herramienta esté en la lista permitida (`tools`).
 - Llamar a la función correspondiente internamente (puede ser una llamada HTTP a otro microservicio o una invocación de función local).
 - Obtener el resultado de la herramienta (p. ej., la respuesta del microservicio de agenda o los fragmentos del RAG).

- Añadir el resultado como un mensaje de la *función* (role "assistant" con el resultado, siguiendo la idea de OpenAI Functions) y **volver a invocar al modelo** para que genere la respuesta final al usuario usando ese resultado.
 - *Ejemplo:* El modelo decide `<CALL doc-generar_respuesta_llm(pregunta="...")>`. El código de agente entonces ejecuta `generar_respuesta_llm` (ya implementado en `gateway.py`) con la pregunta dada, obtiene la respuesta y referencias [GitHub](#), y luego alimenta al LLM con un nuevo mensaje del tipo: `{"role": "function", "name": "doc-generar_respuesta_llm", "content": "...resultado JSON..."}` + `{"role": "user", "content": "<El usuario espera la respuesta final>"}`. Finalmente, el modelo generará una respuesta final que combina su lenguaje con los datos obtenidos (por ejemplo, citando las fuentes).
- Este ciclo se repite si la nueva respuesta del modelo vuelve a llamar otra herramienta (aunque idealmente, para RAG y flujos sencillos, solo habría una llamada).

5. **Retornar la respuesta final** al cliente (orquestador), incluyendo si corresponde las referencias o datos estructurados. En este punto, el `tools/call` actuaría como un coordinador flexible: según la complejidad de la consulta, puede haber llamado 0, 1 o varias herramientas, pero finalmente retorna un JSON con la respuesta lista para el usuario.

Implementar lo anterior **requiere trabajo extra** porque el modelo Llama local no está explícitamente entrenado para “function calling”. Se puede lograr con *prompt engineering* y reglas de parsing, pero habrá que iterar en pruebas para asegurar que entienda las instrucciones. En caso de dificultad, una alternativa es usar una versión del modelo afinada para seguir instrucciones estructuradas, o incluso (opcionalmente) delegar esta lógica a un modelo externo con `api` de OpenAI (que sí soporta nativamente funciones). Sin embargo, dado que deseamos mantener todo local, la prioridad sería intentar con Llama.cpp local.

Código a modificar/incluir: La sección en `gateway.py` del endpoint `/tools/call` deberá ampliarse. Actualmente solo maneja casos con "tool" explícito [GitHub](#) [GitHub](#). Se debería añadir la rama cuando llega "messages":

```
if "messages" in req:
    ## Nuevo: modo agente
    conv = req["messages"]
    tools_available = req.get("tools", [])
    ## (Pseudo-código) Lógica explicada arriba...
    ## 1. construir prompt para Llama con conv + descripción de tools_available
    ## 2. respuesta = llama.generate(prompt)
    ## 3. if respuesta indica función:
    ##     identificar func_name, params
    ##     ejecutar herramienta (puede reutilizar this function recursivamente o llamar internamente)
    ##     añadir resultado y volver a llamar llama.generate()
    ## 4. else: finalizar
    ## 5. retornar respuesta final (y datos como fuentes si vienen en formato estructurado)
```

Además, se puede aprovechar funciones auxiliares existentes. Por ejemplo, se puede reutilizar la lógica de `tools_call` para ejecutar la herramienta solicitada: internamente ya tenemos un mapa de `if tool == ... para doc-generar_respuesta_llm, doc-classify_intent_llm`, etc. [GitHub](#) [GitHub](#). Quizá convenga refactorizar esa lógica en funciones separadas para poder llamarlas directamente cuando el agente lo pida (en lugar de duplicar código).

En suma, el microservicio LLM debe evolucionar de un *simple prompt responder* a un *intérprete de instrucciones de alto nivel*. Esto **no elimina nada existente**; por el contrario, extiende sus capacidades. Los endpoints directos (`doc-generar_respuesta_llm`, etc.) seguirán allí para compatibilidad, pero el nuevo método agente los usará internamente.

2. Representar los flujos transaccionales como herramientas utilizables

Para que el agente LLM pueda invocar los flujos (“que se aplique a todos”), necesitamos exponer las acciones de **agenda de citas** y **registro de reclamos** como *funciones* en la lista de herramientas disponibles. Actualmente, esas funcionalidades residen en el orquestador y en microservicios externos (`scheduler-mcp`, `complaints-mcp`). Hay un inicio de esto en el código: por ejemplo, `TOOL_HANDLERS` en el orquestador define `"scheduler-appointment_create": _handle_scheduler_flow` [GitHub](#), insinuando que se planeaba usarlas como funciones. Sin embargo, hoy el LLM no las llama directamente; es el orquestador quien las invoca según la intención clasificada.

Qué hacer: definir una interfaz clara para que el agente pueda disparar estos flujos. Hay dos enfoques posibles:

- **Enfoque 1 (llamada directa al orquestador):** Crear endpoints en el orquestador que correspondan a “iniciar flujo de agenda” y “iniciar flujo de reclamo”. Podrían ser similares a `/orchestrate` pero especializados, donde se le pasa, por ejemplo, un JSON con todos los datos necesarios para agendar, o con un indicador de inicio. Sin embargo, dado que los flujos requieren interacción paso a paso (preguntar datos faltantes), no es trivial una única llamada. Probablemente seguiría siendo el orquestador quien conduzca varias rondas.
- **Enfoque 2 (manejo en el agente con varias llamadas):** Tratar cada paso del flujo como función separada. Por ejemplo:
 - Una función `schedule_create(date, time, name, ...)` que reserva la cita si recibe todos los campos, o devuelve un mensaje de qué falta si no.
 - Una función `get_next_appointment_question(contexto)` que, dada la conversación hasta ahora o los datos recopilados, devuelva la siguiente pregunta a hacer en el flujo.

Esto permitiría al modelo iterar: preguntar algo, recoger la respuesta del usuario en la conversación, y luego llamarse a sí mismo de nuevo para la siguiente pregunta. **Esta aproximación es compleja** y equivaldría a recrear la lógica del flujo dentro del modelo, lo que quizá no sea deseable dado que ya la tenemos implementada.

Una solución pragmática es optar por **un enfoque híbrido interno**: mantener el orquestador como está para flujos, pero **habilitar al agente a delegar al orquestador cuando identifique un caso de flujo**. Concretamente, se puede hacer que el agente LLM *no maneje él mismo todo el flujo*, sino que simplemente reconozca la intención y llame a una función representativa, por ejemplo `"init_scheduler()"` o `"init_complaint()"`. Esa “función” no haría más que notificar al sistema que se debe cambiar de modo al flujo existente. En la práctica, el agente podría devolverte algo como `{"intent": "agenda", "sub_intent": "iniciar"}` (similar a la salida actual de la clasificación) o invocar una pseudo-herramienta `"scheduler-init"`; entonces el orquestador (que recibe la respuesta del agente) sabrá que debe entrar al flujo agendamiento tradicional.

Implementación concreta recomendada:

Seguir usando la **clasificación actual como respaldo para los flujos transaccionales**, al menos inicialmente, por fiabilidad. Es decir, podemos permitir que el agente LLM intente decidir, pero si su decisión es incierta o su respuesta no es una llamada de herramienta clara, usamos la ruta existente. En la primera fase, podríamos **no eliminar** la lógica de `INTENT_MAP` y las condiciones en el orquestador [GitHub](#) [GitHub](#). En cambio, la modificaríamos así:

- El orquestador consulta al agente LLM (vía `llm_docs`) en lugar de o además de la función de clasificación actual. Por ejemplo, enviar la pregunta al nuevo endpoint `/tools/call` con *todas* las herramientas disponibles (`doc-generar_respuesta_llm`, `scheduler-appointment_create`, `complaint-register`, etc.).
- Si el resultado del agente indica una acción de agenda o reclamo, entonces el orquestador inicia el flujo correspondiente **como hasta ahora**. Podríamos hacer que el agente simplemente devuelva una intención categorizada (por ejemplo `"intent": "agenda"`), o una llamada específica que traduzcamos.
Ejemplo: Supongamos que el usuario dice *"Quiero sacar una cita para mañana"*. El agente podría devolverte: `{"tool": "scheduler-appointment_create", "params": {"fecha": "...", "hora": "...", ...}}` si logra extraer los datos. Pero es probable que no tenga todos los datos (quizá falta RUT). Entonces, podría devolver solo la intención. En tal caso, el orquestador al ver eso puede asignar `intent = init_scheduler` y entrar al flujo normal (preguntando lo que falta).
- Si el resultado del agente es una respuesta final (por ejemplo, contestó una pregunta tipo FAQ usando RAG), entonces simplemente se devuelve esa respuesta al usuario, fuera de cualquier flujo transaccional.
- En caso de incertidumbre o fallo del agente (por ejemplo, no está seguro si es un trámite o no entendió), **mantener fallback**: es decir, podemos seguir la estrategia actual de contar un fallo como `intent="n/a"` y luego intentar métodos alternativos. Con el agente en marcha, este fallback quizá se pueda simplificar, porque el agente mismo ya haría un intento de RAG para `n/a`. Sin embargo, en etapas iniciales es útil conservar `handle_fallback` para respuestas como *"Lo siento, no entendí"* [GitHub](#) [GitHub](#).

En resumen, **no borrar la lógica de flujos existente**, sino adaptarla para que funcione en conjunto con el nuevo agente. Al final, el orquestador seguirá teniendo secciones muy similares a las actuales para `"init_scheduler"/"cont_scheduler"` y `"init_complaint"/"cont_complaint"` [GitHub](#) [GitHub](#). La diferencia es que la decisión de entrar a esos flujos puede provenir de la salida del agente LLM en vez de solo de la función de clasificación por reglas.

3. Simplificar la clasificación de intenciones redundante

Con el agente en funcionamiento, parte de la lógica de clasificación actual podría volverse redundante. Por ejemplo, la función `classify_intent_with_llm` hace primero un matching sobre la base de conocimiento (`IntentEngine`) y luego usa LLM como fallback [GitHub](#) [GitHub](#). Si el agente LLM ya está buscando en la base de conocimiento para responder (vía RAG) **y** puede llamar funciones, entonces podríamos **eventualmente retirar la doble clasificación** (intent engine + LLM). En su lugar, el flujo sería: el agente recibe la entrada del usuario y decide en un solo paso qué hacer.

No obstante, es prudente hacerlo por fases. Inicialmente, conviene **mantener la clasificación actual** para asegurarse de no romper nada, pero envíarle esa información al agente como contexto para ayudarlo. Por ejemplo, podemos pasar como parte de los `messages` un mensaje del sistema con la “opinión” del clasificador de reglas: *“(Sistema: la consulta parece ser de tipo 'agenda')"*. Así el LLM tiene una sugerencia. Una vez probado el agente, si vemos que acierta bien las decisiones, podríamos eliminar la llamada a `IntentEngine` y confiar totalmente en el modelo.

Otro punto a reconsiderar: las respuestas de smalltalk predefinidas. Dado que el agente LLM puede manejar saludos/despedidas, podríamos dejar que él responda libremente a estos, o continuar con la vía rápida actual (regex + frase fija). Mantener el fastpath no afecta negativamente, solo que en un contexto totalmente controlado por el agente, esos mensajes podrían parecer un cambio de “voz” del bot. Una opción es incluir las variantes de saludo y despedida en la *prompt* del agente para que el propio modelo las utilice (por ejemplo, proveerle las frases posibles para que imite ese estilo). **Recomendación:** Conservar por ahora las respuestas estáticas de saludo/closing, pero evaluar integrarlas en el agente más adelante para consistencia.

En definitiva, módulos como `intent_engine.py` y partes de `intent_classifier.py` podrían simplificarse una vez el agente funcione. Por ejemplo, la función `try_rag_probe` del orquestador, que actualmente intenta una llamada fija a `generar_respuesta_llm` cuando `intent=="n/a"` [GitHub](#), se vuelve innecesaria porque el agente ya estaría haciendo esa búsqueda al intentar contestar. Podemos eliminar ese bloque o usarlo únicamente si tanto la clasificación como el agente fallan.

Resumen: *No borrar de inmediato* la clasificación basada en reglas ni los atajos smalltalk. Ir migrando gradualmente esas responsabilidades al agente y testear. Mientras tanto, algunos componentes serán respaldo (por ejemplo, se puede dejar `ENABLE_FASTPATH_SMALLTALK` activado [GitHub](#) hasta asegurar que el modelo maneja bien los saludos).

4. Manejo del estado y contexto de la conversación

El orquestador actual lleva un contexto de conversación en Redis, con `ConversationalContextManager`, para almacenar el historial y estado de flujos [GitHub](#) [GitHub](#). Con un agente más autónomo, **el historial cobra aún más importancia**, ya que el modelo puede

necesitar saber qué preguntó antes o si ya se proporcionó cierta información. Habrá que asegurarse de alimentar al agente con el historial pertinente en cada turno.

Concretamente: antes de llamar a `/tools/call` con el agente, el orquestador debe compilar los últimos mensajes relevantes. Quizá usar el mismo contexto que ya almacena (que guarda pares pregunta-respuesta recientes). Puede enviarlos en `req["messages"]` para que el agente los considere. Esto permite que, por ejemplo, si el usuario ya dio su nombre en un turno anterior, el modelo lo recuerde para no preguntarlo de nuevo al agendar.

El manejo de **finalización de sesión** también debe contemplarse. Hoy se hace manualmente: si un ítem de FAQ tiene tag “session_end”, se borra el contexto [GitHub](#). El agente podría encargarse de indicar cuando un flujo termina (por ejemplo, tras confirmar la cita, podría producir un mensaje final tipo "finish": True). Se puede seguir usando la señal actual (tag en metadata) o decidir una convención en la respuesta de la herramienta para indicarlo.

En suma, hay que **incluir el contexto conversacional en las llamadas al agente** y procesar sus efectos en el contexto igual que ahora. Los métodos del context manager (`set_current_flow`, `clear_context`, etc.) seguirán siendo útiles; se invocarán desde el orquestador según lo que dicte el agente o la herramienta.

Pasos de Configuración y Despliegue

Implementar lo anterior implica varios pasos concretos por parte del desarrollador (usted):

1. Actualizar el código del microservicio `llm_docs-mcp`:

- Añadir la lógica de **funciones/toolkit** en el endpoint `/tools/call` como se describió, para habilitar al agente. Esto incluye decidir el formato de invocación (p. ej. `<CALL ...>` o JSON explícito) y programar el *loop* de decisión del modelo.
- Incluir descripciones de las herramientas disponibles. Puede hacer esto estáticamente (un diccionario con nombre de herramienta -> descripción de qué hace y qué params espera) o incluso generar un schema JSON por herramienta. Como punto de partida, se puede listar: `"doc-generar_respuesta_llm"` (busca en la base de conocimiento y responde), `"scheduler-appointment_create"` (agenda una cita dados los datos) y `"complaint-register_user" / "complaint-registrar_reclamo"` para reclamos, entre otras. Documente estas descripciones en el prompt de sistema del agente.
- Probar la generación con ejemplos sencillos. Por ejemplo, enviar un mensaje de usuario "Necesito agendar una cita" y ver si el modelo responde con una llamada a la función de agenda. Ajustar el prompt hasta que logre resultados consistentes.

2. Exponer las funciones de flujo al agente:

Decidir cómo el agente invocará los flujos:

- Puede implementar en `llm_docs-mcp` funciones stub que llamen al orquestador. Por ejemplo, una herramienta `"scheduler-appointment_create"` podría hacer un `requests.post` al endpoint correspondiente de `scheduler-mcp` si existe, o al orquestador `/orchestrate` con intent forzado. Otra opción es que el *handler* de esta herramienta en `gateway.py` llame directamente a la función Python del orquestador (aunque por separación de microservicios no es tan limpio).
- Una vía simple es: cuando el agente llame `"scheduler-appointment_create"`, hacer que `tools_call` retorne un JSON indicando **transferencia de control** al orquestador. Por ejemplo: `{"handover": "scheduler"}`. El orquestador al recibir eso sabría que debe entrar en modo flujo scheduler. Esto se puede lograr devolviendo un campo especial en la respuesta del agente. En el orquestador, `handle_document_query` ya inspecciona la respuesta por campos como `item` o `candidates` [GitHub](#); de forma similar podría chequear si viene `"handover": "scheduler"` y entonces no formatear respuesta sino iniciar el flujo.
- Configurar credenciales o accesos:** si `llm_docs-mcp` va a llamar a otros servicios, asegúrese de que tenga la URL y credenciales necesarias (posiblemente vía variables de entorno como ya se hace para `QDRANT_HOST`, etc.). Por ejemplo, la URL interna del orquestador o del servicio de citas.

3. Modificar el orquestador (`orchestrator.py`):

- Cambiar la función `orchestrate()` para que, en lugar de usar `llm_client.classify_intent + handle_document_query` por separado, pueda consultar al nuevo agente. Esto podría hacerse sustituyendo la llamada actual a `classify_intent_remotely` [GitHub](#) por una llamada a un nuevo método `llm_client.agent_call(...)` que invoque `/tools/call` con la conversación y herramientas. Tendrá que construir la lista de herramientas a ofrecer (quizá configurable por entorno para poder ajustar sin cambiar código).
- Ajustar la lógica de decisión: la sección `if intent == "ask_document": ... elif intent in ["init_scheduler", ...]` puede transformarse, porque el agente devolverá directamente o una respuesta o quizás una instrucción. Una forma es: si el resultado de `agent_call` trae una clave `respuesta` (respuesta final para el usuario), entonces retornar eso de una vez. Si trae algo como `"tool": "scheduler-..."` o una marca de entregar control, entonces seguir por el flujo correspondiente.
- Mantener los bloques de smalltalk y fallback. Incluso con agente, se pueden conservar como salvaguarda: si por alguna razón el agente no devolvió nada usable, entrar a `handle_fallback()` como última instancia [GitHub](#) [GitHub](#).

4. Pruebas de regresión en flujos existentes:

- Verificar que pedir una cita o hacer un reclamo sigue funcionando correctamente. Idealmente, el agente detectará la intención y delegará al flujo sin cambios en la experiencia. Si nota que el agente intenta responder con texto libre en lugar de activar el flujo, será necesario ajustar el prompt de sistema para que aprenda a no “intentar resolver por sí mismo” esos casos y en vez de eso usar la herramienta. Por ejemplo, recalcar en las reglas: *“Si la consulta es para agendar cita, debes llamar la función correspondiente en vez de responder directamente”*.
- Testear preguntas informativas (FAQ, trámites) y comparar respuestas antes vs. después. Deben seguir siendo correctas; cualquier diferencia probablemente venga del estilo del modelo. Asegúrese de que sigue citando fuentes cuando corresponda. Aquí cabe mencionar: en la implementación actual, las fuentes se devuelven en un campo `referencias` [GitHub](#) [GitHub](#) que luego el orquestador no mostraba al usuario final (al menos en el código dado, solo las usa internamente). Con el agente, podría incluso incluirlas en la respuesta textual (`"... [fuente X] "`). Defina cómo manejar eso para mantener consistencia.

5. **Ajustes en el modelo LLM local:**

- Si el modelo local no sigue bien las instrucciones de formato, podría considerarse ajustar parámetros de inferencia (p. ej. temperatura baja para outputs más deterministas cuando deba llamar funciones). En `generate_response` se usan `temperatura=0.3/top_p=0.9` [GitHub](#); quizás para instrucciones estructuradas quiera algo aún más bajo (cercano a 0).
- Asegúrese de cargar el modelo con suficiente contexto (N_CTX). Actualmente N_CTX=2048 [GitHub](#). Si va a enviar varios mensajes de historial + descripciones de herramientas, vigile no superar ese límite.

6. **Despliegue y variables de entorno:**

- Añada cualquier variable de entorno nueva al `.env` correspondiente (por ejemplo, si creó `AGENT_MODE=1` para habilitar el agente condicionalmente, o URLs para servicios). Documente estas en el README para futuros mantenimientos.
- Actualice los contenedores (`docker-compose.yml`) si fuera necesario, aunque posiblemente los cambios son internos de código Python.

Tras implementar y configurar todo lo anterior, tendrá un **agente LLM híbrido** funcionando. En tiempo de ejecución, para el usuario final el cambio será sutil pero poderoso: podrá hacer preguntas libres y solicitudes en la misma conversación, y el bot sabrá cuándo responder directamente y cuándo encauzar un trámite, sin perder información en el camino. Todo esto **sin sacrificar los flujos transaccionales** actuales – más bien incorporándolos como componentes del agente.

Conclusiones

Implementar este sistema híbrido conlleva una refactorización moderada pero factible. No es necesario eliminar nada crítico (los flujos existentes permanecen); más bien se **añade una capa de inteligencia** sobre ellos. El enfoque propuesto mantiene la arquitectura minimalista (sin introducir dependencias externas significativas ni servicios en la nube, a menos que se desee) y aprovecha al máximo su infraestructura actual:

- El microservicio de documentos se vuelve más inteligente al manejar *decision making* además de RAG.
- El orquestador sigue orquestando, pero ahora con soporte de un LLM más poderoso en la toma de decisiones.
- Los microservicios de agenda y reclamos continúan brindando las funciones de negocio concretas, ahora accesibles vía el agente.

En definitiva, el bot resultante podrá **aplicarse a todos los tipos de consulta**. Cuando detecte un trámite, utilizará el flujo correspondiente tal cual está configurado; cuando reciba preguntas de información, hará búsquedas en la base de conocimientos y responderá; y en casos ambiguos, podrá interactuar para clarificar. Todo esto mantiene un marco controlado, evitando que el modelo “se salga del libreto” en operaciones críticas, ya que siempre que se requiera ejecutará las funciones aprobadas.

FASES

Fase 0 · Preparación y “feature flags”

Objetivo: aislar cambios y habilitar canary rollout.

Cambios en código/config

- Añadir flags en `.env` y lectura en código:
 - `AGENT_MODE=0|1` (híbrido activado)
 - `RAG_CATEGORY_AWARE=0|1` (RAG sensible a categoría)
 - `AGENT_MAX_TOOL_CALLS=2` (límite de invocaciones de herramienta)
 - `RAG_COLLECTION_FAQ` , `RAG_COLLECTION_TRAMITES` , `RAG_COLLECTION_NORMATIVA`
- Cargar flags en `orchestrator.py` y `services/llm_docs-mcp/gateway.py`.

Checklist

- ☐ Variables presentes en `.env.example`
- ☐ Lectura segura (valores por defecto)
- ☐ Log inicial de flags al boot

Criterios de aceptación

- Arranque sin cambios funcionales cuando los flags están en `0`.

Instrucciones

Paso 1 — Añadir variables al `.env.example`

Acción (aplicar patch):

```
--- a/.env.example
+++ b/.env.example
@@
+# === Feature flags (híbrido agente + RAG categoría) ===
+# Activa el modo agente en llm_docs-mcp/orchestrator (0=off, 1=on)
+AGENT_MODE=0
```

```

+# Hace que el RAG seleccione colección/filtro según la categoría (faq/tramite/documento)
+RAG_CATEGORY_AWARE=0
+# Límite de llamadas a herramientas por turno del agente (para acotar latencia)
+AGENT_MAX_TOOL_CALLS=2
+# Colecciones (o alias) por categoría (ajusta a tus nombres en Qdrant/índice)
+RAG_COLLECTION_FAQ=faq
+RAG_COLLECTION_TRAMITES=tramites
+RAG_COLLECTION_NORMATIVA=normativa

```

Notas:

- Los nombres `faq`, `tramites`, `normativa` son **placeholders**; ajusta a tus colecciones reales si difieren.
- Con `AGENT_MODE=0` y `RAG_CATEGORY_AWARE=0` no hay cambios de comportamiento.

Paso 2 — Cargar flags en `services/llm_docs-mcp/gateway.py`

Acción (insertar al inicio, tras imports existentes):

```

# === Feature flags / config ===
import os
import logging

def _getenv_int(name: str, default: int) -> int:
    try:
        return int(os.getenv(name, str(default)).strip())
    except Exception:
        return default

def _getenv_str(name: str, default: str) -> str:
    val = os.getenv(name)
    return val.strip() if isinstance(val, str) and val.strip() else default

AGENT_MODE          = _getenv_int("AGENT_MODE", 0)
RAG_CATEGORY_AWARE  = _getenv_int("RAG_CATEGORY_AWARE", 0)
AGENT_MAX_TOOL_CALLS = _getenv_int("AGENT_MAX_TOOL_CALLS", 2)
RAG_COLLECTION_FAQ   = _getenv_str("RAG_COLLECTION_FAQ", "faq")
RAG_COLLECTION_TRAMITES = _getenv_str("RAG_COLLECTION_TRAMITES", "tramites")
RAG_COLLECTION_NORMATIVA = _getenv_str("RAG_COLLECTION_NORMATIVA", "normativa")

_logger = logging.getLogger("llm_docs_mcp")

```

Acción (log de flags al boot — agrega cerca de donde se crea la app FastAPI):

```

from fastapi import FastAPI
app = FastAPI()

@app.on_event("startup")
async def _log_feature_flags():
    _logger.info(
        "FeatureFlags llm_docs-mcp | AGENT_MODE=%s RAG_CATEGORY_AWARE=%s "
        "AGENT_MAX_TOOL_CALLS=%s RAG_COLLECTIONS={faq:%s, tramite:%s, doc:%s}",
        AGENT_MODE, RAG_CATEGORY_AWARE, AGENT_MAX_TOOL_CALLS,
        RAG_COLLECTION_FAQ, RAG_COLLECTION_TRAMITES, RAG_COLLECTION_NORMATIVA
    )

```

Resultado esperado:

- El servicio arranca y registra los valores actuales de flags.
- No cambia ningún flujo mientras `AGENT_MODE=0` y `RAG_CATEGORY_AWARE=0`.

Paso 3 — Cargar flags en `mcp-core/orchestrator.py`

Acción (insertar al inicio, tras imports existentes):

```

# === Feature flags / config ===
import os
import logging

def _getenv_int(name: str, default: int) -> int:
    try:
        return int(os.getenv(name, str(default)).strip())
    except Exception:
        return default

```

```
def _getenv_str(name: str, default: str) -> str:
    val = os.getenv(name)
    return val.strip() if isinstance(val, str) and val.strip() else default

AGENT_MODE = _getenv_int("AGENT_MODE", 0)
RAG_CATEGORY_AWARE = _getenv_int("RAG_CATEGORY_AWARE", 0)
AGENT_MAX_TOOL_CALLS = _getenv_int("AGENT_MAX_TOOL_CALLS", 2)
RAG_COLLECTION_FAQ = _getenv_str("RAG_COLLECTION_FAQ", "faq")
RAG_COLLECTION_TRAMITES = _getenv_str("RAG_COLLECTION_TRAMITES", "tramites")
RAG_COLLECTION_NORMATIVA = _getenv_str("RAG_COLLECTION_NORMATIVA", "normativa")

_logger = logging.getLogger("orchestrator")
_logger.info(
    "FeatureFlags orchestrator | AGENT_MODE=%s RAG_CATEGORY_AWARE=%s "
    "AGENT_MAX_TOOL_CALLS=%s RAG_COLLECTIONS={faq:%s, tramite:%s, doc:%s}",
    AGENT_MODE, RAG_CATEGORY_AWARE, AGENT_MAX_TOOL_CALLS,
    RAG_COLLECTION_FAQ, RAG_COLLECTION_TRAMITES, RAG_COLLECTION_NORMATIVA
)
```

Resultado esperado:

- El orquestador también conoce y reporta los flags, permitiendo políticas condicionales en fases siguientes.
- Sin cambios funcionales con flags en 0.

Paso 4 — Robustecer logging (opcional pero recomendado)

Acción (si no configuras logging en uvicorn/gunicorn):

Añade una configuración mínima donde centralices logs a stdout con nivel INFO (en los entrypoints o __main__):

```
import logging, sys
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s :: %(message)s",
    handlers=[logging.StreamHandler(sys.stdout)],
)
```

Paso 5 — Verificación rápida (checklist)

- ☐ Variables nuevas existen en .env.example.
- ☐ Servicios arrancan sin error con flags por defecto (0).
- ☐ Logs de arranque muestran los flags y colecciones.
- ☐ No hay cambios de comportamiento con flags en 0.

Comandos sugeridos (si usas docker-compose):

```
# 1) Rebuild y arranque
docker compose build
docker compose up -d

# 2) Revisar logs de llm_docs-mcp
docker compose logs -f llm_docs-mcp | sed -n '1,120p'

# 3) Revisar logs de orquestador
docker compose logs -f mcp-core | sed -n '1,120p'
```

Criterios de aceptación (automáticos)

- Con AGENT_MODE=0 y RAG_CATEGORY_AWARE=0, peticiones típicas (FAQ/trámite/documento/agenda/reclamo) devuelven **exactamente** lo mismo que antes.
- Los servicios reportan en log:
FeatureFlags llm_docs-mcp | AGENT_MODE=0 RAG_CATEGORY_AWARE=0 ...
FeatureFlags orchestrator | AGENT_MODE=0 RAG_CATEGORY_AWARE=0 ...

Observaciones finales

- Esta fase **solo introduce configuración y visibilidad**.
- En fases siguientes, los flags controlarán **cuándo** usar el **modo agente** y **cómo** activar el **RAG categoría**.
- Si tus colecciones reales difieren, **ajústalas ahora** en .env para evitar sorpresas cuando actives RAG_CATEGORY_AWARE=1.

Fase 1 · Normalización de intención + propagación de categoría

Objetivo: eliminar falsos `n/a` por tildes/puntuación y **preservar** `faq|tramite|documento` hasta RAG/Agente.

Cambios en código

- `mcp-core/orchestrator.py`:
 - Función `_norm_intent()` (lowercase, strip, remover diacríticos/puntuación).
 - Singular simple: si termina en `s` y existe la versión singular en el mapa, usarla.
 - Mantener `INTENT_MAP` actual, pero pasar `intent_type=k` a `handle_document_query(...)`.
- `services/llm_docs-mcp/gateway.py`:
 - En `doc-generar_respuesta_llm`, aceptar `categoria` y loguearla.

Checklist

- ☒ `_norm_intent()` aplicada antes de consultar el `INTENT_MAP`
- ☐ `handle_document_query(..., intent_type=k)`
- ☐ Logs: `intent_raw`, `intent_norm`, `categoria`

Criterios de aceptación

- Pruebas con “Trámite.”, “documentos”, “FAQ?” no terminan en `n/a`.
- `categoria` llega al microservicio.

Instrucciones

Objetivo: suprimir falsos `n/a` por variaciones ortográficas y **preservar** la señal `faq|tramite|documento` hasta el RAG/Agente (sin cambiar aún la selección de colecciones).

Paso 1 — `orchestrator.py` : añadir normalización de etiqueta

Acción (aplicar patch en la sección de imports y utilidades):

```
--- a/mcp-core/orchestrator.py
+++ b/mcp-core/orchestrator.py
@@
+import unicodedata
+import re
+import logging
+
+_log = logging.getLogger("orchestrator")
+
+# --- Normalización robusta de etiquetas de intención ---
+def _norm_intent(label: str) -> str:
+    """
+    Normaliza etiquetas de intención:
+    - lower()
+    - strip()
+    - remueve diacríticos
+    - remueve puntuación/símbolos (conserva letras/números y '/')
+    """
+    if not label:
+        return ""
+    s = unicodedata.normalize("NFKD", label)
+    s = s.encode("ascii", "ignore").decode("ascii")
+    s = s.strip().lower()
+    s = re.sub(r"[\^\w/]+", "", s) # permite 'n/a' y rutas tipo 'x/y'
+    return s
```

Paso 2 — `orchestrator.py` : aplicar normalización antes de consultar el `INTENT_MAP`

Inserta esto justo donde hoy se toma la etiqueta del clasificador y se mapea a la acción (antes de `INTENT_MAP.get(...)`).

Acción (aplicar patch):

```
--- a/mcp-core/orchestrator.py
+++ b/mcp-core/orchestrator.py
@@
-raw_intent = classification.get("intent", "") if classification else ""
-intent = INTENT_MAP.get(raw_intent, "n/a")
+raw_intent = (classification.get("intent") if classification else "") or ""
+# 1) normalizar
+norm_intent = _norm_intent(raw_intent)
+# 2) singular simple (tramites -> tramite; documentos -> documento; faqs -> faq)
```

```
+if norm_intent not in INTENT_MAP and norm_intent.endswith("s"):
+    singular = norm_intent[:-1]
+    if singular in INTENT_MAP:
+        norm_intent = singular
+# 3) mapear
+intent = INTENT_MAP.get(norm_intent, "n/a")
+# 4) logging de trazabilidad
+_log.info("intent_raw=%s intent_norm=%s mapped_action=%s", raw_intent, norm_intent, intent)
```

Checklist parcial

- `_norm_intent()` aplicada antes de `INTENT_MAP`.
- Singular simple contemplado.
- Log con `intent_raw` y `intent_norm`.

Paso 3 — `orchestrator.py`: preservar la categoría y pasarla al handler informativo

Mantén el `INTENT_MAP` actual (que agrupa `faq|documento|tramite` en `ask_document`). **Además**, pasa la etiqueta normalizada `norm_intent` al handler.

Acción (aplicar patch en el bloque que maneja `ask_document`):

```
--- a/mcp-core/orchestrator.py
+++ b/mcp-core/orchestrator.py
@@
-elif intent == "ask_document":
-    return handle_document_query(session_id, user_text, entidades, contexto)
+elif intent == "ask_document":
+    # Propagamos la categoría original normalizada (faq|tramite|documento)
+    return handle_document_query(session_id, user_text, entidades, contexto, intent_type=norm_intent)
```

Si tu `handle_document_query(...)` aún **no** acepta `intent_type`, ajusta su firma:

```
--- a/mcp-core/orchestrator.py
+++ b/mcp-core/orchestrator.py
@@
-def handle_document_query(session_id, user_text, entidades, contexto):
+def handle_document_query(session_id, user_text, entidades, contexto, intent_type: str | None = None):
    """
    Genera respuesta informativa vía llm_docs-mcp (RAG).
    """
    # ...
```

Paso 4 — `orchestrator.py`: pasar categoría en la llamada HTTP al microservicio

Dentro de `handle_document_query(...)`, donde hoy construyes el payload para `llm_docs-mcp`, agrega `categoria=intent_type` si viene.

Acción (aplicar patch en el cuerpo de `handle_document_query`):

```
--- a/mcp-core/orchestrator.py
+++ b/mcp-core/orchestrator.py
@@
- payload = {
-     "tool": "doc-generar_respuesta_llm",
-     "params": {
-         "query": user_text,
-         "top_k": 5
-     }
- }
+ params = {
+     "query": user_text,
+     "top_k": 5
+ }
+ if intent_type in ("faq", "tramite", "documento"):
+     params["categoria"] = intent_type
+     _log.info("propagate_categoria=%s", intent_type)
+ payload = {"tool": "doc-generar_respuesta_llm", "params": params}

# llamada HTTP como antes...
resp = llm_client.tools_call(payload)
```

Checklist parcial

- `handle_document_query(..., intent_type=k)`
- Log de `categoria` propagada

Paso 5 — `gateway.py` : aceptar y loguear `categoria` (sin cambiar lógica aún)

Esta fase **solo** acepta el nuevo parámetro y lo registra; la selección de colección/filtro por categoría vendrá en la Fase 2.

Acción (aplicar patch en el branch del `tool == "doc-generar_respuesta_llm"`):

```
--- a/services/llm_docs-mcp/gateway.py
+++ b/services/llm_docs-mcp/gateway.py
@@
-elif tool == "doc-generar_respuesta_llm":
-    query = params.get("query", "").strip()
-    top_k = int(params.get("top_k", 5))
-    return await generar_respuesta_llm(query, top_k=top_k)
+elif tool == "doc-generar_respuesta_llm":
+    query = (params.get("query") or "").strip()
+    top_k = int(params.get("top_k", 5))
+    categoria = params.get("categoria") # puede ser None en Fase 1
+    _logger.info("doc-generar_respuesta_llm: categoria=%s top_k=%s", categoria, top_k)
+    # Fase 1: aún no usamos 'categoria' para filtrar colecciones; solo se loguea.
+    return await generar_respuesta_llm(query, top_k=top_k)
```

Asegúrate de tener `_logger` definido (Fase 0 ya lo introduce).

Checklist final

- Logs en `gateway.py` muestran `categoria` al invocar `doc-generar_respuesta_llm`.

Paso 6 — Verificación (pruebas rápidas)

Comandos sugeridos (ajusta host/puerto según tu entorno):

```
# 1) Caso con tilde y puntuación
curl -s http://localhost:5000/orchestrate \
  -H 'Content-Type: application/json' \
  -d '{"pregunta":"Trámite."}' | jq .

# 2) Plural
curl -s http://localhost:5000/orchestrate \
  -H 'Content-Type: application/json' \
  -d '{"pregunta":"¿Qué documentos necesito?"}' | jq .

# 3) Mayúsculas/puntuación FAQ
curl -s http://localhost:5000/orchestrate \
  -H 'Content-Type: application/json' \
  -d '{"pregunta":"FAQ?"}' | jq .
```

Qué observar en logs:

- En **orquestador**:
 - `intent_raw=...`, `intent_norm=tramite|documento|faq`, `mapped_action=ask_document`
 - `propagate_categoria=tramite|documento|faq` cuando aplica.
- En **llm_docs-mcp**:
 - `doc-generar_respuesta_llm: categoria=tramite|documento|faq top_k=5`

Criterios de aceptación

- Las tres pruebas de ejemplo **no** terminan en `intent: "n/a"`.
- Se ve `categoria` registrada por `llm_docs-mcp`.
- Con `RAG_CATEGORY_AWARE=0` el comportamiento final (contenido) **no cambia** respecto a antes; solo trazabilidad extra.

Observaciones y buenas prácticas

- Esta fase **no cambia** la lógica de recuperación; únicamente **limpia la etiqueta** y **preserva** la categoría hacia el microservicio.
- Si tu orquestador tiene tests, añade casos para: `"Trámite."`, `"documentos"`, `"FAQ?"` verificando que `intent_norm` sea el esperado.
- Si en tu código el `INTENT_MAP` ya contiene claves con tildes, revisa y unifica a **sin tildes**; la normalización lo asume.
- Deja `RAG_CATEGORY_AWARE=0` hasta completar la Fase 2 (donde sí filtrarás por colección/filtro y prompts por tipo).

Fase 2 · RAG “category-aware” (colecciones/filtros + prompts)

Objetivo: mejorar precisión top-k en RAG usando la categoría.

Cambios en código

- services/llm_docs-mcp/gateway.py :
 - Selección de **colección** o **filtro** por categoria :
 - tramite → RAG_COLLECTION_TRAMITES o filter tipo=tramite
 - documento → RAG_COLLECTION_NORMATIVA o filter tipo=documento
 - faq → RAG_COLLECTION_FAQ o filter tipo=faq
 - Tres plantillas de prompt:
 - PROMPT_TRAMITE : “Requisitos / Pasos / Documentos / Plazos (+fuente)”
 - PROMPT_DOCUMENTO : “Resumen normativo + Artículo/Cláusula + Vigencia + Referencia”
 - PROMPT_FAQ : “Respuesta breve, clara, con 1–2 fuentes”
- Migración opcional de metadatos: garantizar tipo en índice o separar colecciones.

Checklist

- ☐ categoria mapea a colección/filtro
- ☐ Prompts específicos aplicados
- ☐ Logs: colección/filtro elegidos, top_k, scores

Criterios de aceptación

- Queries de cada categoría recuperan documentos esperables (smoke test por categoría).
- Latencia no empeora significativamente.

Instrucciones

Objetivo: hacer que llm_docs-mcp use la categoría (faq|tramite|documento) para elegir colección/filtro y prompt de salida, mejorando la precisión del top-k.
Supone que **Fase 1** ya está pasando categoria desde orchestrator.py.

Paso 1 — Variables de entorno para selección por colección o filtro

Acción (aplicar patch en .env.example):

```
--- a/.env.example
+++ b/.env.example
@@
# === Feature flags (híbrido agente + RAG categoría) ===
AGENT_MODE=0
RAG_CATEGORY_AWARE=0
AGENT_MAX_TOOL_CALLS=2
RAG_COLLECTION_FAQ=faq
RAG_COLLECTION_TRAMITES=tramites
RAG_COLLECTION_NORMATIVA=normativa
+#
+# Modo de selección de RAG por categoría:
+# - collection: usa colecciones separadas en Qdrant/índice
+# - filter      : usa una sola colección + filtro por metadato (ej. tipo=faq)
+RAG_SELECTION_MODE=collection
+# Si RAG_SELECTION_MODE=filter, indica el nombre del metacampo (ej. "tipo")
+RAG_FILTER_FIELD=tipo
```

Puedes dejar RAG_SELECTION_MODE=collection si ya indexaste por colecciones separadas.
Si usas **una sola colección** con metadatos, cambia a filter y define RAG_FILTER_FIELD.

Paso 2 — Cargar las nuevas variables en gateway.py

Acción (insertar cerca de donde cargaste flags en Fase 0):

```
--- a/services/llm_docs-mcp/gateway.py
+++ b/services/llm_docs-mcp/gateway.py
@@
RAG_COLLECTION_FAQ      = _getenv_str("RAG_COLLECTION_FAQ", "faq")
RAG_COLLECTION_TRAMITES = _getenv_str("RAG_COLLECTION_TRAMITES", "tramites")
RAG_COLLECTION_NORMATIVA = _getenv_str("RAG_COLLECTION_NORMATIVA", "normativa")
+RAG_SELECTION_MODE      = _getenv_str("RAG_SELECTION_MODE", "collection") # collection|filter
+RAG_FILTER_FIELD        = _getenv_str("RAG_FILTER_FIELD", "tipo")
```

Log al arranque (completa lo que ya tienes):


```
@app.on_event("startup")
async def _log_feature_flags():
    _logger.info(
        - "FeatureFlags llm_docs-mcp | AGENT_MODE=%s RAG_CATEGORY_AWARE=%s "
        - "AGENT_MAX_TOOL_CALLS=%s RAG_COLLECTIONS={faq:%s, tramite:%s, doc:%s}",
        - AGENT_MODE, RAG_CATEGORY_AWARE, AGENT_MAX_TOOL_CALLS,
        - RAG_COLLECTION_FAQ, RAG_COLLECTION_TRAMITES, RAG_COLLECTION_NORMATIVA
        + "FeatureFlags llm_docs-mcp | AGENT_MODE=%s RAG_CATEGORY_AWARE=%s "
        + "AGENT_MAX_TOOL_CALLS=%s RAG_SELECTION_MODE=%s RAG_FILTER_FIELD=%s "
        + "RAG_COLLECTIONS={faq:%s, tramite:%s, doc:%s}",
        + AGENT_MODE, RAG_CATEGORY_AWARE, AGENT_MAX_TOOL_CALLS,
        + RAG_SELECTION_MODE, RAG_FILTER_FIELD,
        + RAG_COLLECTION_FAQ, RAG_COLLECTION_TRAMITES, RAG_COLLECTION_NORMATIVA
    )
```

Paso 3 — Definir prompts específicos por categoría

Acción (añadir constantes en gateway.py , sección superior):

```
# === Plantillas de prompt por categoría (Fase 2) ===
PROMPT_FAQ = """Responde de forma breve y clara, en español, usando EXCLUSIVAMENTE la información del CONTEXTO.
Incluye 1-2 fuentes si están disponibles.

CONTEXTO:
{contexto}

PREGUNTA:
{pregunta}

RESPUESTA: """

PROMPT_TRAMITE = """Responde en español, estructurando la salida con estos apartados:
- Requisitos:
- Pasos:
- Documentos:
- Plazos:
Usa EXCLUSIVAMENTE el CONTEXTO y cita al final 1-2 fuentes si están disponibles.

CONTEXTO:
{contexto}

PREGUNTA:
{pregunta}

RESPUESTA: """

PROMPT_DOCUMENTO = """Genera una respuesta en español, con foco normativo:
- Resumen normativo
- Artículo/Cláusula relevante
- Vigencia (si aplica)
- Referencia/Fuente
Usa EXCLUSIVAMENTE el CONTEXTO.

CONTEXTO:
{contexto}

PREGUNTA:
{pregunta}

RESPUESTA: """
```

Paso 4 — Helper para mapear categoría → (colección/filtro, prompt)

Acción (añadir en gateway.py):

```
VALID_CATS = {"faq", "tramite", "documento"}

def _select_collection_and_prompt(categoria: str | None):
    """
    Devuelve (collection, filtro_dict, prompt_template)
    según RAG_SELECTION_MODE y categoria.
    """
    cat = (categoria or "").strip().lower()
    if cat not in VALID_CATS:
        # fallback por defecto (FAQ)
        return (RAG_COLLECTION_FAQ, None, PROMPT_FAQ) if RAG_SELECTION_MODE == "collection" \
```

```

        else (None, {RAG_FILTER_FIELD: "faq"}, PROMPT_FAQ)

if RAG_SELECTION_MODE == "collection":
    if cat == "tramite":
        return (RAG_COLLECTION_TRAMITES, None, PROMPT_TRAMITE)
    if cat == "documento":
        return (RAG_COLLECTION_NORMATIVA, None, PROMPT_DOCUMENTO)
    return (RAG_COLLECTION_FAQ, None, PROMPT_FAQ)
# modo filter
if cat == "tramite":
    return (None, {RAG_FILTER_FIELD: "tramite"}, PROMPT_TRAMITE)
if cat == "documento":
    return (None, {RAG_FILTER_FIELD: "documento"}, PROMPT_DOCUMENTO)
return (None, {RAG_FILTER_FIELD: "faq"}, PROMPT_FAQ)

```

Paso 5 — Adaptar generar_respuesta_llm para usar categoría

- Mantén **retrocompatibilidad**: si RAG_CATEGORY_AWARE=0, se comporta como antes.
- Si 1 y viene categoria, elige colección/filtro y prompt.

Acción (modificar firma y cuerpo):

```

--- a/services/llm_docs-mcp/gateway.py
+++ b/services/llm_docs-mcp/gateway.py
@@
-async def generar_respuesta_llm(query: str, top_k: int = 5):
+async def generar_respuesta_llm(query: str, top_k: int = 5, categoria: str | None = None):
    """
    - Recupera fragmentos y genera respuesta con el LLM (modo legacy).
    + Recupera fragmentos y genera respuesta con el LLM.
    + Si RAG_CATEGORY_AWARE=1 y se pasa 'categoria', selecciona colección/filtro y prompt específicos.
    """

    - # 1) recuperar contexto (legacy)
    - chunks = await retrieve_context(query, top_k=top_k)
    - contexto = build_context_text(chunks)
    - prompt = DEFAULT_PROMPT.format(contexto=contexto, pregunta=query)
    + collection = None
    + filtro = None
    + prompt_tpl = None
    + if RAG_CATEGORY_AWARE and categoria:
    +     collection, filtro, prompt_tpl = _select_collection_and_prompt(categoria)
    +     _logger.info("RAG category-aware ON | categoria=%s mode=%s collection=%s filtro=%s",
    +                 categoria, RAG_SELECTION_MODE, collection, filtro)
    + else:
    +     _logger.info("RAG category-aware OFF | usando configuración legacy")
    +
    + # 1) recuperar contexto
    + # Intento por colección/filtro si está activo; si falla o retorna vacío, fallback a legacy.
    + chunks = []
    + try:
    +     if RAG_CATEGORY_AWARE and (collection or filtro):
    +         chunks = await retrieve_context(query, top_k=top_k, collection=collection, filtro=filtro)
    + except Exception as e:
    +     _logger.warning("retrieve_context con categoría falló: %s", e)
    +     chunks = []
    + if not chunks:
    +     # Legacy sin categoría (compatibilidad)
    +     chunks = await retrieve_context(query, top_k=top_k)
    +
    + # 2) construir prompt
    + contexto = build_context_text(chunks)
    + if not prompt_tpl:
    +     prompt_tpl = PROMPT_FAQ # fallback razonable
    + prompt = prompt_tpl.format(contexto=contexto, pregunta=query)

    # 3) generar con LLM (igual que antes)
    answer = await generate_response(prompt)

    # 4) armar referencias y logging
    - refs = build_references(chunks)
    - _logger.info("rag: top_k=%s hits=%s", top_k, len(chunks))
    + refs = build_references(chunks)
    + top1 = chunks[0]["score"] if chunks and "score" in chunks[0] else None
    + _logger.info("rag: top_k=%s hits=%s top1=%s collection=%s filtro=%s",
    +             top_k, len(chunks), top1, collection, filtro)
    return {"respuesta": answer, "referencias": refs}

```

Notas importantes:>

- Se asume que tu `retrieve_context(...)` acepta `collection` y `filtro`; si no, crea un wrapper que los ignore sin romper.>
- `build_context_text`, `build_references`, `generate_response` ya existen en tu código. No cambies sus contratos.

Paso 6 — Usar `categoria` en el handler del tool

Completa lo que dejaste en Fase 1 para pasar `categoria` a `generar_respuesta_llm`.

Acción (donde procesas `"doc-generar_respuesta_llm"`):

```
- return await generar_respuesta_llm(query, top_k=top_k)
+ return await generar_respuesta_llm(query, top_k=top_k, categoria=categoria)
```

Paso 7 — Logging de scores/colección/filtro

Ya incluido en el diff (Paso 5). Si quieres más detalle, loguea cada chunk.

Acción (opcional):

```
for i,c in enumerate(chunks[:min(len(chunks), top_k)]):
    _logger.debug("hit#%d score=%s id=%s meta=%s", i+1, c.get("score"), c.get("id"), c.get("metadata"))
```

Paso 8 — Verificación (smoke tests por categoría)

Pruebas (con `RAG_CATEGORY_AWARE=1`):

```
# tramite
curl -s http://localhost:5000/orchestrate \
-H 'Content-Type: application/json' \
-d '{"pregunta":"¿Qué necesito para obtener el permiso de circulación?"}' | jq .

# documento (normativa)
curl -s http://localhost:5000/orchestrate \
-H 'Content-Type: application/json' \
-d '{"pregunta":"¿Qué dice la ordenanza municipal sobre ruidos molestos?"}' | jq .

# faq
curl -s http://localhost:5000/orchestrate \
-H 'Content-Type: application/json' \
-d '{"pregunta":"¿Cuál es el horario de atención del municipio?"}' | jq .
```

Qué observar en logs de `llm_docs-mcp`:

- Línea de **arranque** con `RAG_SELECTION_MODE` y `RAG_FILTER_FIELD`.
- Para cada query:
 - `RAG category-aware ON | categoria=... mode=collection|filter collection=... filtro=...`
 - `rag: top_k=... hits=... top1=... collection=... filtro=...`
 - (opcional) `hit#i score=... meta=...`

Criterios de aceptación

- Cada categoría recupera documentos esperables (inspecciona referencias o debug logs).
- **Latencia** similar a antes ($\pm 10\text{--}15\%$ aceptable). Si sube, reduce `top_k` o revisa filtros.

Paso 9 — (Opcional) Migración de metadatos `tipo`

Solo si usas `RAG_SELECTION_MODE=filter`.

Acción: script rápido de backfill (pseudo-ETL):

```
# scripts/backfill_tipo.py (ejemplo, adáptalo a tu índice/Qdrant cliente)
from qdrant_client import QdrantClient
qc = QdrantClient(host="qdrant", port=6333)

def backfill(collection, filtro_por_ruta):
    # filtro_por_ruta: dict con {prefix_ruta: "tipo"}
    # escanea puntos y setea payload["tipo"]=...
    pass
```

- Asegúrate de que **todos** los payloads tengan `tipo` $\in$ `{faq, tramite, documento}`.
- Si no es viable el backfill, cambia a `RAG_SELECTION_MODE=collection` y separa por colecciones.

Checklist (Fase 2)

- `categoria` mapea a colección **o** filtro (según `RAG_SELECTION_MODE`).
- Prompts específicos (`FAQ/TRÁMITE/DOCUMENTO`) aplicados.
- Logs con colección/filtro elegidos, `top_k`, y `score` del top-1.

Observaciones finales

- Esta fase **no altera** el orquestador (más allá de lo hecho en Fase 1).
- El control está **feature-flagged**: con `RAG_CATEGORY_AWARE=0` todo se comporta como antes.
- Si el retrieval por categoría falla o no retorna hits, el código **falla a legacy** automáticamente.
- En la **Fase 3** usarás esta `categoria` también dentro del **modo agente** para que elija `rag.query` con el contexto adecuado.

Fase 3 · Agente en `llm_docs-mcp` (modo mensajes+herramientas)

Objetivo: habilitar un **agente** sin frameworks externos.

Cambios en código

- `services/llm_docs-mcp/gateway.py`:
 - Extender `/tools/call`:
 - Si `req` contiene `"messages"` → **modo agente**.
 - Registrar `tools_available` (lista de herramientas permitidas).
 - **Tool Registry** interno (dict nombre→handler + schema parámetros).
 - **Mini-DSL** de llamada de función:
 - Formato simple y parseable: `<CALL tool_name param1="..." param2="...">`
 - Alternativa: JSON inline de `{ "call": { "tool": "...", "params": {...} } }`
 - **Prompt del agente (role=system)**:
 - Lista de herramientas con nombre, propósito y parámetros requeridos.
 - Reglas: “si acción transaccional → llama herramienta; si info → llama `rag.query` (doc-generar_respuesta_llm) con `categoria` si está; pide aclaración si faltan campos obligatorios; no inventes valores; máximo `AGENT_MAX_TOOL_CALLS` .”
 - **Bucle de decisión**:
 - Generar → detectar `<CALL ...>` → validar herramienta → ejecutar handler → inyectar resultado como mensaje → regenerar → emitir respuesta final.
 - **Salvaguardas**:
 - Límite de pasos, timeouts, validación de tipos, sanitización de strings.

Checklist

- ☐ Rama `"messages"` implementada
- ☐ Registro de herramientas y validación de params
- ☐ Parser robusto de `<CALL ...>`
- ☐ Límite `AGENT_MAX_TOOL_CALLS` respetado
- ☐ Logs: decisiones del agente, herramientas invocadas, duración

Criterios de aceptación

- Prompt “Quiero denunciar un bache” → `<CALL complaint-init>` o handover a reclamos.
- Prompt “Horario de atención” → `<CALL doc-generar_respuesta_llm ...>` y respuesta con fuente.

Instrucciones

Objetivo: que `llm_docs-mcp` pueda **leer mensajes**, **decidir** qué herramienta invocar (RAG o handover a flujos) y **devolver** un resultado final u orden de traspaso, **sin frameworks externos** y **sin romper el modo legacy** `tool+params` .

Precondiciones>

- Fase 0: `AGENT_MODE` , `AGENT_MAX_TOOL_CALLS` ya cargados y logueados.>
- Fase 1/2: `generar_respuesta_llm(query, top_k, categoria)` disponible.

Paso 1 — Extender `/tools/call` con rama de modo agente

Acción (patch en `services/llm_docs-mcp/gateway.py`) Inserta arriba los imports si faltan:

```
+import json +import re +import time
```

Ubica el handler actual de `POST /tools/call` . Agrega la rama de **modo agente** antes del camino legacy:


```

--- a/services/llm_docs-mcp/gateway.py
+++ b/services/llm_docs-mcp/gateway.py
@@
 @app.post("/tools/call")
 async def tools_call(request: Request):
     req = await request.json()
-    # Camino legacy: requiere {"tool": "...", "params": {...}}
-    tool = req.get("tool")
-    if not tool:
-        return JSONResponse({"error": "missing 'tool'", status_code=400})
+    # NUEVO: Modo agente (messages + tools)
+    if AGENT_MODE and "messages" in req:
+        return await agent_mode(req)
+
+    # Camino legacy: requiere {"tool": "...", "params": {...}}
+    tool = req.get("tool")
+    if not tool:
+        return JSONResponse({"error": "missing 'tool' or 'messages'", status_code=400})

```

Paso 2 — Tool Registry (nombre → handler + schema)

Acción (añade a nivel de módulo, cerca de los flags):

```

# === Tool Registry (una sola fuente de verdad) ===
# schema: nombre_param -> tipo esperado o tupla (tipo, None si opcional)
TOOLS = {
    "doc-generar_respuesta_llm": {
        "desc": "Consulta RAG con generación LLM",
        "schema": {"query": str, "top_k": (int, None), "categoria": (str, None)},
        "handler": "handle_rag_call",
    },
    "scheduler-init": {
        "desc": "Entrega control al flujo de agenda (orquestador)",
        "schema": {},
        "handler": "handle_scheduler_handover",
    },
    "complaint-init": {
        "desc": "Entrega control al flujo de reclamos (orquestador)",
        "schema": {},
        "handler": "handle_complaint_handover",
    },
}

```

Paso 3 — Prompt del Agente (role=system) y docs de herramientas

Acción (añade constantes y helper):

```

AGENT_SYSTEM_TPL = """
Eres un asistente municipal con HERRAMIENTAS disponibles.

HERRAMIENTAS:
{tools_doc}

Política de uso:
- Si la petición implica ACCIÓN (agendar, denunciar, registrar), llama la herramienta correspondiente.
- Si la petición es INFORMACIÓN (faq/trámite/documento), llama "doc-generar_respuesta_llm".
  - Si te pasan 'categoria' en hints, utilízala como parámetro 'categoria'.
- Si faltan datos obligatorios, pide aclaración (no inventes).
- Máximo {max_calls} llamadas de herramienta por turno.
- Formato de llamada (elige UNO):
  1) JSON: {{ "call": {{ "tool": "<name>", "params": {{...}} }} }}
  2) DSL : <CALL <name> param="..." otros="...">
Responde en español. No muestres tu razonamiento interno.
""".strip()

def build_tools_doc(allowed_tools: list[dict]) -> str:
    """
    allowed_tools: [{"name": "...", "desc": "...", "schema": {...}}, ...]
    Si viene vacío, usar todas las herramientas del registry (por compatibilidad).
    """
    if not allowed_tools:
        # rellenar desde registry
        for name, meta in TOOLS.items():
            allowed_tools.append({"name": name, "desc": meta["desc"], "schema": meta["schema"]})
    lines = []

```

```
for t in allowed_tools:
    lines.append(f"- {t['name']}: {t.get('desc','')}. Parámetros: {list((t.get('schema') or {}).keys()))}")
return "\n".join(lines)
```

Paso 4 — Parsers de llamada: JSON inline y mini-DSL <CALL ...>

Acción (añade helpers de parseo y validación):

```
JSON_CALL_RE = re.compile(r'\{[\s\S]*"call"[\s\S]*\}', re.MULTILINE)
DSL_CALL_RE = re.compile(r'<CALL\s+([a-zA-Z0-9_\-\.\./]+\s*(.??))>', re.DOTALL)

def try_parse_call(text: str):
    """
    Intenta parsear primero JSON con clave 'call', luego DSL <CALL ...>.
    Retorna dict {"tool": str, "params": dict} o None.
    """
    # 1) JSON inline
    m = JSON_CALL_RE.search(text)
    if m:
        try:
            obj = json.loads(m.group(0))
            if isinstance(obj, dict) and "call" in obj and "tool" in obj["call"]:
                tool = obj["call"]["tool"]
                params = obj["call"].get("params", {}) or {}
                if isinstance(params, dict):
                    return {"tool": tool, "params": params}
        except Exception:
            pass
    # 2) DSL
    m = DSL_CALL_RE.search(text)
    if m:
        tool = m.group(1).strip()
        kvs = m.group(2) or ""
        params = {}
        # pares key="value"
        for key, val in re.findall(r'([a-zA-Z0-9_\-]+\s*=\s*"([^"]*)"')', kvs):
            params[key] = val
        return {"tool": tool, "params": params}
    return None

def _typename(pytype):
    if isinstance(pytype, tuple):
        t = pytype[0]
    else:
        t = pytype
    return getattr(t, "__name__", str(t))

def validate_params(tool_name: str, params: dict) -> tuple[bool, str | None]:
    meta = TOOLS.get(tool_name)
    if not meta:
        return False, f"tool '{tool_name}' not registered"
    schema = meta.get("schema") or {}
    # obligatorios (tipo simple) y opcionales ((tipo, None))
    for key, t in schema.items():
        is_optional = isinstance(t, tuple) and t[1] is None
        if not is_optional and key not in params:
            return False, f"missing required param '{key}'"
        if key in params and params[key] is not None:
            val = params[key]
            base_t = t[0] if isinstance(t, tuple) else t
            # coerción simple
            try:
                if base_t is int:
                    params[key] = int(val)
                elif base_t is float:
                    params[key] = float(val)
                elif base_t is str:
                    params[key] = str(val)
            except Exception:
                return False, f"param '{key}' must be {_typename(t)}"
    # rechazar params desconocidos? (suave: permitir)
    return True, None
```

Paso 5 — Serialización de mensajes y wrapper de generación LLM

Acción (añade helpers):

```
def serialize_messages(messages: list[dict]) -> str:
    """
    Convierte mensajes estilo [{"role":"user","content":"..."}] a un bloque de texto instructivo.
    """

    parts = []
    for m in messages:
        role = m.get("role","user")
        content = (m.get("content") or "").strip()
        if not content:
            continue
        if role == "system":
            parts.append(f"[SISTEMA]\n{content}\n")
        elif role == "assistant":
            parts.append(f"[ASISTENTE]\n{content}\n")
        else:
            parts.append(f"[USUARIO]\n{content}\n")
    return "\n".join(parts).strip()

async def llama_generate(prompt: str, temperature: float = 0.1, top_p: float = 0.9, stop: list[str] | None = None) -> str:
    """
    Wrapper simple sobre tu función de generación existente (generate_response).
    Mantiene parámetros deterministas para reducir alucinaciones en llamadas de herramienta.
    """

    # Si tu generate_response ya acepta temp/top_p/stop, pásalos; si no, ajusta internamente.
    return await generate_response(prompt, temperature=temperature, top_p=top_p, stop=stop)
```

Si tu `generate_response` no acepta `temperature/top_p/stop`, crea una segunda función interna que sí los pase a llama.cpp. Mantén compatibilidad.

Paso 6 — Bucle de decisión del agente

Acción (añade la función `agent_mode`):

```
async def agent_mode(req: dict):
    """
    Modo agente: decide herramienta y devuelve 'final' o 'handover'.
    Entrada:
    - messages: historial
    - tools: lista permitida [{"name":"...", "desc":"...", "schema":{...}}, ...] (opcional)
    - hints: {"categoria":"..."} (opcional)
    """

    t0 = time.time()
    messages = req.get("messages") or []
    allowed_tools = req.get("tools") or []
    hints = req.get("hints") or {}
    categoria_hint = hints.get("categoria")

    # Construir doc y sistema
    tools_doc = build_tools_doc(allowed_tools)
    system = AGENT_SYSTEM_TPL.format(tools_doc=tools_doc, max_calls=AGENT_MAX_TOOL_CALLS)

    # Materializar transcript + sistema
    conv_text = system + "\n\n" + serialize_messages(messages) + "\n"

    # Conjunto de nombres permitidos
    allowed_names = set([t["name"] for t in allowed_tools]) if allowed_tools else set(TOOLS.keys())

    # Loop de decisión
    step = 0
    while step <= AGENT_MAX_TOOL_CALLS:
        step += 1
        out = await llama_generate(conv_text, temperature=0.1, top_p=0.9)
        _logger.info("agent.step=%s raw_out=%s", step, out[:500])

        call = try_parse_call(out)
        if not call:
            # Respuesta final directa
            dt = int((time.time()-t0)*1000)
            _logger.info("agent.final step=%s duration_ms=%s", step, dt)
            return {"type": "final", "text": out}

        tool_name = call["tool"]
        params = call.get("params", {}) or {}
        _logger.info("agent.tool_selected=%s params=%s", tool_name, params)

        if tool_name not in allowed_names:
```

```
        return {"type": "error", "reason": "tool_not_allowed", "tool": tool_name}

    ok, err = validate_params(tool_name, params)
    if not ok:
        return {"type": "error", "reason": "schema_invalid", "detail": err}

    # Ejecutar handler
    handler_name = TOOLS[tool_name]["handler"]
    handler_fn = globals().get(handler_name)
    if not handler_fn:
        return {"type": "error", "reason": "handler_not_found", "tool": tool_name}

    # Tip: pasar hints (categoria) a RAG si no vino explícita
    if tool_name == "doc-generar_respuesta_llm" and categoria_hint and "categoria" not in params:
        params["categoria"] = categoria_hint

    result = await handler_fn(params, hints)
    _logger.info("agent.tool_result for %s: %s", tool_name, str(result)[:500])

    # Handover: devolvemos inmediato para que orquestador active flujo
    if isinstance(result, dict) and result.get("type") == "handover":
        dt = int((time.time()-t0)*1000)
        _logger.info("agent.handover flow=%s duration_ms=%s", result.get("flow"), dt)
        return result

    # Inyectar resultado como 'ASISTENTE' y pedir redacción final (sin otra llamada)
    conv_text += f'\n[ASISTENTE]\n[RESULTADO {tool_name}]: {json.dumps(result, ensure_ascii=False)}\n'
    final_out = await llama_generate(conv_text, temperature=0.1, top_p=0.9)
    dt = int((time.time()-t0)*1000)
    _logger.info("agent.final_after_tool step=%s duration_ms=%s", step, dt)
    # puedes incluir refs si las trae el result (RAG)
    return {"type": "final", "text": final_out, "refs": result.get("referencias") if isinstance(result, dict)
    else None}

    return {"type": "error", "reason": "max_tool_calls_exceeded"}
```

Paso 7 — Handlers de herramientas

Acción (añade handlers mínimos reutilizando tu lógica):

```
async def handle_rag_call(params: dict, hints: dict):
    query = (params.get("query") or "").strip()
    if not query:
        return {"respuesta": "Necesito tu pregunta para buscar en los documentos.", "referencias": []}
    top_k = int(params.get("top_k", 5))
    categoria = params.get("categoria") # ya llega de Fase 1/2 si procede
    return await generar_respuesta_llm(query, top_k=top_k, categoria=categoria)

async def handle_scheduler_handover(params: dict, hints: dict):
    return {"type": "handover", "flow": "scheduler"}

async def handle_complaint_handover(params: dict, hints: dict):
    return {"type": "handover", "flow": "complaint"}
```

Nota: deliberadamente **no** ejecutamos la transacción aquí; solo devolvemos `handover` para que el **orquestador** mantenga el flujo y las validaciones.

Paso 8 — Logging y salvaguardas

Ya añadiste logs de:

- `agent.step`, `agent.tool_selected`, `agent.tool_result`, `agent.final*`, `duration_ms`.
- Errores `tool_not_allowed`, `schema_invalid`, `handler_not_found`, `max_tool_calls_exceeded`.

Si quieres reforzar **timeouts**:

- Envuelve `llama_generate` y las llamadas de handlers con `asyncio.wait_for(..., timeout=SECONDS)` y captura `asyncio.TimeoutError`.

Checklist (Fase 3)

- Rama `"messages"` en `/tools/call` implementada.
- `TOOLS` registry con schemas + handlers.
- Parser robusto: intenta **JSON** y luego **DSL** `<CALL ...>`.
- Respetar `AGENT_MAX_TOOL_CALLS`.
- Logs de decisiones, herramienta invocada y duración.

Pruebas (Criterios de aceptación)

1) Handover a Reclamos

```
curl -s http://localhost:<PUERTO_LLM_DOCS>/tools/call \
-H 'Content-Type: application/json' \
-d '{
  "messages": [
    {"role":"user","content":"Quiero denunciar un bache frente a mi casa"}
  ],
  "tools": [{"name":"complaint-init"}]
}' | jq .
```

Esperado:

{ "type": "handover", "flow": "complaint" } (o un <CALL complaint-init ...> parseado que derive en ese objeto).

2) RAG informativo con fuente

```
curl -s http://localhost:<PUERTO_LLM_DOCS>/tools/call \
-H 'Content-Type: application/json' \
-d '{
  "messages": [
    {"role":"user","content":"¿Cuál es el horario de atención del municipio?"}
  ],
  "tools": [{"name":"doc-generar_respuesta_llm","schema":{"query":"str","categoria":"str"}}],
  "hints": {"categoria":"faq"}
}' | jq .
```

Esperado:

{ "type": "final", "text": "...", "refs": [...] } y logs con RAG category-aware ON (si activaste RAG_CATEGORY_AWARE=1 en Fase 2).

3) Tool no permitida

```
curl -s http://localhost:<PUERTO_LLM_DOCS>/tools/call \
-H 'Content-Type: application/json' \
-d '{
  "messages": [{"role":"user","content":"agenda una cita"}],
  "tools": [{"name":"doc-generar_respuesta_llm"}]
}' | jq .
```

Esperado:

{ "type": "error", "reason": "tool_not_allowed" } si el modelo intenta otra herramienta no listada.

4) Legacy intacto

```
curl -s http://localhost:<PUERTO_LLM_DOCS>/tools/call \
-H 'Content-Type: application/json' \
-d '{"tool":"doc-generar_respuesta_llm","params":{"query":"horario","top_k":3}}' | jq .
```

Esperado: mismo comportamiento que antes.

Notas operativas

- Mantén AGENT_MODE=1 solo en **canary** al principio.
- Ajusta temperature a **0.0–0.2** para reducir “creatividad” en el formateo de llamadas.
- Si el modelo tiende a **responder en texto sin llamar** cuando debería llamar, refuerza en AGENT_SYSTEM_TPL con **ejemplos** (few-shot) de llamadas JSON y DSL, especialmente para agenda/reclamo.
- Si el modelo emite **JSON parcial**, tu parser (JSON_CALL_RE) ya intenta capturarlo; aun así, considera **stop-seqs** para cortar la salida al cerrar el JSON.

Con estos pasos, llm_docs-mcp queda habilitado como **agente** con herramientas, manteniendo intacto el flujo legacy y preparado para integrarse con el orquestador en la siguiente fase.

Fase 4 · Exponer flujos transaccionales como “herramientas”

Objetivo: el agente **no reemplaza** flujos; los activa.

Cambios en código

- services/llm_docs-mcp/gateway.py:

- Handlers de herramientas “proxy”:
 - scheduler-init → retorna { "handover": "scheduler" }
 - complaint-init → { "handover": "complaint" }
 - (Opcional) handlers que llamen endpoints internos si existen.
- Devolver handover al orquestador de forma explícita.

Checklist

- ☐ Herramientas scheduler-init / complaint-init registradas
- ☐ Respuesta incluye handover reconocido por orquestador

Criterios de aceptación

- El agente devuelve handover y el orquestador entra al flujo respetando la lógica actual de slots/preguntas.

Instrucciones

Objetivo: que el **agente** pueda **activar** (no reemplazar) los flujos de **Agenda** y **Reclamos** devolviendo un **handover** explícito al orquestador.
Precondiciones: Fase 3 implementada (agent_mode , registry, parsers, handlers básicos).

Paso 1 — Registrar herramientas “proxy” en el Tool Registry

Archivo: services/llm_docs-mcp/gateway.py

Si en Fase 3 ya agregaste estas entradas, revisa que coincidan y continúa al Paso 2.

Acción (patch en la definición de TOOLS):

```
--- a/services/llm_docs-mcp/gateway.py
+++ b/services/llm_docs-mcp/gateway.py
@@
TOOLS = {
    "doc-generar_respuesta_llm": {
        "desc": "Consulta RAG con generación LLM",
        "schema": {"query": str, "top_k": (int, None), "categoria": (str, None)},
        "handler": "handle_rag_call",
    },
+   "scheduler-init": {
+       "desc": "Entrega control al flujo de agenda (orquestador)",
+       "schema": {},
+       "handler": "handle_scheduler_handover",
+   },
+   "complaint-init": {
+       "desc": "Entrega control al flujo de reclamos (orquestador)",
+       "schema": {},
+       "handler": "handle_complaint_handover",
+   },
}
```

Checklist parcial

- Herramientas scheduler-init y complaint-init **registradas**.

Paso 2 — Implementar handlers “proxy” que devuelven handover

Archivo: services/llm_docs-mcp/gateway.py

Estos handlers **no ejecutan** el flujo ni llaman microservicios; solo devuelven la orden de traspaso. Mantiene la lógica transaccional en el **orquestador**.

Acción (añadir si no existen):

```
async def handle_scheduler_handover(params: dict, hints: dict):
    # No ejecutar acciones aquí. Devolver orden explícita al orquestador.
    return {"type": "handover", "flow": "scheduler"}

async def handle_complaint_handover(params: dict, hints: dict):
    # No ejecutar acciones aquí. Devolver orden explícita al orquestador.
    return {"type": "handover", "flow": "complaint"}
```

Checklist parcial

- Handlers devuelven { "type": "handover", "flow": "<scheduler|complaint>" }.

Paso 3 — Asegurar que el bucle del agente propague el handover “tal cual”

Archivo: `services/llm_docs-mcp/gateway.py` (función `agent_mode`)

En Fase 3 ya lo hicimos: si el handler retorna `"type": "handover"`, **devolvemos inmediato** al cliente (orquestador). Verifica este bloque:

Acción (verificación / patch idempotente):

```
-         # Handover: devolvemos inmediato para que orquestador active flujo
+         # Handover: devolvemos inmediato para que el orquestador active el flujo transaccional
         if isinstance(result, dict) and result.get("type") == "handover":
             dt = int((time.time()-t0)*1000)
             _logger.info("agent.handover flow=%s duration_ms=%s", result.get("flow"), dt)
             return result
```

Checklist parcial

- `agent_mode` corta y **devuelve** el handover sin intentar redactar respuesta final.

Paso 4 — (Opcional) Exponer proxy “directo” en el modo legacy

Archivo: `services/llm_docs-mcp/gateway.py` (endpoint `POST /tools/call`)

Útil para pruebas o integraciones que llamen `tool+params` sin “messages”.
Si te sirve, permite que `{"tool": "scheduler-init"}` y `{"tool": "complaint-init"}` devuelvan handover directamente:

Acción (verificación / patch idempotente en rama legacy):

```
elif tool == "doc-generar_respuesta_llm":
    ...
+elif tool == "scheduler-init":
+    return {"type": "handover", "flow": "scheduler"}
+elif tool == "complaint-init":
+    return {"type": "handover", "flow": "complaint"}
else:
    return JSONResponse({"error": f"unknown tool '{tool}'"}, status_code=400)
```

Paso 5 — (Opcional) Preparar handlers con proxy HTTP interno

Si ya tienes microservicios con endpoints de **salud** o **prevalidación**, puedes (opcional) llamarles aquí **solo** para confirmar disponibilidad, sin iniciar el flujo.

Acción (opcional, requiere `httpx/requests`):

```
import os
import httpx

SCHEDULER_HEALTH_URL = os.getenv("SCHEDULER_HEALTH_URL") # ej: http://scheduler-mcp:8080/health
COMPLAINTS_HEALTH_URL = os.getenv("COMPLAINTS_HEALTH_URL")

async def handle_scheduler_handover(params: dict, hints: dict):
    if SCHEDULER_HEALTH_URL:
        try:
            async with httpx.AsyncClient(timeout=2.0) as client:
                await client.get(SCHEDULER_HEALTH_URL)
        except Exception:
            _logger.warning("scheduler health check failed")
    return {"type": "handover", "flow": "scheduler"}

async def handle_complaint_handover(params: dict, hints: dict):
    if COMPLAINTS_HEALTH_URL:
        try:
            async with httpx.AsyncClient(timeout=2.0) as client:
                await client.get(COMPLAINTS_HEALTH_URL)
        except Exception:
            _logger.warning("complaints health check failed")
    return {"type": "handover", "flow": "complaint"}
```

No iniciar transacciones aquí; solo *health-check*. Mantén la lógica de slots/validaciones en el **orquestador**.

Paso 6 — Logging estructurado del handover

Archivo: `services/llm_docs-mcp/gateway.py`

Acción (ya incluido en `agent_mode`; reforzar si quieres):

```
_logger.info("handover.emitted flow=%s source=agent", result.get("flow"))
```

Checklist (Fase 4)

- scheduler-init y complaint-init **registradas** en TOOLS .
- Handlers devuelven { "type": "handover", "flow": "<...>" } .
- agent_mode **devuelve tal cual** el handover (sin redacción posterior).
- (Opcional) Camino **legacy** admite tool: scheduler-init|complaint-init .
- Logs muestran emisión de handover y duración.

Pruebas (Criterios de aceptación)

1) Handover a Reclamos (modo agente)

```
curl -s http://localhost:<PUERTO_LLM_DOCS>/tools/call \
-H 'Content-Type: application/json' \
-d '{
  "messages": [{"role":"user","content":"Quiero denunciar un bache en mi calle"}],
  "tools": [{"name":"complaint-init"}]
}' | jq .
```

Esperado: { "type": "handover", "flow": "complaint" }

2) Handover a Agenda (modo agente)

```
curl -s http://localhost:<PUERTO_LLM_DOCS>/tools/call \
-H 'Content-Type: application/json' \
-d '{
  "messages": [{"role":"user","content":"Necesito agendar una cita para mañana"}],
  "tools": [{"name":"scheduler-init"}]
}' | jq .
```

Esperado: { "type": "handover", "flow": "scheduler" }

3) Camino legacy (opcional)

```
curl -s http://localhost:<PUERTO_LLM_DOCS>/tools/call \
-H 'Content-Type: application/json' \
-d '{"tool":"scheduler-init","params":{}}' | jq .
```

Esperado: { "type": "handover", "flow": "scheduler" }

4) Integración con orquestador (smoke)

(Si ya tienes la Fase 5 preparada) Lanza una consulta que dispare agenda/reclamo con AGENT_MODE=1 .
Esperado: el orquestador **detecta** type=handover , entra en el flujo correspondiente y continúa con su lógica de **slots/preguntas** sin cambios.

Observaciones

- Esta fase **no** cambia la lógica de negocio de agenda/reclamos; solo añade “**puntos de enganche**” para que el agente los active.
- Mantiene la separación de responsabilidades: **decisión** en el agente, **ejecución** en el orquestador.
- El esquema de respuesta ({ "type": "handover", "flow": "..."}) es **estable** y sencillo de consumir desde el orquestador en la Fase 5.

Fase 5 · Integración en orchestrator.py

Objetivo: política híbrida: router determinista + agente.

Cambios en código

- LlmDocsClient :
 - Nuevo método agent_call(messages, tools, categoria?) (HTTP a /tools/call).
- orchestrate() :
 - Política:
 - Si AGENT_MODE=1 y (intención n/a o baja confianza o multi-intención) → agent_call .
 - Pasar historial y categoria si existe.
 - Procesar respuesta del agente:
 - Si handover → activar flujo transaccional existente.

- Si `respuesta_final` → devolver al usuario.
- Si fallo/ambigua → fallback determinista actual.
- Heurísticas de **multi-intención/ambigüedad** (simples):
 - Conjunción “ y ” + términos de familias distintas (agenda/trámite/info).
 - Verbos de acción (“agendar/denunciar/solicito”) + keywords de trámite/documento.

Checklist

- Método `agent_call()` implementado
- Paso de historial (N mensajes recientes) al agente
- Handover procesado correctamente
- Fallback intacto

Criterios de aceptación

- En `AGENT_MODE=1`, casos ambiguos pasan por agente; casos claros siguen por ruta determinista.

Instrucciones

Objetivo: aplicar **política híbrida**. Mantener el ruteo determinista como columna vertebral y **usar el agente** (en `llm_docs-mcp`) solo cuando haya `n/a`, **baja confianza** o **multi-intención**.
Precondiciones: Fases 0–4 completadas (especialmente `agent_mode` en `/tools/call`).

Paso 1 — `LlmDocsClient`: añadir `agent_call(...)`

Archivo: `mcp-core/orchestrator.py` (o donde está definida tu clase cliente hacia `llm_docs-mcp`)

Acción (patch):

```
--- a/mcp-core/orchestrator.py
+++ b/mcp-core/orchestrator.py
@@
+ import os
+ import requests
+ import logging
@@
+ class LlmDocsClient:
+     def __init__(self, base_url: str, timeout: int = 8):
+         self.base_url = base_url.rstrip("/")
+         self.timeout = timeout
+         self._log = logging.getLogger("llm_docs_client")
@@
+     def tools_call(self, payload: dict) -> dict:
+         url = f"{self.base_url}/tools/call"
+         res = requests.post(url, json=payload, timeout=self.timeout)
+         res.raise_for_status()
+         return res.json()
+
+     # === Nuevo: llamada al modo agente ===
+     def agent_call(self, messages: list[dict], tools: list[dict] | None = None,
+                   categoria: str | None = None, timeout: int | None = None) -> dict:
+         """
+         Envía conversación al agente y herramientas permitidas.
+         - messages: [{"role": "user|assistant|system", "content": "..."}]
+         - tools: [{"name": "doc-generar_respuesta_llm"}, {"name": "scheduler-init"}, {"name": "complaint-init"}]
+         - categoria: hint opcional ("faq"|"tramite"|"documento")
+         """
+         url = f"{self.base_url}/tools/call"
+         payload = {"messages": messages}
+         if tools:
+             payload["tools"] = tools
+         if categoria:
+             payload["hints"] = {"categoria": categoria}
+         to = timeout or self.timeout
+         self._log.info("agent_call -> tools=%s categoria=%s", [t.get("name") for t in (tools or [])],
+ categoria)
+         res = requests.post(url, json=payload, timeout=to)
+         res.raise_for_status()
+         return res.json()
```

Paso 2 — Helpers en `orchestrator.py`: historial y heurísticas

Acción (añadir utilidades, cerca de otros helpers):


```

--- a/mcp-core/orchestrator.py
+++ b/mcp-core/orchestrator.py
@@
+ import re
@@
+# === Construcción de mensajes para el agente (N últimos turnos + turno actual) ===
+def _build_agent_messages(contexto, session_id: str, user_text: str, history_k: int = 4) -> list[dict]:
+    msgs = []
+    try:
+        # Ajusta a tu API real de contexto. Usa un try/except para no romper si difiere.
+        hist = contexto.get_history(session_id, limit=history_k) # esperado: lista de dicts {"role","content"}
+        for m in hist or []:
+            role = m.get("role") or ("assistant" if m.get("from") == "assistant" else "user")
+            content = (m.get("content") or m.get("text") or "").strip()
+            if content:
+                msgs.append({"role": role, "content": content})
+    except Exception:
+        pass
+    # turno actual
+    msgs.append({"role": "user", "content": user_text.strip()})
+    return msgs
+
+# === Heurísticas simples ===
+_ACTION_VERBS = r"(agendar|reserv(ar|a)|denunciar|report(ar|e)|solicito|solicitar|quiero|necesito)"
+_TRAMITE_TOKS = r"(permiso|certificado|tr[aa]mite|licencia|turno|cita)"
+_INFO_TOKS     = r"(horario|normativa|ordenanza|documento|requisitos|pasos)"
+
+def _is_multi_intent(text: str) -> bool:
+    t = text.lower()
+    # conjunción y mezcla de familias: acción + info o trámite + info
+    has_and = " y " in t or " además " in t
+    action  = re.search(_ACTION_VERBS, t)
+    tramite = re.search(_TRAMITE_TOKS, t)
+    info    = re.search(_INFO_TOKS, t)
+    return bool(has_and and ((action and info) or (tramite and info)))
+
+def _low_confidence(classification: dict | None) -> bool:
+    if not classification:
+        return True
+    # si viene score/probabilidad:
+    score = classification.get("score") or classification.get("confidence")
+    try:
+        if score is not None and float(score) < 0.18: # umbral conservador
+            return True
+    except Exception:
+        pass
+    # también considerar etiquetas vacías o 'n/a'
+    intent = (classification.get("intent") or "").strip().lower()
+    return intent in ("", "n/a")

```

Ajusta `contexto.get_history(...)` a tu API real si el nombre difiere (o deja el `try/except`, que simplemente no enviará historial si falla).

Paso 3 — Política híbrida en `orchestrate()`

Acción (modificar bloque principal donde decides la ruta tras clasificar):

```

--- a/mcp-core/orchestrator.py
+++ b/mcp-core/orchestrator.py
@@
+ def orchestrate(session_id: str, user_text: str, contexto, llm_client: LlmDocsClient):
@@
-     raw_intent = (classification.get("intent") if classification else "") or ""
-     norm_intent = _norm_intent(raw_intent)
+     raw_intent = (classification.get("intent") if classification else "") or ""
+     norm_intent = _norm_intent(raw_intent)
+     if norm_intent not in INTENT_MAP and norm_intent.endswith("s"):
+         singular = norm_intent[:-1]
+         if singular in INTENT_MAP:
+             norm_intent = singular
+     intent = INTENT_MAP.get(norm_intent, "n/a")
+     _log.info("intent_raw=%s intent_norm=%s mapped_action=%s", raw_intent, norm_intent, intent)
+
+     # Determinar 'categoria' (solo para rama informativa/RAG)
+     categoria = norm_intent if norm_intent in ("faq", "tramite", "documento") else None
+
+     # Heurísticas de activación del agente

```

```

+     use_agent = bool(AGENT_MODE == 1 and (
+         intent == "n/a" or
+         _low_confidence(classification) or
+         _is_multi_intent(user_text)
+     ))
+     _log.info("agent_policy use_agent=%s (AGENT_MODE=%s)", use_agent, AGENT_MODE)
+
+     if use_agent:
+         # Construir messages + tools permitidas
+         messages = _build_agent_messages(contexto, session_id, user_text, history_k=4)
+         tools = [
+             {"name": "doc-generar_respuesta_llm"},
+             {"name": "scheduler-init"},
+             {"name": "complaint-init"},
+         ]
+         try:
+             agent_resp = llm_client.agent_call(messages, tools=tools, categoria=categoria)
+             # Procesar respuesta del agente
+             rtype = agent_resp.get("type")
+             if rtype == "handover":
+                 flow = agent_resp.get("flow")
+                 _log.info("agent -> handover flow=%s", flow)
+                 if flow == "scheduler":
+                     return handle_scheduler_flow(session_id, user_text, contexto) # tu flujo actual
+                 if flow == "complaint":
+                     return handle_complaint_flow(session_id, user_text, contexto) # tu flujo actual
+                 # Si recibimos un flow desconocido, caer a fallback determinista:
+                 _log.warning("agent handover desconocido: %s", flow)
+             elif rtype == "final":
+                 text = agent_resp.get("text") or ""
+                 refs = agent_resp.get("refs") or []
+                 _log.info("agent -> final (len=%s, refs=%s)", len(text), len(refs))
+                 return {"respuesta": text, "referencias": refs}
+             else:
+                 _log.warning("agent -> respuesta inesperada: %s", rtype)
+         except Exception as e:
+             _log.exception("agent_call error: %s", e)
+         # Si el agente falla o no decide, seguir por ruta determinista
+         _log.info("agent fallback -> ruteo determinista")

# === Ruta determinista (existente) ===
if intent == "saludo":
    return {"respuesta": _pick_smalltalk("saludo")}
elif intent == "despedida":
    return {"respuesta": _pick_smalltalk("despedida")}
elif intent == "agradecimiento":
    return {"respuesta": _pick_smalltalk("agradecimiento")}
elif intent == "ask_document":
-     return handle_document_query(session_id, user_text, entidades, contexto)
+     # Propagar categoria si existiese (Fase 1/2)
+     return handle_document_query(session_id, user_text, entidades, contexto, intent_type=categoria)
elif intent == "init_scheduler":
    return handle_scheduler_flow(session_id, user_text, contexto)
elif intent == "init_complaint":
    return handle_complaint_flow(session_id, user_text, contexto)
else:
    return handle_fallback(session_id, user_text, contexto)

```

Sustituye `handle_scheduler_flow` / `handle_complaint_flow` por los nombres reales de tus handlers (en el repo algunos usan `init_...` / `cont_...`; apunta al **entry** del flujo).

Paso 4 — Logs y compatibilidad

- La política deja **intacta** la rama determinista.
- Si `AGENT_MODE=0`, `use_agent` será `False` y **no cambia nada**.
- Con `AGENT_MODE=1`, solo disparas el agente cuando `n/a` o baja confianza o multi-intención.

Checklist (Fase 5)

- Método `agent_call(...)` implementado en `LlmDocsClient`.
- Historial (hasta 4 turnos) se pasa al agente vía `_build_agent_messages`.
- `handover` del agente enrutado a **flujos existentes** (agenda/reclamo).
- Fallback determinista intacto si el agente falla o responde inesperado.

Pruebas (Criterios de aceptación)

1) Caso ambiguo → Agente

Usuario: “Quiero agendar y saber los requisitos del permiso”

- AGENT_MODE=1
- Esperado: `_is_multi_intent=True` ⇒ `use_agent=True`.
- Agente podría:
 - Devolver `handover a scheduler`
 - Llamar RAG y redactar respuesta final.
- Si `handover`, el orquestador entra al flujo de agenda (pregunta datos).
- Si `final`, devuelve texto con requisitos y (si Fase 2 ON) fuente(s).

2) Caso claro → Determinista

Usuario: “¿Cuál es el horario de atención?”

- AGENT_MODE=1 pero no hay ambigüedad ni baja confianza.
- Esperado: ruta determinista `ask_document + categoria="faq"` (si aplica).

3) n/a → Agente

Forzar un ejemplo que antes caía en `n/a`.

- Esperado: se activa agente; si decide `doc-generar_respuesta_llm`, retorna `final`.
- Si el agente falla, cae al fallback determinista (sin romper).

4) Error del agente → Fallback

Simula fallo de `/tools/call` o respuesta inesperada.

- Esperado: log de excepción y continuación por ruta determinista.

Notas y ajustes finos

- Si tu clasificador **no expone** `score`, `_low_confidence` ya considera vacío/`n/a` como baja confianza.
- Puedes endurecer o relajar `_is_multi_intent` ajustando los regex de verbos/keywords.
- Si quieres **más control** sobre “casos claros”, añade una whitelist (e.g., si el intent ya es `init_scheduler` directo, **no** uses agente).
- Mantén `temperature` **baja** en el agente para minimizar salidas no estructuradas (ya configurado en Fase 3).
- Revisa que `handle_document_query(...)` acepte `intent_type=categoria` (Fases 1–2).

Fase 6 · Validaciones y guardarraíles

Objetivo: seguridad, consistencia y control.

Cambios en código

- Validación de parámetros de herramientas (tipos, rangos, formatos).
- Confirmaciones antes de acciones sensibles (p. ej., crear cita/registrar reclamo requiere “sí/confirmo” si el modelo deduce datos).
- Timeouts y reintentos controlados; retorno seguro al fallback.

Checklist

- ☐ Schemas de parámetros por herramienta
- ☐ Confirmación explícita en transacciones
- ☐ Timeouts definidos y gestionados

Criterios de aceptación

- No se ejecutan acciones transaccionales sin datos mínimos y confirmación.

Instrucciones

Fase 6 · Validaciones y guardarraíles — Instrucciones ejecutables (paso a paso)

Objetivo: elevar la **seguridad, consistencia y control** del sistema híbrido.

Alcance: **endurecer** el modo agente en `llm_docs-mcp` (parámetros, timeouts, reintentos, sanitización) y **blindar** los commits transaccionales en el orquestador con confirmación explícita del usuario.

Paso 1 — Especificación de schemas con validaciones fuertes (gateway)

Archivo: `services/llm_docs-mcp/gateway.py`

- Añade **constantes** de límites y conjunto permitido:

```
+ALLOWED_CATEGORIAS = {"faq", "tramite", "documento"}
+QUERY_MIN_LEN = 2
+QUERY_MAX_LEN = 512
+TOPK_MIN = 1
+TOPK_MAX = 8
```

2. Extiende el **Tool Registry** para incluir **constraints** por parámetro:

```
TOOLS = {
    "doc-generar_respuesta_llm": {
        "desc": "Consulta RAG con generación LLM",
-       "schema": {"query": str, "top_k": (int, None), "categoria": (str, None)},
+       "schema": {"query": str, "top_k": (int, None), "categoria": (str, None)},
+       "constraints": {
+           "query": {"min_len": QUERY_MIN_LEN, "max_len": QUERY_MAX_LEN},
+           "top_k": {"min": TOPK_MIN, "max": TOPK_MAX},
+           "categoria": {"one_of": list(ALLOWED_CATEGORIAS)}
+       },
        "handler": "handle_rag_call",
    },
    "scheduler-init": {
        "desc": "Entrega control al flujo de agenda (orquestador)",
        "schema": {},
        "handler": "handle_scheduler_handover",
    },
    "complaint-init": {
        "desc": "Entrega control al flujo de reclamos (orquestador)",
        "schema": {},
        "handler": "handle_complaint_handover",
    },
}
```

3. Añade **sanitización** de strings y **validación de constraints**:

```
+def _sanitize_str(s: str, max_len: int) -> str:
+    # quita controles, normaliza espacios y recorta longitud
+    s = (s or "").replace("\x00", " ").strip()
+    s = re.sub(r"\s+", " ", s)
+    if len(s) > max_len:
+        s = s[:max_len]
+    return s
+
def validate_params(tool_name: str, params: dict) -> tuple[bool, str | None]:
    meta = TOOLS.get(tool_name)
    if not meta:
        return False, f"tool '{tool_name}' not registered"
    schema = meta.get("schema") or {}
+    constraints = meta.get("constraints") or {}
    # obligatorios (tipo simple) y opcionales ((tipo, None))
    for key, t in schema.items():
        is_optional = isinstance(t, tuple) and t[1] is None
        if not is_optional and key not in params:
            return False, f"missing required param '{key}'"
        if key in params and params[key] is not None:
            val = params[key]
            base_t = t[0] if isinstance(t, tuple) else t
            # coerción simple
            try:
                if base_t is int:
                    params[key] = int(val)
                elif base_t is float:
                    params[key] = float(val)
                elif base_t is str:
-                    params[key] = str(val)
+                    params[key] = str(val)
            except Exception:
                return False, f"param '{key}' must be {_typename(t)}"
+
+    # constraints
+    c = constraints.get(key) or {}
+    if base_t is str:
+        if "min_len" in c and len(params[key]) < c["min_len"]:
+            return False, f"param '{key}' length < {c['min_len']}"
+        if "max_len" in c and len(params[key]) > c["max_len"]:
+            params[key] = params[key][:c["max_len"]]
+        # sanitiza siempre strings relevantes
+        params[key] = _sanitize_str(params[key], c.get("max_len", 1024))
```

```
+         if "one_of" in c and params[key] not in c["one_of"]:
+             return False, f"param '{key}' not in allowed set"
+     if base_t in (int, float):
+         if "min" in c and params[key] < c["min"]:
+             return False, f"param '{key}' < {c['min']}"
+         if "max" in c and params[key] > c["max"]:
+             params[key] = c["max"]
+
+     return True, None
```

Resultado: los parámetros quedan **coercionados**, **sanitizados** y **limitados** (longitud, rangos, dominio).

Paso 2 — Timeouts y reintentos con fallback seguro (gateway)

Archivo: `services/llm_docs-mcp/gateway.py`

1. Define **timeouts** y **reintentos**:

```
+import asyncio
+
+AGENT_LLM_TIMEOUT_SEC = int(os.getenv("AGENT_LLM_TIMEOUT_SEC", "8"))
+AGENT_HANDLER_TIMEOUT_SEC = int(os.getenv("AGENT_HANDLER_TIMEOUT_SEC", "4"))
+AGENT_MAX_RETRIES = int(os.getenv("AGENT_MAX_RETRIES", "1")) # 0 o 1 recomendado
```

2. Wrapper **genérico** con timeout+retry:

```
+async def with_timeout_retry(coro_factory, timeout_s: int, retries: int = 0):
+    """
+    Ejecuta la co-rutina producida por coro_factory() con timeout y reintento exponencial.
+    """
+    attempt = 0
+    delay = 0.5
+    while True:
+        try:
+            return await asyncio.wait_for(coro_factory(), timeout=timeout_s)
+        except asyncio.TimeoutError:
+            attempt += 1
+            _logger.warning("op timeout (attempt=%s timeout=%ss)", attempt, timeout_s)
+            if attempt > retries:
+                raise
+            await asyncio.sleep(delay)
+            delay *= 2.0
+        except Exception as e:
+            attempt += 1
+            _logger.warning("op error '%s' (attempt=%s)", e, attempt)
+            if attempt > retries:
+                raise
+            await asyncio.sleep(delay)
+            delay *= 2.0
```

3. Usa el wrapper en `agent_mode`:

```
-     out = await llama_generate(conv_text, temperature=0.1, top_p=0.9)
+     out = await with_timeout_retry(
+         lambda: llama_generate(conv_text, temperature=0.1, top_p=0.9),
+         timeout_s=AGENT_LLM_TIMEOUT_SEC,
+         retries=AGENT_MAX_RETRIES
+     )
@@
-     result = await handler_fn(params, hints)
+     result = await with_timeout_retry(
+         lambda: handler_fn(params, hints),
+         timeout_s=AGENT_HANDLER_TIMEOUT_SEC,
+         retries=0
+     )
@@
-     final_out = await llama_generate(conv_text, temperature=0.1, top_p=0.9)
+     final_out = await with_timeout_retry(
+         lambda: llama_generate(conv_text, temperature=0.1, top_p=0.9),
+         timeout_s=AGENT_LLM_TIMEOUT_SEC,
+         retries=0
+     )
```

4. **Errores** → respuesta **controlada** para fallback:


```

        while step <= AGENT_MAX_TOOL_CALLS:
            step += 1
-         out = await llama_generate(...)
+         try:
+             out = await with_timeout_retry(...)
+         except Exception as e:
+             _logger.exception("agent.llm_error: %s", e)
+             return {"type":"error","reason":"llm_timeout_or_error"}
@@
-         result = await handler_fn(...)
+         try:
+             result = await with_timeout_retry(...)
+         except Exception as e:
+             _logger.exception("agent.handler_error: %s", e)
+             return {"type":"error","reason":"handler_timeout_or_error","tool":tool_name}
```

Resultado: el orquestador recibirá un `type:error` y activará su **ruta determinista** (Fase 5 ya lo hace).

Paso 3 — Confirmación explícita antes de acciones sensibles (orquestador)

Archivo: `mcp-core/orchestrator.py`

Agrega una **pasarela de confirmación** justo antes del “commit” (crear cita / registrar reclamo). No cambiamos la recolección de `slots`, solo **interponemos** un resumen y requerimos un “sí/confirmo”.

1. Utilidades de confirmación:

```
+import unicodedata
+
+def _is_affirmative(text: str) -> bool:
+    t = unicodedata.normalize("NFKD", (text or "").lower())
+    t = t.replace("á", "a").replace("é", "e").replace("í", "i").replace("ó", "o").replace("ú", "u")
+    return bool(re.search(r"\b(si|sí|confirmo|de acuerdo|ok)\b", t))
+
+def _format_summary(data: dict) -> str:
+    # resume campos claves; ajusta nombres reales de tus slots
+    parts = []
+    for k,v in data.items():
+        if v is None or v == "":
+            continue
+        parts.append(f"- {k}: {v}")
+    return "\n".join(parts) if parts else "(sin datos)"
```

2. Estado de “esperando confirmación” (en tus flows):

Busca el punto donde **ya** tienes todos los datos para:

- **Agenda:** antes de `appointment_create` /llamar microservicio.
- **Reclamo:** antes de `complaint_register`.

Introduce una bandera en el contexto de sesión:

```
-def handle_scheduler_flow(session_id, user_text, contexto):
+def handle_scheduler_flow(session_id, user_text, contexto):
+    state = contexto.get_state(session_id) or {}
+    # si estamos esperando confirmación:
+    if state.get("scheduler_awaiting_confirm"):
+        if _is_affirmative(user_text):
+            state.pop("scheduler_awaiting_confirm", None)
+            contexto.set_state(session_id, state)
+            # >>> aquí continúa el commit real (crear cita) como antes <<<
+            return _create_appointment_and_reply(session_id, state, contexto)
+        else:
+            state.pop("scheduler_awaiting_confirm", None)
+            contexto.set_state(session_id, state)
+            return {"respuesta": "Operación cancelada. ¿Deseas modificar algún dato o agendar más tarde?"}
+
+    # ... lógica de slots existente ...
+    all_fields_collected = _scheduler_ready(state)
+    if all_fields_collected:
-        # commit directo (antes)
-        return _create_appointment_and_reply(session_id, state, contexto)
+        # pedir confirmación primero
+        resumen = _format_summary(state.get("scheduler_slots", state))
+        state["scheduler_awaiting_confirm"] = True
+        contexto.set_state(session_id, state)
+        msg = ("Voy a crear tu cita con los siguientes datos:\n"
```

```
+         f"{resumen}\n\n¿Confirmas? Responde “sí” para continuar o “no” para cancelar.")
+         return {"respuesta": msg}
+         # ... resto del flujo ...
```

Haz lo análogo en **reclamos**:

```
-def handle_complaint_flow(session_id, user_text, contexto):
+def handle_complaint_flow(session_id, user_text, contexto):
+     state = contexto.get_state(session_id) or {}
+     if state.get("complaint_awaiting_confirm"):
+         if _is_affirmative(user_text):
+             state.pop("complaint_awaiting_confirm", None)
+             contexto.set_state(session_id, state)
+             return _register_complaint_and_reply(session_id, state, contexto)
+         else:
+             state.pop("complaint_awaiting_confirm", None)
+             contexto.set_state(session_id, state)
+             return {"respuesta": "Operación cancelada. Puedes indicarme el problema nuevamente cuando gustes."}
+     # ... slots ...
+     if _complaint_ready(state):
-         return _register_complaint_and_reply(session_id, state, contexto)
+         resumen = _format_summary(state.get("complaint_slots", state))
+         state["complaint_awaiting_confirm"] = True
+         contexto.set_state(session_id, state)
+         msg = ("Registraré tu reclamo con estos datos:\n"
+             f"{resumen}\n\n¿Confirmas? Responde “sí” para continuar o “no” para cancelar.")
+         return {"respuesta": msg}
```

Nota: `_scheduler_ready`, `_complaint_ready`, `_create_appointment_and_reply` y `_register_complaint_and_reply` son **placeholders** para tus funciones reales de verificación/commit.

Paso 4 — Validación mínima de formatos en flujos (opcional pero recomendado)

Archivo: `mcp-core/orchestrator.py` (reutiliza en extracción de slots)

Añade **checkers** simples para campos habituales:

```
+RUT_RE    = re.compile(r"\b\d{1,2}\.\.? \d{3}\.\.? \d{3}-[\dkK]\b")
+MAIL_RE   = re.compile(r"\b[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}\b", re.I)
+PHONE_RE  = re.compile(r"\b(\+?56)?\s?9\s?\d{4}\s?\d{4}\b")
+DATE_RE   = re.compile(r"\b(20\d{2})-(0[1-9]|1[0-2])-(0[1-9]|[12]\d|3[01])\b") # ISO yyyy-mm-dd
+
+def _valid_rut(s: str) -> bool: return bool(RUT_RE.search(s or ""))
+def _valid_email(s: str) -> bool: return bool(MAIL_RE.search(s or ""))
+def _valid_phone(s: str) -> bool: return bool(PHONE_RE.search(s or ""))
+def _valid_date_iso(s: str) -> bool: return bool(DATE_RE.search(s or ""))
```

Úsalos cuando completes *slots*; si algo no valida, **pregunta de nuevo** o ofrece ejemplo de formato.

Paso 5 — Timeouts y reintentos en HTTP del orquestador (microservicios)

Archivo: `mcp-core/orchestrator.py`

- 1. Ajusta **timeouts** y opcional **reintentos** (para llamadas a `llm_docs-mcp`, `scheduler` o `complaints`):

```
class LlmDocsClient:
    def __init__(self, base_url: str, timeout: int = 8):
        self.base_url = base_url.rstrip("/")
-        self.timeout = timeout
+        self.timeout = timeout
        self._log = logging.getLogger("llm_docs_client")
@@
    def tools_call(self, payload: dict) -> dict:
        url = f"{self.base_url}/tools/call"
-        res = requests.post(url, json=payload, timeout=self.timeout)
+        res = requests.post(url, json=payload, timeout=self.timeout)
        res.raise_for_status()
        return res.json()
```

Si no tienes un wrapper de **reintentos**, crea uno **solo** para operaciones idempotentes (NO para commits transaccionales), p. ej., clasificación/RAG.

Paso 6 — Rutas de fallback claras ante errores

- El **agente** ya retorna `{"type": "error", ...}` en timeouts/errores → el orquestador **sigue** por ruta determinista (Fase 5).

- Si el **commit** de un flujo falla (HTTP 5xx), **no** reintentas automáticamente; responde al usuario: “Ocurrió un problema al registrar/crear. ¿Deseas reintentar?” y **vuelve** al estado pre-confirmación.

Checklist (Fase 6)

- **Schemas** con **constraints** por herramienta (longitud, rangos, `one_of`).
- Sanitización de strings y coerción de tipos.
- **Timeouts** y **reintentos** en `agent_mode` (LLM y handlers).
- Confirmación **explícita** antes de **commits transaccionales** (agenda/reclamos).
- Validación mínima de formatos de *slots* (RUT, email, teléfono, fecha).
- Fallback seguro en errores/tiempos de espera.

Criterios de aceptación — pruebas rápidas

1) No se ejecuta sin confirmar

- Usuario: “Agendar para mañana 10:00 con Juan Pérez, mi correo `jp@x.cl`”.
- Bot (antes del commit): muestra **resumen** y pide “**¿Confirmas?**”.
- Usuario: “sí” → crea cita.
- Usuario: “no” → **cancela** sin efectos.

2) Validación de parámetros del agente

- Llamada a `/tools/call` con `{"messages":[...], "tools":[{"name":"doc-generar_respuesta_llm"}], "hints":{"categoria":"xxx"}}`
- **Esperado:** `type:error` por `categoria` fuera de `one_of`.

3) Timeouts

- Simula modelo lento → `AGENT_LLM_TIMEOUT_SEC` expira.
- **Esperado:** `{"type":"error","reason":"llm_timeout_or_error"}` y el orquestador **sigue** ruta determinista.

4) RAG robusto

- Query larga >512 chars → se **trunca** y pasa validación.
- `top_k=99` → se **cap** en `TOPK_MAX`.

Notas finales

- Mantén `AGENT_MAX_RETRIES` **bajo** (0–1) para no disparar latencias; recuerda que en CPU/Q4_K_L tu presupuesto es estrecho.
- Evita **reintentos** automáticos en commits transaccionales; pide confirmación del usuario para reintentar.
- Estas defensas son **compatibles** con `AGENT_MODE=0`: el legacy sigue funcionando, solo que ahora con inputs más higiénicos y rutas de error más claras.

Fase 7 · Telemetría y observabilidad

Objetivo: medir impacto y depurar.

Cambios en código

- Logging estructurado:
 - `intent_raw/norm/categoria`, colección/filtro, top-k
 - Decisiones del agente, herramientas, duración, errores
- Métricas (Prometheus/StatsD o logs):
 - Tasa de `n/a`, handovers, éxito por categoría, latencia p95
- Trazas de conversación **anonimizadas** para revisión (sin PII).

Checklist

- ☐ Campos de log consistentes
- ☐ Panel básico (aunque sea con logs + jq)

Criterios de aceptación

- Se pueden comparar antes/después: hit-rate, `n/a`, latencia, errores de herramienta.

Instrucciones

Objetivo: **medir impacto y depurar** con logs estructurados, métricas (Prometheus/StatsD o, en su defecto, logs+ `jq`), y **trazas anonimizadas**.

Precondiciones: Fases 0–6 aplicadas; servicios `mcp-core/orchestrator.py` y `services/llm_docs-mcp/gateway.py` operativos.

Paso 1 — Variables de entorno (feature flags y tuning)

Archivo: `.env.example`

```
## === Telemetría y Observabilidad ===
## Activa logging estructurado y exportación de métricas
+TELEMETRY_ENABLED=1
+LOG_FORMAT=json          # json|text
+REDACTION_ENABLED=1      # 1=redactar PII en logs
+TRACE_SAMPLING=0.15      # 15% de conversaciones se trazan (anonimizadas)
+TRACE_SALT=munbot_salt   # sal para hashing de session_id
+
## Prometheus (si se usa)
+PROMETHEUS_ENABLED=1
+PROMETHEUS_NAMESPACE=munbot
+PROMETHEUS_PORT=0        # 0 = reusar FastAPI y ruta /metrics; >0 = arrancar servidor propio
```

Mantener `TELEMETRY_ENABLED=1` aun si `PROMETHEUS_ENABLED=0` (seguirás teniendo logs estructurados y trazas).

Paso 2 — Dependencias

Archivos: `services/llm_docs-mcp/requirements.txt` y `mcp-core/requirements.txt`

```
+prometheus-client>=0.20.0
```

Si usas un único requirements para todo, añade allí. Reconstruye imágenes si usas Docker.

Paso 3 — Helpers de logging estructurado y PII redaction

3.1 `gateway.py` (llm_docs-mcp)

Inserta tras los imports existentes:

```
+import json, hashlib
+
+def _getenv_bool(name: str, default: bool) -> bool:
+    v = os.getenv(name)
+    if v is None: return default
+    return v.strip() in ("1","true","TRUE","yes","on")
+
+TELEMETRY_ENABLED = _getenv_bool("TELEMETRY_ENABLED", True)
+REDACTION_ENABLED = _getenv_bool("REDACTION_ENABLED", True)
+LOG_FORMAT        = os.getenv("LOG_FORMAT","json").strip()
+TRACE_SAMPLING    = float(os.getenv("TRACE_SAMPLING","0.15"))
+TRACE_SALT        = os.getenv("TRACE_SALT","munbot_salt")
+
+EMAIL_RE  = re.compile(r"\b[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}\b", re.I)
+PHONE_RE  = re.compile(r"\b(\+?56)?\s?9\s?\d{4}\s?\d{4}\b")
+RUT_RE    = re.compile(r"\b\d{1,2}\.\.? \d{3}\.\.? \d{3}-[\dkK]\b")
+
+def _redact(text: str) -> str:
+    if not REDACTION_ENABLED: return text
+    t = text or ""
+    t = EMAIL_RE.sub("<EMAIL>", t)
+    t = PHONE_RE.sub("<PHONE>", t)
+    t = RUT_RE.sub("<RUT>", t)
+    return t
+
+def _sid_hash(session_id: str) -> str:
+    s = f"{TRACE_SALT}:{session_id}".encode("utf-8")
+    return hashlib.sha256(s).hexdigest()[:16]
+
+def _jlog(event: str, **fields):
+    if LOG_FORMAT == "json":
+        rec = {"event": event, **fields}
+        _logger.info(json.dumps(rec, ensure_ascii=False))
+    else:
+        _logger.info("%s | %s", event, fields)
```

3.2 `orchestrator.py`

Inserta tras imports:

```
+import json, hashlib
+
+def _getenv_bool(name: str, default: bool) -> bool:
+    v = os.getenv(name)
+    if v is None: return default
+    return v.strip() in ("1","true","TRUE","yes","on")
+
+TELEMETRY_ENABLED = _getenv_bool("TELEMETRY_ENABLED", True)
+REDACTION_ENABLED = _getenv_bool("REDACTION_ENABLED", True)
+LOG_FORMAT = os.getenv("LOG_FORMAT","json").strip()
+TRACE_SAMPLING = float(os.getenv("TRACE_SAMPLING","0.15"))
+TRACE_SALT = os.getenv("TRACE_SALT","munbot_salt")
+
+EMAIL_RE = re.compile(r"\b[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}\b", re.I)
+PHONE_RE = re.compile(r"\b(\+?56)?\s?9\s?\d{4}\s?\d{4}\b")
+RUT_RE = re.compile(r"\b\d{1,2}\.\.? \d{3}\.\.? \d{3}-[\dkK]\b")
+
+def _redact(text: str) -> str:
+    if not REDACTION_ENABLED: return text
+    t = text or ""
+    t = EMAIL_RE.sub("<EMAIL>", t)
+    t = PHONE_RE.sub("<PHONE>", t)
+    t = RUT_RE.sub("<RUT>", t)
+    return t
+
+def _sid_hash(session_id: str) -> str:
+    s = f"{TRACE_SALT}:{session_id}".encode("utf-8")
+    return hashlib.sha256(s).hexdigest()[:16]
+
+def _jlog(logger, event: str, **fields):
+    if LOG_FORMAT == "json":
+        rec = {"event": event, **fields}
+        logger.info(json.dumps(rec, ensure_ascii=False))
+    else:
+        logger.info("%s | %s", event, fields)
```

Paso 4 — Prometheus: métricas y endpoint `/metrics`

4.1 `gateway.py` (llm_docs-mcp)

Añade las métricas cerca de los flags:

```
+from prometheus_client import Counter, Histogram, Summary, CONTENT_TYPE_LATEST, generate_latest
+PROMETHEUS_ENABLED = _getenv_bool("PROMETHEUS_ENABLED", True)
+PROMETHEUS_NAMESPACE = os.getenv("PROMETHEUS_NAMESPACE","munbot")
+
+MET_AGENT_DECISIONS = Counter(
+    f"{PROMETHEUS_NAMESPACE}_agent_decisions_total",
+    "Decisiones del agente por tipo",
+    ["type"] # final|handover|error
+)
+MET_HANDOVERS = Counter(
+    f"{PROMETHEUS_NAMESPACE}_handovers_total",
+    "Handovers emitidos por flujo",
+    ["flow"] # scheduler|complaint
+)
+MET_RAG_HITS = Counter(
+    f"{PROMETHEUS_NAMESPACE}_rag_hits_total",
+    "Consultas RAG por categoria y si hubo hits",
+    ["categoria","hit"] # yes|no
+)
+MET_AGENT_LATENCY = Histogram(
+    f"{PROMETHEUS_NAMESPACE}_agent_duration_seconds",
+    "Duración del ciclo de decisión del agente (s)",
+    buckets=(.1,.2,.5,1,2,4,8,16)
+)
+MET_TOOLS_CALL_LATENCY = Histogram(
+    f"{PROMETHEUS_NAMESPACE}_tools_call_duration_seconds",
+    "Latencia del endpoint /tools/call (s)",
+    buckets=(.05,.1,.2,.5,1,2,4,8)
+)
```

Endpoint `/metrics`:

```
+from fastapi.responses import Response
+
```



```
+@app.get("/metrics")
+async def metrics():
+    if not PROMETHEUS_ENABLED:
+        return Response(status_code=404, content="disabled")
+    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)
```

Middleware simple para cronometrar `/tools/call`:

```
+@app.middleware("http")
+async def _metrics_middleware(request, call_next):
+    if request.url.path == "/tools/call":
+        import time
+        t0 = time.time()
+        resp = await call_next(request)
+        MET_TOOLS_CALL_LATENCY.observe(time.time()-t0)
+        return resp
+    return await call_next(request)
```

4.2 `orchestrator.py`

Puedes optar por solo logs estructurados o también métricas. Si usas Prometheus aquí también:

```
+from prometheus_client import Counter, Histogram, CONTENT_TYPE_LATEST, generate_latest
+PROMETHEUS_ENABLED = _getenv_bool("PROMETHEUS_ENABLED", True)
+PROMETHEUS_NAMESPACE = os.getenv("PROMETHEUS_NAMESPACE","munbot")
+
+MET_INTENT_NA = Counter(f"{PROMETHEUS_NAMESPACE}_intent_na_total","Intentos con intent n/a")
+MET_FLOW_LAT = Histogram(f"{PROMETHEUS_NAMESPACE}_orchestrate_duration_seconds","Duración de orchestrate()",buckets=(.05,.1,.2,.5,1,2,4,8))
+MET_SUCCESS_BY_CAT = Counter(f"{PROMETHEUS_NAMESPACE}_success_by_category_total","Respuestas exitosas por categoria",["categoria"])
+
+## Si el orquestador tiene FastAPI y quieres /metrics (opcional):
+try:
+    from fastapi import FastAPI
+    app # si ya existe
+    @app.get("/metrics")
+    async def metrics():
+        if not PROMETHEUS_ENABLED:
+            return Response(status_code=404, content="disabled")
+        return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)
+except Exception:
+    pass
```

Instrumenta `orchestrate()`:

```
def orchestrate(session_id: str, user_text: str, contexto, llm_client: LlmDocsClient):
+    import time
+    t0 = time.time()
+    # ... clasificación y normalización ...
+    if intent == "n/a":
+        MET_INTENT_NA.inc()
+    # ...
+    resp = <tu lógica existente>
+    MET_FLOW_LAT.observe(time.time()-t0)
+    # Marca éxito por categoría cuando corresponda (solo informativo)
+    if isinstance(resp, dict) and resp.get("respuesta"):
+        cat = categoria or "unknown"
+        MET_SUCCESS_BY_CAT.labels(cat).inc()
+    return resp
```

Paso 5 — Logs estructurados en puntos clave

5.1 `orchestrator.py` — Intent y decisión

Reemplaza/añade donde ya logueas intent:

```
-_log.info("intent_raw=%s intent_norm=%s mapped_action=%s", raw_intent, norm_intent, intent)
+_jlog(_log, "orchestrator.intent",
+    intent_raw=_redact(raw_intent),
+    intent_norm=norm_intent,
+    mapped_action=intent,
+    session=_sid_hash(session_id))
```

Cuando invocas agente:

```
+_jlog(_log, "orchestrator.agent.invoke",
+      use_agent=use_agent, categoria=categoria, session=_sid_hash(session_id))
```

En handover:

```
+_jlog(_log, "orchestrator.handover", flow=flow, session=_sid_hash(session_id))
```

5.2 gateway.py — RAG y agente

Al recuperar contexto (Fase 2 ya añadía logs):

```
_logger.info("rag: top_k=%s hits=%s top1=%s collection=%s filtro=%s",
             top_k, len(chunks), top1, collection, filtro)
+_jlog("rag.retrieve",
+      categoria=categoria, collection=collection, filtro=filtro,
+      top_k=top_k, hits=len(chunks), top1_score=top1)
+MET_RAG_HITS.labels(categoria or "unknown", "yes" if chunks else "no").inc()
```

En agent_mode:

```
_logger.info("agent.step=%s raw_out=%s", step, out[:500])
+_jlog("agent.step", step=step)
# tras parseo:
_logger.info("agent.tool_selected=%s params=%s", tool_name, params)
+_jlog("agent.decision", step=step, tool=tool_name)
# handover:
_logger.info("agent.handover flow=%s duration_ms=%s", result.get("flow"), dt)
+MET_AGENT_DECISIONS.labels("handover").inc()
+MET_HANDOVERS.labels(result.get("flow") or "unknown").inc()
+MET_AGENT_LATENCY.observe(dt/1000.0)
# final:
_logger.info("agent.final_after_tool step=%s duration_ms=%s", step, dt)
+MET_AGENT_DECISIONS.labels("final").inc()
+MET_AGENT_LATENCY.observe(dt/1000.0)
# errores:
return {"type":"error",...}
+MET_AGENT_DECISIONS.labels("error").inc()
```

Paso 6 — Trazas anonimizables (sampling)

6.1 orchestrator.py

Al inicio de orchestrate(), decide si trazas esta conversación:

```
+import random
+trace_on = (random.random() < TRACE_SAMPLING)
+if trace_on:
+    _jlog(_log, "trace.begin", session=_sid_hash(session_id))
+## Al finalizar:
+if trace_on:
+    _jlog(_log, "trace.end", session=_sid_hash(session_id))
```

Loguea cada turno (redactando):

```
+if trace_on:
+    _jlog(_log, "trace.turn",
+          session=_sid_hash(session_id),
+          user=_redact(user_text),
+          intent=norm_intent,
+          action=intent)
```

6.2 gateway.py

En agent_mode, si quieres ver mensajes que recibe el agente, **redáctalos**:

```
+if random.random() < TRACE_SAMPLING:
+    safe_msgs = [{"role":m.get("role"),"content":_redact(m.get("content") or "")} for m in messages]
+    _jlog("trace.agent.messages", messages=safe_msgs)
```

└ No vuelques respuestas completas del modelo si incluyen datos personales del usuario; redacta siempre.

Paso 7 — “Panel” mínimo con logs + jq

Si no usas Prometheus/Grafana, puedes derivar métricas básicas de los logs JSON:

- Tasa de n/a :

```
jq -r 'select(.event=="orchestrator.intent") | .mapped_action' app.log \
| awk '{total++; if($1=="n/a"){na++;}} END{print "n/a_rate=" (na/total)*100 "%"}'
```

- Handovers por flujo:

```
jq -r 'select(.event=="orchestrator.handover") | .flow' app.log \
| sort | uniq -c
```

- Top-k y hits promedio del RAG por categoría:

```
jq -r 'select(.event=="rag.retrieve") | [.categoria, .hits, .top_k] | @tsv' app.log \
| awk -F'\t' '{cnt[$1]++; hits[$1]+=$2; tk[$1]+=$3} END{for(k in cnt){print k, "avg_hits="hits[k]/cnt[k], "avg_topk="tk[k]/cnt[k]}}'
```

- Distribución de latencia del agente (p95 aproximado):

```
jq -r 'select(.event=="agent.final" or .event=="agent.handover") | .duration_ms' app.log \
| sort -n | awk ' {arr[NR]=$1} END{p=int(NR*0.95); print "p95_ms="arr[p]}'
```

Cambia los nombres de event si ajustaste etiquetas.

Checklist (Fase 7)

- Campos de log consistentes** (event, intent_raw, intent_norm, categoria, collection/filtro, top_k, hits, top1_score, tool, flow, duration_ms, session).
- Endpoint** /metrics operativo donde corresponda.
- Métricas Prometheus** incrementándose (decisiones del agente, handovers, RAG hits, latencias).
- Trazas** con PII redactada y session_id anonimizado.
- Panel mínimo** con jq o Prometheus/Grafana.

Criterios de aceptación

- Se pueden **comparar antes/después**:
 - Hit-rate** RAG por categoría (rag.retrieve hits/consultas).
 - Tasa de** n/a (contador intent_na_total o logs de mapped_action).
 - Latencia p95** del agente y de /tools/call (histogramas o jq).
 - Errores de herramienta** (agent_decisions_total{type="error"} y logs schema_invalid, tool_not_allowed, handler_timeout_or_error).
- La **PII no aparece** en los logs/trazas; en su lugar <EMAIL>, <PHONE>, <RUT>.
- Los **handovers** son visibles (métrica y logs) y el **flujo** correspondiente se activa normalmente (verificado en Fase 5).

Notas finales

- Consistencia de etiquetas**: usa siempre los mismos nombres de event y de campos para facilitar agregación.
- Coste**: JSON logging es barato; Prometheus añade overhead mínimo (~µs por operación) y es adecuado incluso en la VM Oracle de 24 GB.
- Privacidad**: mantén REDACTION_ENABLED=1 en producción; si haces debugging local, desactívalo temporalmente con cuidado.
- Escalabilidad**: si más adelante agregas StatsD, puedes duplicar los Counter/Histogram con un back-end StatsD sin cambiar el contrato de logs.

Fase 8 · Test set, canary y rollback

Objetivo: validar sin arriesgar servicio.

Cambios/Acciones

- Construir **suite** de ~50–100 prompts representativos (faq/trámite/documento/agenda/reclamo/multi-intención).
- Canary**: habilitar AGENT_MODE=1 para un % reducido (o por parámetro oculto).
- Documentar **rollback**: flags a 0 y revertir rutas al determinista.

Checklist

- ☐ Suite reproducible (script + resultados esperados)

- ☐ Plan de rollback verificado

Criterios de aceptación

- No degradación en casos claros; mejora en ambigüos/multi-intención; sin aumento significativo de latencia p95.

Instrucciones

Objetivo: validar el sistema híbrido sin arriesgar el servicio. Entregables: suite reproducible, canary gating y plan de rollback listo.

Paso 1 — Estructura del test set (repositorio)

Acción (crear carpeta y archivos):

```
tests/chat/
├─ testset.jsonl           # prompts y expectativas
├─ baseline_results.jsonl  # se genera al correr baseline
├─ canary_results.jsonl    # se genera al correr canary
├─ summary_baseline.json   # métricas (auto)
├─ summary_canary.json     # métricas (auto)
└─ run_tests.py            # runner
```

Acción (crear tests/chat/testset.jsonl con ~50–100 casos):

Formato JSONL (una línea = un caso). Mezcla faq, tramite, documento, agenda, reclamo, multi-intención, y casos edge.

```
{ "id": "faq-001", "prompt": "¿Cuál es el horario de atención del municipio?", "expected_kind": "final", "expected_categoria": "faq", "must_contain": ["horario"], "max_latency_ms": 2500 }
{ "id": "tramite-010", "prompt": "¿Qué necesito para obtener el permiso de circulación?", "expected_kind": "final", "expected_categoria": "tramite", "must_contain": ["requisitos"], "max_latency_ms": 3500 }
{ "id": "documento-005", "prompt": "¿Qué dice la ordenanza sobre ruidos molestos?", "expected_kind": "final", "expected_categoria": "documento", "must_contain": ["ordenanza"], "max_latency_ms": 3500 }
{ "id": "agenda-002", "prompt": "Quiero agendar una cita para mañana a las 10", "expected_kind": "handover", "expected_flow": "scheduler", "max_latency_ms": 3500 }
{ "id": "complaint-003", "prompt": "Quiero denunciar un bache frente a mi casa", "expected_kind": "handover", "expected_flow": "complaint", "max_latency_ms": 3500 }
{ "id": "multi-001", "prompt": "Agéndame una cita y además dime los requisitos del permiso", "expected_kind": "either", "max_latency_ms": 4000 }
{ "id": "n-a-001", "prompt": "Hola, necesito info y tramitar algo a la vez", "expected_kind": "either", "max_latency_ms": 4000 }
```

- expected_kind: final (debe responder), handover (debe transferir), either (aceptas cualquiera si es razonable).
- expected_categoria/expected_flow: opcionales según el caso.
- must_contain: validación blanda de contenido.
- Ajusta max_latency_ms según tu VM.

Paso 2 — Runner reproducible (baseline vs canary)

Acción (crear tests/chat/run_tests.py):

```
import json, time, os, statistics, sys
import requests

ORCH_URL = os.getenv("ORCH_URL", "http://localhost:5000/orchestrate")
HEADERS = {"Content-Type": "application/json"}
# Canary por header (si lo activas en el endpoint HTTP del orquestador):
CANARY_HEADER_KEY = os.getenv("AGENT_CANARY_HEADER_KEY", "x-agent-canary")
CANARY_HEADER_VAL = os.getenv("AGENT_CANARY_HEADER_VAL", "1")

def call_orchestrator(prompt, canary=False):
    payload = {"pregunta": prompt}
    headers = HEADERS.copy()
    if canary:
        headers[CANARY_HEADER_KEY] = CANARY_HEADER_VAL
    t0 = time.time()
    r = requests.post(ORCH_URL, json=payload, headers=headers, timeout=15)
    dt = int((time.time() - t0) * 1000)
    r.raise_for_status()
    return r.json(), dt

def evaluate_case(case, resp, latency_ms):
    eid = case["id"]
    expected_kind = case.get("expected_kind", "either")
    expected_flow = case.get("expected_flow")
```

```

must = case.get("must_contain", [])
maxlat = case.get("max_latency_ms", 4000)

ok_latency = latency_ms <= maxlat
kind = "unknown"
flow = None

# Convención de respuesta del orquestador:
if isinstance(resp, dict) and resp.get("respuesta"):
    kind = "final"
elif isinstance(resp, dict) and (resp.get("handover") or resp.get("flow")):
    kind = "handover"
    flow = resp.get("handover") or resp.get("flow")

ok_kind = (expected_kind == "either") or (kind == expected_kind)
ok_flow = True
if expected_flow and kind == "handover":
    ok_flow = (flow == expected_flow)

ok_content = True
if kind == "final" and must:
    text = (resp.get("respuesta") or "").lower()
    ok_content = all(m.lower() in text for m in must)

return {
    "id": eid, "ok": (ok_kind and ok_flow and ok_content and ok_latency),
    "kind": kind, "flow": flow, "latency_ms": latency_ms,
    "ok_kind": ok_kind, "ok_flow": ok_flow, "ok_content": ok_content, "ok_latency": ok_latency
}

def summarize(results):
    latencies = [r["latency_ms"] for r in results]
    p95 = round(sorted(latencies)[int(0.95*len(latencies))-1], 1) if latencies else 0.0
    ok_rate = round(100.0 * sum(1 for r in results if r["ok"]) / max(1, len(results)), 2)
    kinds = {}
    for r in results:
        kinds[r["kind"]] = kinds.get(r["kind"], 0) + 1
    return {
        "total": len(results),
        "pass_rate_pct": ok_rate,
        "p95_ms": p95,
        "by_kind": kinds
    }

def run(path, outfile, canary=False):
    results = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            if not line.strip(): continue
            case = json.loads(line)
            resp, dt = call_orchestrator(case["prompt"], canary=canary)
            res = evaluate_case(case, resp, dt)
            results.append({"case": case, "result": res, "raw": resp})
            print(f"[{case['id']}] {res['ok']} kind={res['kind']} flow={res['flow']} dt={res['latency_ms']}ms")
    with open(outfile, "w", encoding="utf-8") as f:
        for r in results:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")
    summary = summarize([r["result"] for r in results])
    with open(outfile.replace(".jsonl", "_summary.json"), "w", encoding="utf-8") as f:
        json.dump(summary, f, ensure_ascii=False, indent=2)
    print("SUMMARY:", json.dumps(summary, ensure_ascii=False))
    return summary

if __name__ == "__main__":
    path = sys.argv[1] if len(sys.argv)>1 else "tests/chat/testset.jsonl"
    mode = sys.argv[2] if len(sys.argv)>2 else "baseline" # baseline|canary
    outfile = "tests/chat/baseline_results.jsonl" if mode=="baseline" else "tests/chat/canary_results.jsonl"
    canary = (mode=="canary")
    run(path, outfile, canary=canary)

```

Uso:

```

# Baseline (AGENT_MODE=0)
python tests/chat/run_tests.py tests/chat/testset.jsonl baseline

# Canary (AGENT_MODE=1 y canary gating activo)
python tests/chat/run_tests.py tests/chat/testset.jsonl canary

```


Paso 3 — Canary gating (ratio + header forzado)

Acción (añadir flags a .env.example):

```
+# === Canary rollout ===
+AGENT_CANARY_RATIO=0.2          # 20% de sesiones/peticiones usan agente
+AGENT_CANARY_HEADER_KEY=x-agent-canary
+AGENT_CANARY_HEADER_ON=1        # si el header llega con este valor, fuerza agente
```

Acción (en el endpoint HTTP del orquestador, donde recibes la petición):

Añade detección de header y marcación en el contexto de la sesión.

```
# ejemplo FastAPI
from fastapi import Request

@app.post("/orchestrate")
async def orchestrate_route(req: Request):
    body = await req.json()
    pregunta = body.get("pregunta","")
    headers = req.headers
    force_canary = headers.get(os.getenv("AGENT_CANARY_HEADER_KEY","x-agent-canary")) ==
os.getenv("AGENT_CANARY_HEADER_ON","1")
    # guarda flag en el contexto de la sesión (o pásalo como arg) y llama a orchestrate()
```

Acción (en orchestrator.py, dentro de orchestrate()):

```
import random
AGENT_CANARY_RATIO = float(os.getenv("AGENT_CANARY_RATIO","0.2"))
AGENT_CANARY_HEADER_KEY = os.getenv("AGENT_CANARY_HEADER_KEY","x-agent-canary")

def _should_use_canary(contexto, session_id, force_canary: bool) -> bool:
    if force_canary:
        return True
    # sampling por sesión (estable) — usa hash del session_id para consistencia
    h = hash(session_id) % 1000
    return (h / 1000.0) < AGENT_CANARY_RATIO

# En la política de Fase 5:
use_agent = use_agent or _should_use_canary(contexto, session_id, force_canary)
```

Resultado: puedes forzar canary por header (runner ya lo soporta) y/o habilitarlo para un % de sesiones.

Paso 4 — Correr baseline y canary + comparar

Acción (ejecución):

```
# 1) Baseline
export AGENT_MODE=0
docker compose up -d --build
python tests/chat/run_tests.py tests/chat/testset.jsonl baseline
cat tests/chat/summary_baseline.json

# 2) Canary
export AGENT_MODE=1
export AGENT_CANARY_RATIO=0.2
docker compose up -d --build
python tests/chat/run_tests.py tests/chat/testset.jsonl canary
cat tests/chat/summary_canary.json
```

Acción (comparación mínima con jq):

```
jq -s '[][0] as $a | .[1] as $b | {
  baseline_pass: $a.pass_rate_pct,
  canary_pass: $b.pass_rate_pct,
  baseline_p95: $a.p95_ms,
  canary_p95: $b.p95_ms,
  by_kind: {baseline: $a.by_kind, canary: $b.by_kind}
}]' tests/chat/summary_baseline.json tests/chat/summary_canary.json
```

Umbral es sugeridos (rechaza canary si se incumplen):

- pass_rate canary ≥ baseline – 2 p.p.

- p95_ms canary ≤ baseline × 1.15
- En casos multi-intención, canary debe superar baseline (≥ +10% de acierto “razonable”).

Paso 5 — Integración con telemetría (Fase 7)

Mientras corres pruebas, observa:

- /metrics: *_agent_decisions_total*, *_handovers_total*, **_tools_call_duration_seconds*.
- Logs JSON: eventos *orchestrator.intent*, *rag.retrieve*, *agent.decision*, *orchestrator.handover*.

Consultas rápidas (jq):

```
# tasa de n/a baseline vs canary (si guardaste logs separados)
jq -r 'select(.event=="orchestrator.intent") | .mapped_action' baseline.log | awk '{t++; if($1=="n/a") n++} END{print "baseline_na="n/t}'
jq -r 'select(.event=="orchestrator.intent") | .mapped_action' canary.log | awk '{t++; if($1=="n/a") n++} END{print "canary_na="n/t}'
```

Paso 6 — Rollback documentado y verificado

Plan (documentar en RUNBOOK.md):

Flags a 0:

```
export AGENT_MODE=0
export RAG_CATEGORY_AWARE=0 # si necesitas desactivar selección por categoría
docker compose up -d --build
```

Deshacer canary:

```
export AGENT_CANARY_RATIO=0
```

- Verificación post-rollback:
 - Correr 10–20 casos del test set; la salida debe coincidir con baseline_results.jsonl para esos casos.
 - Ver logs: ausencia de eventos *agent.** y *handovers* solo si los generaba el ruteo determinista.

Checklist (Fase 8)

- Suite reproducible (tests/chat/testset.jsonl + run_tests.py) con resúmenes.
- Canary gating por % y por header de forzado.
- Comparación baseline vs canary (pass-rate, p95, by_kind).
- Runbook con rollback (flags a 0 + verificación rápida).

Criterios de aceptación

- No degradación en casos claros (faq/trámite/documento simples) — pass_rate no peor que baseline – 2 p.p.
- Mejora en ambiguos/multi-intención — canary supera baseline en esos ítems.
- Sin aumento significativo de p95 — ≤ 15% sobre baseline.
- Rollback probado — al poner flags 0, resultados vuelven a baseline y desaparecen eventos *agent.**.

Notas finales

- Mantén el test set vivo: añade casos reales fallidos como regresiones para próximas iteraciones.
- Para más fineza, separa el test set en buckets (faq, tramite, documento, agenda, complaint, multi), y reporta métricas por bucket.
- Si sube la latencia en canary, ajusta AGENT_MAX_TOOL_CALLS=1, top_k, o prompts, y repite el ciclo.

Fase 9 · Documentación y runbook

Objetivo: mantenibilidad y operación.

Entregables

- README del agente: flags, endpoints, formato `<CALL ...>`, límites.
- Runbook: cómo activar/desactivar, dónde mirar logs, métricas clave, troubleshooting.
- Especificación de herramientas (parámetros obligatorios/opcionales, ejemplos).

Checklist

- ☐ README actualizado
- ☐ Runbook con escenarios de fallo

Criterios de aceptación

- Un tercero puede operar y depurar sin asistencia.

PROMPT

Hallazgos (debilidades actuales)

1. **#Control** de alucinaciones incompleto
Aunque indicas “usa EXCLUSIVAMENTE el CONTEXTO”, no especificas qué hacer cuando falta evidencia para algún apartado (riesgo: el LLM completa huecos). Esto aplica en los tres prompts por categoría.
2. **#Citas** duplicadas o inconsistentes
Los prompts piden “1–2 fuentes”, pero tu backend ya construye y adjunta referencias a partir de los chunks recuperados. Eso puede generar duplicidad o divergencias entre lo que “cita” el modelo y lo que arma el servicio.
3. **#Ausencia** de política “no hay respuesta” estandarizada
No hay una instrucción explícita tipo: “si el CONTEXTO no basta, responde ‘Información no disponible en las fuentes’ y sugiere un siguiente paso”. Hoy ese criterio existe en el gateway como fallback por score, pero no está reforzado en el prompt
4. **#Ambigüedad** inter-municipal y vigencia normativa
El prompt DOCUMENTO no instruye cómo actuar si hay versiones/ordenanzas múltiples o fechas de vigencia distintas; tampoco pide explicitar “municipio/fecha” con precisión, algo clave en normativa.
5. **#Estructura** “sección vacía” en TRÁMITE
Pides secciones fijas (Requisitos, Pasos, Documentos, Plazos) pero no indicas qué hacer si el CONTEXTO no trae, por ejemplo, “Plazos”. El modelo podría inventarlos.
6. **#Criterios** de selección de evidencia no explicitados en el prompt
En la fase 2 defines colecciones y filtros por categoría, pero el prompt no fija reglas de priorización cuando hay múltiples fragmentos (p.ej., “elige el fragmento con score más alto/top-1”). Lo resuelves por código (logging/top1), no por instrucción al LLM.
7. **#No** instruyes manejo de conflictos entre fuentes
Si dos fragmentos dicen cosas parcialmente distintas (muy común en normativa y FAQs desactualizadas), el prompt no indica “reporta el conflicto, prioriza la fuente más reciente/municipal oficial”.
8. **#Estilo** de salida variable
FAQ pide “breve y clara”, TRÁMITE y DOCUMENTO piden listas, pero no fijas longitud, ni un cierre consistente (p.ej., “Resumen de 1–2 frases + secciones”). Eso afecta previsibilidad para frontends/reportes.

Prompts mejorados (drop-in)

Copia estas versiones donde defines las plantillas de la Fase 2. Mantienen tus objetivos, pero agregan salvaguardas y consistencia. (Sustituye los bloques de PROMPT_* mostrados en tu documento).

PROMPT_FAQ (reemplazo)

```
Responde en español, de forma breve y clara (máx. 120 palabras), usando EXCLUSIVAMENTE la información del CONTEXTO.
- Si el CONTEXTO no contiene información suficiente para responder, di textualmente: “Información no disponible en las fuentes”.
- No inventes datos ni asumas hechos no presentes en el CONTEXTO.
- Si hay múltiples fragmentos, prioriza la evidencia más específica y reciente indicada en el CONTEXTO.

CONTEXTO:
{contexto}

PREGUNTA:
{pregunta}

RESPUESTA:
```

PROMPT_TRAMITE (reemplazo)

```
Responde en español con la siguiente estructura, usando EXCLUSIVAMENTE el CONTEXTO. No inventes datos.
- Resumen (1-2 frases)
- Requisitos: (si no hay en CONTEXTO, escribe “No informado en las fuentes”)
- Pasos: (idem)
- Documentos: (idem)
- Plazos: (idem)

Reglas:
- Si el CONTEXTO es insuficiente para algún apartado, escribe “No informado en las fuentes” en ese apartado.
- Si hay múltiples fragmentos, prioriza la información más específica y consistente; evita contradicciones.
- Si el CONTEXTO no permite responder, di: “Información no disponible en las fuentes”.
```

```
CONTEXT0:
{contexto}

PREGUNTA:
{pregunta}

RESPUESTA:
```

PROMPT_DOCUMENTO (reemplazo)

```
Genera una respuesta en español, con foco normativo, usando EXCLUSIVAMENTE el CONTEXT0. No inventes datos.
Estructura:
- Resumen normativo (1-2 frases)
- Artículo/Cláusula relevante (si aplica; si no, “No informado en las fuentes”)
- Vigencia/Fecha (si aplica; si no, “No informado en las fuentes”)
- Referencia (nombre/identificador del documento según CONTEXT0)

Reglas:
- Si hay versiones/fechas diferentes en el CONTEXT0, indica la más reciente y menciona la discrepancia.
- Si el CONTEXT0 no permite responder, di: “Información no disponible en las fuentes”.
- No incluyas contenido que no esté en el CONTEXT0.

CONTEXT0:
{contexto}

PREGUNTA:
{pregunta}

RESPUESTA:
```

Notas de implementación rápidas

- **Evitar doble cita:** ya que el backend arma `referencias` con los chunks recuperados, no pidas “1–2 fuentes” al modelo en el texto final, para no duplicar o desalinear. (Tu doc actual pide fuentes dentro del prompt; mejor retirar esa línea o, si la conservas, exige un formato que mapee a las referencias del backend, p.ej. `[[id]]` del chunk).
- **Consistencia categoría→prompt:** ya definiste el selector de prompt por `categoria`. Mantén el mapeo tal cual (faq→FAQ, tramite→TRAMITE, documento→DOCUMENTO).
- **Conflictos:** incorpora en los prompts (como arriba) la regla de “reportar discrepancias y priorizar lo más reciente” para DOCUMENTO/FAQ; complementa con logging de `top1` que ya propones.
- **Sin respuesta:** estandariza la frase “Información no disponible en las fuentes” en prompts y maneja en el gateway el caso sin-chunks para que el texto sea coherente con tu fallback.

Pruebas sugeridas (rápidas)

1. **Sección ausente en TRÁMITE:** pregunta por un trámite cuyo contexto no tenga “Plazos”: la salida debe mostrar “No informado en las fuentes” en ese apartado, no inventar.
2. **Normativa con fechas distintas:** DOCUMENTO con dos versiones; debe escoger la más reciente y señalar la discrepancia.
3. **FAQ sin evidencia:** debe responder exactamente “Información no disponible en las fuentes”.

(Estos casos se suman a tus smoke tests de categoría y logs `top1/collection/filtro`.)

MEJORAS

prompts/tests del clasificador intent-regression.json

Objetivos

- Recolectar frases reales que están fallando (por ejemplo, capturadas en la auditoría).
- Añadirlas a un pequeño conjunto de regresión automática: invoca el endpoint del clasificador y comprueba que devuelve documento, agenda, etc.
- Si el LLM se equivoca, ajusta el prompt o añade ejemplos de few-shot.
- Mantén esos casos en tests (pytest) que llamen al clasificador con un mock controlado.

1. Recolectar frases reales

- Usa la auditoría que ya deja el orquestador (`mcp-core/orchestrator.py:387`) para localizar entradas donde `intent_norm` quedó en n/a o en un intent equivocado.

- Guarda esas frases (junto con el intent deseado, p. ej. ask_document) en un archivo sencillo, por ejemplo tests/data/intent_regression.json.

Formato sugerido:

```
[
{
    "texto": "Necesito saber requisitos para permiso de aterrizaje",
    "esperado": "ask_document",
    "intent_detectado": "ask_document",
    "status": "validated",
    "source": "manual"
},
{
    "texto": "Quiero reservar una hora para el jueves",
    "esperado": "init_scheduler",
    "intent_detectado": "init_scheduler",
    "status": "validated",
    "source": "manual"
}
]
```

2. Exponer un punto único para probar el clasificador El método que se usa en producción ya está en mcp-core/clients/llm_docs.py:62. Para los tests conviene que ese método sea el que se invoque, así el código de aplicación y el de prueba hablan el mismo “idioma”.

3. Crear un test de regresión Añade un archivo (por ejemplo tests/test_intent_classifier_regression.py) que cargue el JSON y llame a LlmDocsClient.classify_intent. Durante el test no queremos invocar realmente al servicio HTTP, así que monkeypatcheamos httpx.post para devolver la estructura que esperas del LLM.

Esquema:

```
import json
import os
import httpx
from unittest.mock import patch
from mcp_core.clients.llm_docs import LlmDocsClient

DATA_PATH = os.path.join(os.path.dirname(__file__), "data", "intent_regression.json")

def fake_http_post(url, json=None, headers=None, auth=None, timeout=None):
    texto = json["texto"]
    # devolvé aquí la estructura que el LLM debería emitir según tus notas
    ...

@patch("httpx.post", side_effect=fake_http_post)
def test_classify_intent_regression(mock_post):
    client = LlmDocsClient("http://llm_docs-mcp:8000/tools/call")
    with open(DATA_PATH) as fh:
        casos = json.load(fh)

    for caso in casos:
        resp = client.classify_intent(caso["texto"])
        assert resp["intent"] == caso["esperado"]
```

- El fake_http_post puede leer de un diccionario precargado con la intención esperada por cada frase.
- Si alguno falla, sabrás que el output del LLM (o tu prompt) ya no está alineado con lo que la aplicación requiere.

4. Ajustar el prompt / añadir few-shot Si el test falla, reabre el prompt principal en services/llm_docs-mcp/prompts (busca el que use el clasificador, normalmente doc-classify_intent_llm.txt). Añade ejemplos concretos (“few-shot”) que cubran los casos conflictivos. Luego vuelve a correr el test.

5. Versionar y repetir

- Deja el archivo de datos (tests/data/...) y el test en el repo: así, cualquier cambio futuro al prompt o al modelo se valida automáticamente.
- Si recopilas más frases problemáticas, agrégalas al JSON y el test crecerá con tu cobertura real.

intent registry configurable

1. Define los “anclas” de intent

- Construye un pequeño catálogo de frases representativas por intent (agenda, init_complaint, ask_document, saludo, etc.).

```
[
  {"intent": "init_scheduler", "texto": "quiero agendar una cita"},
  {"intent": "init_scheduler", "texto": "reservar hora en la muni"},
  {"intent": "init_complaint", "texto": "quiero presentar un reclamo"},
  {"intent": "ask_document", "texto": "informacion permiso de circulación"},
  ...
]
```

- Guárdalo en `mcp-core/config/intent_similarities.json` o similar.

2. Genera embeddings

- Decide una librería/vectorizador (sentence-transformers, tu propia función).
- Precalcula el embedding de cada frase ancla y cárgalo al iniciar mcp-core (o en un módulo separado). Mantén un array vectors y un array paralelo labels.

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
vectors = model.encode([item["texto"] for item in catalogo])
labels = [item["intent"] for item in catalogo]
```

3. Añade el rescate en classify_intent_remotely

- Después de consultar al LLM, si devuelve `faq/n/a`, calcula el embedding de la frase actual y compara contra tus vectores.
- Usa `cosine_similarity` o `sklearn.metrics.pairwise`. Recupera el top-1 (o top-k) y, si la similitud supera un umbral (ej. 0.55), sustituye la intención (agenda, init_complaint, etc.) antes de que `_process_intent_classification` mapee el flujo.

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
vectors = model.encode([item["texto"] for item in catalogo])
labels = [item["intent"] for item in catalogo]
```

4. Considera el contexto de la sesión

- Si quieres reforzar con “intentos recientes”, guarda en el contexto (`context_manager`) los últimos intents confirmados. Si la similitud con el último intent supera otro umbral menor, úsalo como refuerzo (`prefer_last_intent=True`).
- Otra opción rápida: si el vector se parece a una frase ya vista esa sesión (p. ej. guardada en `_record_intent_sample`), toma ese intent.

5. Logs y telemetría

- Loguea cada rescate (`intent.similarity_override`) con la similitud y la etiqueta adoptada. Así sabrás si la heurística se activa demasiado.
- Añade un contador Prometheus si usas métricas (`intent_similarity_rescue_total`).

6. Testea

- Crea tests unitarios donde el LLM está mockeado para devolver `faq`, y verifica que frases como “reservar cita” desencadenan el rescate y terminan en `init_scheduler`.
- Añade casos al dataset de regresión (`intent_regresion.json`) para asegurar que este fallback se comporte como esperas.

7. Ajusta catálogo y umbral

- Empieza con 3–5 ejemplos por intent y un umbral conservador (0.55–0.6).
- Si observas falsos positivos, sube el umbral o agrega frases negativas (`intent: "faq"`) para balancear.

Así reduces dependencia de palabras exactas: el LLM sigue siendo la fuente principal, pero si se equivoca, el “plan C” por similitud rescata frases comunes sin tener que expandir el prompt cada semana.

Implementar un rescate por similitud

1. Catálogo de ejemplos

- Crea/edita `mcp-core/config/intent_similarities.json` con frases representativas por intent (agenda, reclamo, ask_document, smalltalk, etc.).
- Cada entrada lleva al menos `{"intent": "...", "texto": "..."}` y puedes añadir otras columnas si lo deseas.

2. Dependencias de embeddings

- Asegúrate de tener sentence-transformers (y scikit-learn si usarás cosine_similarity) en `mcp-core/requirements.txt`.

- Reconstruye o instala en tu entorno.

3. Módulo de similitud

- Añade mcp-core/intent_similarity.py con funciones:
 - `_load_catalog()` para leer el JSON.
 - `_ensure_model()` que inicializa `SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")` una sola vez.
 - `load_embeddings()` que devuelve vectors (`np.ndarray`) + labels.
 - `encode_text(text)` para obtener el embedding normalizado de la frase a evaluar.

4. Preparar el orquestador

- Importa los helpers y `cosine_similarity` en `mcp-core/orchestrator.py`.
- Durante la inicialización, carga los vectores una vez:

```
SIM_VECTORS, SIM_LABELS = load_embeddings()
SIMILARITY_THRESHOLD = float(os.getenv("INTENT_SIMILARITY_THRESHOLD", "0.55"))
```

- Añade una función `_similarity_rescue(user_text)` que:
 1. Usa `encode_text` para la frase actual.
 2. Calcula la similitud con `SIM_VECTORS`.
 3. Selecciona el intent del top-1 si la similitud $\geq$ umbral y lo devuelve (junto con el score).

5. Aplicar el rescate en la clasificación

- Dentro de `classify_intent_remotely`, luego de llamar al LLM:
 - Si la respuesta es `faq/n/a`, llama a `_similarity_rescue`.
 - Si hay match, sobrescribe `result["intent"]` (y opcionalmente `sub_intent`, `confidence`) con el intent rescatado antes de seguir por el flujo normal.

6. Configuración y observabilidad

- Expón `INTENT_SIMILARITY_PATH` y `INTENT_SIMILARITY_THRESHOLD` en `.env/docker-compose.yml` para que puedas cambiar catálogo y umbral sin tocar código.
- Agrega logs/métricas (`intent.similarity_rescue`) y maneja excepciones por si el modelo no carga.

7. Validación

- Añade tests que mockeen el LLM para devolver `faq` y verifiquen que frases como “reservar una cita” terminan en `init_scheduler`.
- Poblá el JSON con más ejemplos y ajustá el umbral si ves falsos positivos/negativos.

Monitoreo y alertas

1. Emitir el evento

- Dentro de `classify_intent_remotely`, justo antes de retornar desde `_heuristic_classify`, agrega un log estructurado (`_jlog`) con un evento tipo `intent.fallback`, guardando `utterance`, `trace_id` (si lo hay) y quizá el intent heurístico.
- Puedes añadir también un Counter de Prometheus (`intent_fallback_total`) para cuantificar cuántas veces llegas al heurístico.

2. Etiquetar la fuente

- En el diccionario final que retorna la función, añade una bandera (`result["source"] = "heuristic"`) si llegaste a `_heuristic_classify`. Esto facilitará agregar métricas más adelante.

3. Exportar a Prometheus

- Declara un Counter global en `orchestrator.py` (ej. `HEURISTIC_FALLBACK_COUNTER = Counter("intent_heuristic_total", "...", registry=PROM_REGISTRY)`).
- Incrementa el contador cuando uses el heurístico.

4. Dashboard

- En tu stack de observabilidad (Grafana, etc.), construye:
 - Gráfico de series temporales: total de `intent_heuristic_total` vs. total de clasificaciones (`munbot_requests_total` o similar).
 - Indicador “ratio heurístico” = `intent_heuristic_total / munbot_requests_total * 100`.
- Configura alertas en Grafana/Prometheus: si el ratio supera cierto umbral (ej. 5% durante 15 min), dispara una notificación (correo, Slack).

5. Logs persistentes

- Opcional: integra con un SIEM/ELK. Filtra los logs `intent.fallback` y almacénalos: permiten revisar rápidamente qué frases causaron el fallback.

6. Política de acción

- Define un playbook: cuando el ratio supera 5–10%, se revisan los dashboards, se extraen ejemplos recientes directamente desde los logs, se curan en `intent_regresion.json`, y se ajusta `prompt/alias` antes de redeploy.

Persistir contexto semántico

1. **Registrar una sección para entidades en el contexto** En context_manager.py, agrega métodos para guardar y leer un diccionario de entidades por sesión:

```
def set_entities(self, session_id: str, entities: dict) -> None:
    ctx = self.get_context(session_id)
    ctx["entities"] = entities
    self.redis_client.set(...)

def get_entities(self, session_id: str) -> dict:
    return self.get_context(session_id).get("entities", {})
```

2. **Capturar lo que devuelve el LLM** En classify_intent_remotely, además de intent, el LLM ya envía entities. Antes de cualquier rescate/heurística:

```
entities = result.get("entities") or {}
context_manager.set_entities(session_id, entities)
```

Hazlo en el punto donde todavía tienes la sesión a mano (p. ej. en handle_turn, después de obtener classification).

3. **Normalizar cuando el heurístico actúe** Si terminas usando _heuristic_classify, asegúrate de conservar las entities que guardaste antes. No las sobrescribas con {}.
4. **Usar ese contexto al pasar a RAG** Cuando llames a handle_document_query, añade las entidades guardadas:

```
entities = context_manager.get_entities(session_id)
if not classification_entities:
    classification["entities"] = entities
```

Así, aunque la intención sea faq, el flujo de RAG recibe tramite=permiso de circulación y puede construir la consulta correcta.

5. **Limpiar cuando cambie el tema o termine el turno** Tu función _wants_change_topic o el cierre de sesión (despedida) deberían llamar a context_manager.set_entities(session_id, {}) para que la siguiente consulta no herede entidades viejas.

6. **Pruebas**

- Simula que el LLM devolvió entities={"tramite": "permiso de circulación"} pero intent="faq".
- Verifica que, tras almacenar las entidades, handle_document_query reciba el valor al fall back y que RAG lo use.
- Añade un caso de regresión en tus tests para garantizar que la entidad se persiste y se limpia cuando corresponde.

Auditoría del Chatbot MunBoT – Diagnóstico y Mejoras Propuestas

1. Problemas en el flujo RAG actual (clasificador de intención y intent: "n/a")

El flujo **RAG** actual de MunBoT presenta obstáculos que explican por qué el bot no responde bien a consultas informativas como “*permiso de aterrizaje*” o “*principios medioambientales del municipio*”. En concreto:

- **Clasificación imprecisa o vacilante:** El clasificador híbrido (alias + solapamiento de tokens) puede fallar cuando la consulta del usuario no coincide exactamente con alias predefinidos o difiere levemente en terminología. Por ejemplo, “*principios medioambientales del municipio*” no coincide con el alias “*principios ambientales*” (el término “*medioambientales*” no figura en los alias) y sólo comparte algunas palabras en común. El motor de intents calcula un puntaje de coincidencia (Jaccard + bonificaciones) que, si queda bajo el umbral (~0.18 por defecto, adaptativo según longitud [GitHub](#) [GitHub](#)), marca la intención como “**n/a**” (no reconocida) [GitHub](#). Esto ocurre incluso cuando la información sí existe en la base, provocando que el bot no active la búsqueda RAG.
- **Manejo de intent: "n/a" :** Actualmente, si tras clasificación (y una leve “rescate” por similitud) la intención sigue siendo “*n/a*”, el orquestador no intenta RAG sino que inmediatamente activa un **fallback**: el bot pide reformulación genérica (“*Lo siento, no he entendido tu consulta...*”) [GitHub](#). Esto significa que consultas válidas pero fuera de vocabulario (o que el clasificador no supo categorizar) terminan sin respuesta informativa. **No hay un fallback-to-RAG; el sistema no intenta una búsqueda abierta en la base de conocimiento cuando la intención es desconocida, desaprovechando potencial para responder** . **Este manejo agresivo del “n/a” eleva la tasa de fallbacks innecesarios** .
- **Detección de intención insuficientemente robusta:** El clasificador depende de alias exactos y solapamiento de palabras, lo cual es frágil ante sinónimos, variantes ortográficas o preguntas generales. Por ejemplo, la palabra “*permiso*” sí existe como alias (parte de “permiso aterrizaje”), pero la consulta “*permiso de aterrizaje*” contiene la preposición “de” que impidió un match directo de alias (se evalúa substring literal [GitHub](#)). En este caso el clasificador recurre al solapamiento (coinciden “permiso” y “aterrizaje”, dos de tres palabras), obteniendo un score moderado que superó el umbral dinámico (para 3 palabras ≈0.12 [GitHub](#)). **Así clasificó “permiso de aterrizaje” como intent “tramite”, pero con sub-intent posiblemente incorrecto (“requisitos” porque el fragmento top coincidió con requisitos de ese trámite).** Una clasificación incierta como ésta puede resultar en flujo incompleto o respuesta parcial.

- **Contexto seleccionado sin respuesta (caso preguntas genéricas):** Aun cuando el clasificador acierta que la consulta es sobre un *trámite* o *documento*, el diseño actual **no entrega una respuesta si la pregunta no especifica qué dato quiere**. El código del orquestador comprueba si el usuario mencionó un trámite/doc concreto sin un detalle específico, y en ese caso guarda el documento en contexto pero **pide aclaración al usuario en vez de responder** [GitHub](#) [GitHub](#). Por ejemplo, con “*permiso de aterrizaje*” detectó el trámite “PermisoAterrizaje” pero, al no hallar palabras clave como “*requisitos*”, “*horario*”, etc., el bot responde: “Para ayudarte con el trámite 'permiso de aterrizaje', necesito saber qué información específica buscas (por ejemplo, requisitos, horario, costo).” en lugar de proporcionar directamente información. Lo mismo ocurre con “principios medioambientales del municipio”_: el sistema identificó probablemente el documento de normativa ambiental, pero al interpretar que la pregunta es muy general, preguntó qué aspecto específico quiere el usuario. En código, esta lógica `needs_specific` impide la llamada RAG en la primera pregunta genérica (los tests lo confirman: no se invoca microservicio de búsqueda en consultas genéricas [GitHub](#) [GitHub](#)). **Esta decisión de diseño conduce a que el bot parezca no responder**, pues devuelve una pregunta en vez de información, frustrando al usuario.

En suma, **las causas del mal desempeño** se centran en: (1) un clasificador de intención con reglas rígidas que a veces devuelve “n/a” o categoriza mal preguntas válidas (sea por umbrales mal calibrados o falta de sinónimos), y (2) una política de flujo que, ante intenciones genéricas (intent *documento/trámite* sin slot específico), opta por solicitar aclaración en vez de intentar una respuesta con los datos disponibles. Esto último, si bien buscaba evitar respuestas imprecisas, termina empeorando la experiencia al no proveer ningún contenido útil inicialmente.

2. Alternativas y mejoras al pipeline RAG con LangChain

Para mejorar el pipeline RAG actual, vale evaluar componentes de LangChain que podrían simplificar la orquestación o hacerla más robusta. A continuación se describen varias opciones (LangChain y herramientas afines), con sus **ventajas** y **desventajas** para este caso de uso:

- **LangChain** `LangGraph` **: Es un módulo que permite modelar flujos de conversación y decisiones como un grafo o cadena declarativa. Podría replicar la lógica del orquestador (intención → ruta de herramienta) de forma más estructurada. Ventajas: **facilita visualizar y modificar la secuencia de pasos del agente; permite flujos más complejos (ej. múltiples llamadas a herramientas o sub-agentes) sin escribir toda la lógica de bajo nivel. Podría hacer más sencilla la adición de pasos como verificación de atribución o reformulación con LLM**. Desventajas:** está en fase relativamente nueva, requeriría reescribir gran parte del pipeline en términos de LangChain (curva de aprendizaje). La flexibilidad absoluta del código Python podría perderse si LangGraph no soporta alguna lógica específica actual. Además, introducir LangChain añade overhead y posibles pérdidas de control fino sobre rendimiento y concurrencia (importante dado el entorno de CPU y baja latencia). En resumen, LangGraph aporta arquitectura modular pero a costa de complejidad adicional y dependencia de una librería externa en evolución.
- **Enrutador de LLM (`LLMRouter` de LangChain):** Consiste en usar un modelo de lenguaje para dirigir dinámicamente cada pregunta al “pipeline” adecuado. En vez de un clasificador fijo, se entrena o indica al LLM que escoja entre rutas (por ejemplo: `agenda`, `reclamo`, `tramite/documento`, `smalltalk`). LangChain provee `MultiPromptChain` y `RouterChain` que implementan este patrón. **Ventajas:** Un enrutador con LLM podría aprovechar el contexto completo de la pregunta para decidir mejor que reglas rígidas. Por ejemplo, dado “¿Cómo obtener permiso de aterrizaje?”, el router LLM entendería que es un trámite, evitando falsos “n/a”. También permite criterios más difusos (el LLM puede inferir la intención aunque falten palabras exactas). En general, bien diseñado aumentaría la precisión sin depender de umbrales manuales. Incluso se puede entrenar un modelo ligero específico para esta clasificación. Otro beneficio es que se integraría de manera natural con LangChain, unificando la fase de clasificación y la ejecución de la herramienta. **Desventajas:** Implica llamar a un LLM *extra* para cada turno, lo que en CPU añade latencia. Con el modelo Qwen-2.5B local, esa llamada adicional (aunque corta, de ~1 palabra de salida) puede costar varios cientos de ms. Además, el router LLM no es infalible: habría que afinar muy bien el prompt de routing y posiblemente proporcionar ejemplos, sino podría enviar consultas por la ruta equivocada. Dado que ya existe un clasificador basado en reglas + LLM auxiliar, cambiar a un LLMRouter puro conlleva reimplementar lógica ya probada. En síntesis, mejora flexibilidad y posiblemente la **cobertura** de casos, pero añade consumo de CPU y complejidad en control de errores.
- **LCEL (LangChain Expression Language):** Es un lenguaje declarativo para definir cadenas de llamadas y condiciones de forma compacta (p. ej. en YAML o pseudo-código). Permitiría expresar la lógica del orquestador (clasificar intent, luego if/else según intent llama a tal herramienta) en un formato de configuración en lugar de código Python imperativo. **Ventajas:** La lógica de alto nivel sería más fácil de ajustar sin depurar código; por ejemplo, se podría cambiar el umbral de confianza o agregar un paso de validación simplemente editando una expresión. Esto acelera experimentación con distintos pipelines (se podría tener una versión con agente ReAct vs una secuencial y conmutar). Además, LCEL integrado con Guardrails permite especificar validaciones directamente en la definición del flujo. **Desventajas:** Requeriría adaptar todo el flujo actual a esta sintaxis; cualquier comportamiento especial no soportado demandará complementarlo con código Python igualmente. Puede dificultar la depuración, ya que los errores en un flujo declarado pueden ser menos evidentes que un traceback en código. En un sistema ya construido, la ganancia de migrar a LCEL es discutible: es más útil al diseñar un flujo desde cero o uno muy dinámico. En este caso concreto, donde el pipeline tiene ramificaciones bien definidas, LCEL aportaría más legibilidad que funcionalidad nueva, con el riesgo de introducir errores sutiles si la traducción no es exacta.
- **PromptLayer (Observabilidad de prompts):** Es una plataforma para gestionar y versionar prompts, registrar todas las llamadas a LLM y sus respuestas, etc. Integrarla en LangChain (o manualmente) ayudaría a **monitorizar** cómo responde el modelo a ciertos prompts del pipeline (ej. el prompt de generación de respuestas documentales, el prompt de clasificación de intención). **Ventajas:** Se podría rastrear fácilmente qué cambios de prompt producen mejoras, al tener un historial centralizado. También habilita comparar desempeño de distintos modelos o ajustes de temperatura con el mismo prompt. En un entorno sin API externa, PromptLayer igualmente puede servir de bitácora local para entender el comportamiento del LLM y detectar casos de alucinación o respuestas fuera de formato. **Desventajas:** PromptLayer está pensado sobre todo para LLMs en la nube (OpenAI,

etc.), por lo que su utilidad con un modelo local puede ser limitada. Habría que enviar registros de prompts y respuestas a sus servidores para aprovechar su UI, lo cual quizás no se desea por políticas (aunque podrían anonimizarse). Alternativamente, habría que hospedar una solución similar on-premise (p.ej. LangChain debug, o simplemente logging a archivo JSON con cada prompt/response). En resumen, la idea de observabilidad es muy valiosa –sobre todo al iterar mejoras– pero quizá se puede implementar más sencillamente con logs personalizados, dado que ya se tiene Prometheus para métricas. Con PromptLayer se ganaría conveniencia a cambio de una dependencia externa.

- **Guardrails AI:** Es una biblioteca que funciona como capa de validación post-proceso (o incluso en cadena) para las respuestas generadas por el LLM. Permite definir **reglas** o esquemas que la respuesta debe cumplir, haciendo que si la respuesta las viola, se intente corregir o señalar el problema. **Ventajas:** En este caso, guardrails podría asegurar que el formato de las respuestas siga las instrucciones. Por ejemplo, el prompt para trámites exige una respuesta estructurada con secciones de “Resumen, Requisitos, Pasos, ...” – Guardrails puede verificar que la respuesta contenga esas secciones y, si falta alguna, pedirle al modelo reintentar o insertar “No informado en las fuentes” donde corresponda. También puede validar que si el contexto era insuficiente, la respuesta contenga la frase “*Información no disponible en las fuentes*” (evitando que el modelo invente información en su lugar). Igualmente, puede censurar o ajustar salidas fuera de tono. En resumen, añade una capa de **seguridad y consistencia**, acercando la calidad al estilo ChatGPT al atrapar errores antes de llegar al usuario. **Desventajas:** Cada guardrail es esencialmente un post-proceso que puede implicar llamadas adicionales al LLM (por ejemplo, guardrails puede reformular una pregunta al modelo para corregir el formato). Esto aumenta la latencia. En CPU, aplicar múltiples validaciones podría ser costoso. Además, diseñar buenos guardrails requiere definir exactamente los criterios (regex, esquemas JSON, etc.) y probarlos extensivamente para no filtrar también respuestas válidas. Dado que ya se le indica al modelo en el prompt que no “invente datos” ni que responda si no hay contexto suficiente, puede que guardrails no sea crítico – a menos que se detecte que el modelo sigue fallando en esto. En suma, es una excelente garantía extra pero con sobre costo; se podría implementar primero ligeras validaciones manuales (p.ej. chequear si la respuesta contiene “No informado” cuando debía) y sólo escalar a Guardrails completo si persisten salidas problemáticas.

En síntesis, la adopción de LangChain y herramientas relacionadas debe sopesar la **complejidad añadida** vs. los beneficios. Dado que MunBoT ya está implementado con lógica propia optimizada (FastAPI, orquestador custom, etc.), una transición total a LangChain no es trivial. Sin embargo, sí se pueden **integrar mejoras puntuales** inspiradas en estos componentes: por ejemplo, usar un enfoque de *routing* por confianza (como sugiere LangChain Router) para decidir entre múltiples índices [medium.com](#), o incorporar guardrails ligeros para mantener la calidad de respuesta. Una estrategia mixta podría ser implementar internamente algunos conceptos (un enrutador sencillo que combine la clasificación actual con embedding semántico como respaldo, o validaciones post-respuesta) sin migrar completamente el stack a LangChain, obteniendo así lo mejor de ambos mundos (mejor cobertura de preguntas y respuestas más controladas) sin sacrificar rendimiento.

3. Opciones de arquitectura/rediseño para respuestas estilo ChatGPT usando RAG (CPU, baja latencia, Qwen-2.5B)

Lograr que MunBoT entregue respuestas con la **calidad conversacional de ChatGPT** utilizando RAG en un entorno sin GPU requiere repensar partes de la arquitectura. Algunas opciones de rediseño a considerar:

- **3.1. Responder en primera instancia con una síntesis amplia (evitar aclaraciones extra):** Un cambio importante sería que el bot **no siempre espere a que el usuario especifique detalles**, sino que intente proporcionar una **respuesta útil desde la primera pregunta**, imitando a ChatGPT. Esto implica relajar la lógica `needs_specific`: en lugar de forzar al usuario a elegir entre “*requisitos, horario, costo...*”, el bot podría dar un **resumen general** del trámite o documento preguntado. Por ejemplo, ante “*permiso de aterrizaje*”, el bot podría responder con una breve descripción del permiso y luego enumerar los datos disponibles (requisitos, dónde obtenerlo, vigencia, etc.), satisfaciendo la consulta inicial. Este resumen inicial orienta al usuario y reduce frustración. **Ventaja:** la interacción se vuelve más **útil y conversacional** – el usuario obtiene información en el primer turno, similar a ChatGPT que suele dar contexto adicional aun si la pregunta es genérica. **Desafío:** sintetizar múltiples fragmentos en una sola respuesta. Una forma de implementarlo es aprovechar el prompt de trámite existente (que ya prevé secciones de Requisitos, Pasos, etc.) y alimentar al LLM **todos los fragmentos relevantes del trámite**. Actualmente el sistema busca fragmentos por similitud; en vez de sólo “requisitos” (porque la pregunta mencionó “permiso”), se podrían recuperar *todos* los trozos del “permiso de aterrizaje” (o al menos los principales) y dejar que el LLM genere una respuesta integrada. Esto aumenta la cantidad de contexto, pero Qwen-2.5B en principio puede manejarlo si el número de fragmentos es limitado (top 5-8). Alternativamente, se puede implementar un paso de **planificación**: identificar qué subtemas cubrir (quizá usando la función `crear_plan` ya incluida [GitHub](#)) y luego generar una respuesta que toque cada punto. En general, esta opción sacrifica algo de brevedad por **completitud**, alineándose más con el estilo explicativo de ChatGPT.
- **3.2. Introducir un agente conversacional con uso de herramientas (estilo ReAct):** Otra ruta es rediseñar el bot como un **agente** que pueda decidir en tiempo real si necesita buscar en la base de conocimiento o responder directamente. En vez de un flujo fijo (clasificar → buscar → responder), un agente controlado por el LLM evaluaría la consulta y podría generar acciones como `<CALL doc-generar_respuesta_llm{"query": "..."}>` dentro de su respuesta, las cuales el orquestador ya está preparado para manejar (el endpoint `/tools/call` existe para esto [GitHub](#)). En modo `AGENT_MODE`, el bot podría, por ejemplo, recibir la pregunta del usuario, “pensar” (cadena de pensamiento interna) y luego decidir usar la herramienta de RAG, obtener el resultado, y finalmente formatear la respuesta. **Ventajas:** Esto imita cómo ChatGPT plugins funcionan – el modelo tiene más libertad para descomponer tareas, realizar múltiples búsquedas si hace falta, o responder directamente si la pregunta es trivial. Podría manejar casos más variados (e.j. si la pregunta combina conocimiento con razonamiento, el agente puede buscar datos y luego inferir). **Desventajas:** Implementar ReAct con un modelo pequeño (2.5B) es arriesgado. ChatGPT tiene la capacidad para planificar y utilizar herramientas con mínimos errores, pero un modelo de 3B podría confundirse con las sintaxis de `<CALL>` o malinterpretar cuándo terminar. Ya el sistema actual contempla un `AGENT_MODE` desactivado por defecto [GitHub](#), señal de que se probó esta idea. Posiblemente se deshabilitó debido a la **latencia**: un agente típico haría al menos dos pasos (una llamada para decidir y otra para la respuesta final), doblando el tiempo de inferencia. En CPU, donde quizá una respuesta actual toma ~5-8

segundos, un agente se iría a 10-15 segundos o más, lo cual podría ser inaceptable en producción. Además, habría que ajustar cuidadosamente los *prompts* del agente para que no divague (limitando el número de llamadas con algo tipo `AGENT_MAX_TOOL_CALLS=2` ya presente [GitHub](#)). En resumen, aunque atractivo conceptualmente, **un agente ReAct con un LLM pequeño puede no justificar el costo en latencia y complejidad**, a menos que la cobertura de casos que gane sea crucial.

- **3.3. Optimizar la búsqueda y recuperación por categoría/domino:** El pipeline actual ya soporta segmentación por categorías (FAQ, Trámites, Normativa) y usa un reranker cruzado para mejorar la precisión de fragmentos [GitHub](#). Se puede reforzar esto inspirándose en arquitecturas recientes: por ejemplo, utilizar un **router de dominio** (como sugiere LangChain) para decidir a qué índice vectorial consultar primero [medium.com](#). Actualmente la clasificación cumple ese rol (elige `collection` “tramites” vs “normativa”), pero afinando esto se puede reducir ruido. Otra idea es implementar un **paso de verificación de respuesta** antes de responder al usuario: la función `verificar_atribucion` en el código construye un prompt que pregunta al LLM si la respuesta puede sostenerse sólo con el contexto dado [GitHub](#). Esto podría usarse para decidir si la respuesta generada es confiable. Arquitectónicamente, significaría agregar un *mini-ciclo*: generar respuesta -> verificar atribución -> si responde “NO”, entonces o bien intentar con más contexto (ej. aumentar `top_k`) o devolver un mensaje “Información no disponible” fiable. Esto emula cómo ChatGPT se autocorrigue cuando no está seguro. **Ventaja:** aumenta la **confianza** de que las respuestas entregadas provienen de las fuentes (evitando alucinaciones). **Desventaja:** es otra llamada al LLM (pregunta SÍ/NO de atribución), aunque breve; podría mitigarse usando un modelo aún más pequeño o lógica heurística (ej. contar cuántos fragmentos del contexto fueron citados en la respuesta). Pese al costo, garantizar respuestas atribuibles mejoraría la calidad estilo ChatGPT (que raramente inventa datos verificables).
- **3.4. Modo multietapa para preguntas complejas:** ChatGPT suele responder paso a paso si una pregunta lo requiere. En RAG, si un usuario hiciera una pregunta muy amplia (“¿Qué acciones realiza el municipio para proteger el medio ambiente y cómo obtener un permiso de construcción?” en una sola consulta), un bot básico podría confundirse entre dos temas. Una arquitectura mejor sería **dividir la consulta en sub-consultas**, buscar cada una y luego integrar las respuestas. El código ya incluye la función `crear_plan` que intenta generar un plan JSON de sub-preguntas usando el LLM [GitHub](#). Se podría activar este mecanismo para casos donde la pregunta del usuario contiene “y” u operadores lógicos, indicando múltiples requerimientos. El plan resultante (lista de preguntas) luego se pasa por el pipeline RAG para obtener fragmentos pertinentes a cada una, y finalmente el LLM puede componer una respuesta final que aborde todos los puntos. **Ventajas:** Permite respuestas **integrales** a consultas multi-tema, con cada parte sustentada en su contexto. Esto es similar a cómo ChatGPT desglosa internamente peticiones complejas. **Desventajas:** Aumenta mucho la latencia, ya que implica múltiples búsquedas y posiblemente varias invocaciones al modelo (dependiendo de si se responde sección por sección o de una sola vez). Dado que consultas tan complejas quizá no sean la norma, podría activarse condicionalmente. En caso de activarse, habría que asegurar que el plan sea fiable – con un modelo pequeño, el plan JSON podría salir mal formado o mal interpretado. Sería necesario robustecer ese paso (quizá con ejemplos en prompt o usando un modelo mayor offline para generar plan templates). En definitiva, es una mejora avanzada que acercaría la experiencia a la de un asistente experto, pero debe usarse con cautela para no penalizar la rapidez en la mayoría de interacciones simples.

Evaluación general: Para mantener baja latencia en CPU, conviene preferir **soluciones de una sola pasada** (ej. proveer respuestas completas desde el inicio, con un solo llamado al LLM usando más contexto, como en 3.1) sobre soluciones multi-llamada (agentes, verificadores en bucle, etc.). Muchas de las capacidades de ChatGPT (respuestas detalladas, contextuales) se pueden emular ajustando la estrategia de entrega de información más que aumentando el tamaño del modelo. En esta línea, se recomienda: **(a)** modificar el flujo para responder con lo que se tenga en la primera interacción (quizá combinando fragmentos automáticamente si la pregunta es general) y **(b)** sólo si la respuesta resulta muy imprecisa, entonces en la segunda interacción pedir aclaración. Esto haría al bot más proactivo y cercano al estilo ChatGPT que “dice todo lo relevante que sabe” con un solo prompt.

Además, cualquier rediseño debe mantenerse **liviano**: limitar sub-llamadas LLM y usar al máximo herramientas no neuronales (filtros de palabras clave, embeddings precalculados) para decidir caminos. Por ejemplo, se puede seguir usando la detección de palabras clave crítica (como “permiso”, “licencia”) pero en lugar de bloquear la respuesta, usarla para determinar qué agregar en la respuesta inicial. En conclusión, **reorientar la arquitectura hacia entregar la información en vez de esperar clarificaciones** es el paso más importante para mejorar la experiencia dentro de las restricciones de CPU y modelo pequeño.

4. Modelos alternativos en la VM (24GB RAM, sin GPU) para mejorar estilo conversacional

El modelo actualmente en uso es **Qwen-2.5B-Instruct** cuantizado (3B parámetros aprox. en 4-bit). Es un modelo relativamente pequeño; aunque afinado para seguir instrucciones, podría tener limitaciones para producir respuestas elaboradas en español. Conviene explorar si hay otros modelos de tamaño manejable en CPU que ofrezcan mejor calidad de salida tipo ChatGPT. Algunas opciones a considerar:

- **Llama 2 7B Chat:** El modelo Llama 2 de 7 mil millones de parámetros (versión Chat afinada por Meta) ha mostrado ser significativamente más capaz que los modelos de ~2–3B en tareas de lenguaje y soporta bien español. Con 24GB de RAM, un Llama-2-7B en 4-bit puede cargarse (6GB de RAM) y permitir contextos amplios. **Ventajas:** Mejor coherencia y estilo conversacional más pulido; probablemente generará respuestas más largas y detalladas sin perder hilo. Dado que está afinado para diálogo, tiende a seguir indicaciones de formato (por ejemplo, no salirse del contexto, abstenerse de inventar datos) de forma más confiable que modelos pequeños. **Desventajas:** Al tener ~3x más parámetros que Qwen-2.5B, será más lento. La latencia por respuesta podría aumentar notablemente (quizá 2–3 veces más lenta dependiendo de optimizaciones). Además, la versión libre de Llama2 7B tiene ciertas restricciones de uso comercial (licencia), habría que verificar compatibilidad con el proyecto. En términos de manejo, ocupará más memoria, pero 24GB es suficiente para una instancia; el problema sería si se quieren múltiples trabajadores en paralelo. Habría que evaluar la carga concurrente.

- **Llama 2 13B Chat:** Este modelo (13B parámetros) ofrecería aún mayor calidad, acercándose más a ChatGPT en entendimiento y fluidez. En 4-bit cuantizado ocuparía ~10–12GB, por lo que cabe en 24GB incluso con 2 hilos quizá. **Ventaja:** Respuestas más *naturales* y contextualmente precisas; mejor capacidad de inferencia y seguimiento de instrucciones complejas. **Desventaja:** La **latencia** en CPU podría ya ser borderline (13B puede tardar el doble que 7B a igualdad de condiciones). Si una respuesta de 7B toma 6 segundos, 13B podría tomar ~12–15 segundos, lo cual tal vez es demasiado para usuarios en producción. También habría menos headroom de RAM si se quieren correr otros servicios en la VM simultáneamente. Aun así, técnicamente es viable y valdría hacer una prueba offline con conversaciones típicas para medir si la mejora cualitativa compensa el tiempo. Muchas implementaciones de bots similares hallan que 13B ofrece un salto visible en calidad frente a 7B, pudiendo justificarlo si la experiencia lo requiere.
- **Modelos de ~6B orientados a chat (MPT-7B, XGen-7B, Falcon-7B):** Existen varios modelos open-source en el rango 5–7B que han sido entrenados o afinados para diálogo. Por ejemplo, **Mistral 7B** (un modelo europeo de 2024 con muy buen rendimiento), **MPT-7B Chat** de MosaicML, o **Falcon-7B-Instruct**. Estos podrían considerarse alternativas a Llama 2 7B. **Ventajas:** Algunos de ellos tienen licencias más permisivas (Falcon es Apache 2.0) o especializaciones (MPT tiene una variante para respuestas largas). Podrían tener cierta ventaja en velocidad también, dependiendo de optimizaciones. **Desventajas:** En pruebas independientes, Llama2 tiende a superar a otros 7B en muchas tareas, especialmente en coherencia multilinguaje. Modelos como Falcon o XGen podrían requerir más retoques de prompt para español. En general, si se busca consistencia, Llama2 Chat se perfila como la opción más equilibrada por la comunidad; los demás son plan B si hubiera restricción de licencias.
- **Modelos <3B más nuevos:** El modelo Qwen-2.5B fue elegido seguramente por su tamaño reducido. Podría explorarse si hay otros modelos mini (2–4B) entrenados posteriormente con técnicas más avanzadas. Por ejemplo, Alibaba (mismo creador de Qwen) podría lanzar una versión mejorada, o GPT-3 de código abierto distilado a 4B, etc. Hasta la fecha, la mayoría de modelos open source por debajo de 7B no alcanzan un nivel “chatGPT-like” en calidad – suelen dar respuestas más cortas o con menor comprensión contextual. **Recomendación:** Centrarse en la franja 7B–13B para notar un salto cualitativo. Modelos de 3B probablemente seguirán requiriendo mucha ingeniería de prompt y aún así lucirán menos “naturales” que ChatGPT.

Consideraciones de rendimiento: Con 24GB RAM y sin GPU, es fundamental optimizar la inferencia. Ya se está usando un formato GGUF 4-bit que reduce memoria. Se puede habilitar el uso de BLAS multi-thread o incluso GPU offloading parcial si hubiera una GPU pequeña disponible en la VM (no parece el caso). Otra opción es recurrir a técnicas como **quantización más agresiva (GPTQ 4bit, etc.)** o bibliotecas optimizadas (llama.cpp con int4 + CPU SIMD). Al cambiar de modelo, hay que probar su compatibilidad con estas optimizaciones. Por ejemplo, Llama2 13B en 4-bit probablemente funcione bien con llama.cpp.

Estilo conversacional: Más allá de los parámetros, influye la **afinación** del modelo. Qwen Instruct está ajustado a seguir instrucciones, pero su estilo puede ser más telegráfico. Llama2-Chat tiene un estilo más cercano a ChatGPT (por diseño), incluyendo frases de cortesía, explicaciones ordenadas, etc. Eso puede satisfacer mejor la expectativa de los usuarios. Adicionalmente, si se desea un tono aún más personalizado (p.ej. lenguaje más formal o más cercano), se podría aplicar un ligero fine-tune de *instrucción* en español al modelo escogido, usando las propias transcripciones y documentos del municipio para incorporar vocabulario específico. Sin embargo, este fine-tuning requeriría recursos adicionales fuera de la VM (entrenar en CPU no es factible) – es más una posibilidad a mediano plazo.

Resumen: No existe un modelo pequeño mágico que iguale a ChatGPT, pero escalar a ~7B o 13B sí ofrecerá respuestas más ricas. Si la latencia objetivo es muy estricta (<5s), quizá 7B sea el techo práctico. Se recomienda hacer una prueba A/B con Llama2 7B para medir tiempos y calidad en escenarios reales de MunBoT. Si el resultado es claramente superior en satisfacción de usuario (p. ej., menos negativas en feedback, conversaciones más fluidas), valdrá la pena optimizar en torno a ese modelo. Caso contrario, se mantendrá Qwen-2.5B pero implementando el resto de mejoras de pipeline para exprimirlo al máximo. Cabe destacar un punto del artículo de Thinking Loop: *“No necesitas un modelo más grande para mejores respuestas, sino un router que escoja bien la fuente de datos”* [medium.com](#). Esto enfatiza que, antes de escalar modelo, hay que asegurar que el pipeline (clasificación + búsqueda) está sirviendo el **contexto correcto** al LLM. Un modelo más grande no compensará un contexto mal elegido. Por ello, primero afinar la parte RAG (secciones 1, 2, 3, 5) y luego, con esa base sólida, evaluar un modelo superior para cerrar la brecha de estilo conversacional.

5. Mejora de la clasificación de intención (normalización, umbrales, fallback RAG en “n/a”, etc.)

Dado que la clasificación de intención es el **punto de entrada crítico** del flujo, conviene reforzarla para reducir errores y maximizar que las consultas informativas caigan en el pipeline RAG en lugar de terminar en fallbacks. Algunas propuestas concretas:

- **5.1. Normalizar y mapear la salida del LLM classifier:** Actualmente, el clasificador mezcla un enfoque basado en reglas/IR con un LLM de respaldo. Ya se normaliza la etiqueta salida del LLM a minúsculas y se confina a las categorías esperadas [GitHub](#). Asegurarse de incluir todas las variantes posibles: por ejemplo, si el LLM pudiera devolver “tramite” con tilde (trámite), se debe normalizar a tramite . En el código se ve que comparan con versiones acentuadas en algunos lugares [GitHub](#). Sería conveniente centralizar una función `_norm_intent` que elimine tildes y plural/singular de la etiqueta antes de usarla. De hecho, en el manejo de categorías ya se considera equivalentes "tramite"/"trámite(s)" [GitHub](#). Revisar que esto esté aplicado en todos los caminos (p. ej., en el orquestador se hace `_norm_intent(categoria)` antes de decidir colección [GitHub](#)). En resumen, garantizar que trivialidades como mayúsculas, tildes o idioma (“none” vs “n/a”) nunca causen una etiqueta fuera de lugar.
- **5.2. Ajustar umbrales de confianza del IntentEngine:** El motor de intents actualmente usa un **umbral estático** `UMBRAL_SCORE = 0.18` con ajuste dinámico para frases cortas (0.08–0.12) [GitHub](#). Este umbral determina si se devuelve la intent encontrada o se cae a “n/a” [GitHub](#). Sería prudente re-evaluar este valor con datos reales: por ejemplo, ver qué puntajes recibieron consultas que el bot manejó mal. Si se observa que muchas preguntas legítimas quedaron por debajo de 0.18 (pero cerca, digamos 0.15)

produciendo falsos “n/a”, podría bajarse ligeramente el umbral. Alternativamente, se podría graduar mejor: *si* la mejor coincidencia da intent “documento/trámite” con score apenas bajo el umbral, en vez de “n/a” quizá convenga devolver igualmente esa intent pero marcando `needs_disambiguation=True` (ya se hace en el dict de salida [GitHub](#)). Entonces el orquestador podría tratar ese caso no como un *no entendido*, sino como un **caso de baja confianza** donde igualmente se intentará RAG pero con precauciones. Esto enlaza con el siguiente punto.

- **5.3. Fallback a RAG cuando intent = “n/a” (o confianza baja):** En lugar de rendirse cuando la intención es incierta, proponemos un cambio: **intentar la vía RAG por defecto** para consultas clasificadas como desconocidas. Concretamente, si el clasificador principal devuelve “n/a”, el orquestador haría un intento de búsqueda en la colección más amplia (por ejemplo, la base de *Normativa/Documentos* general). Esto se puede implementar de forma sencilla: actualmente el orquestador hace `return _heuristic_classify(text)` en ese caso [GitHub](#), lo cual suele retornar “faq” genérico. En vez de finalizar ahí, se podría saltar al flujo de `intent_action == "ask_document"` pero con una categoría neutra. Por ejemplo, suponer intent “faq” e invocar `handle_document_query` con `categoria=None` para que el sistema busque en todas las colecciones (el código de búsqueda ya soporta `collection=None` para buscar globalmente [GitHub](#)). Así, ante un “n/a” se comportaría como una pregunta informativa general. **Ventajas:** Esto aprovechará la base de conocimiento: si la pregunta efectivamente era respondible, se obtendrán fragmentos y el LLM dará alguna respuesta, en lugar de un mensaje de confusión. **Riesgo:** Podrían haber casos donde la pregunta era realmente fuera de dominio (ej. una pregunta personal o incoherente). Entonces la búsqueda RAG quizás retorne cualquier fragmento vagamente relacionado, y el LLM intente construir una respuesta que no atienda la intención real (posible respuesta irrelevante). Para mitigar esto, podemos combinar con una verificación: si tras intentar RAG la respuesta generada no tiene suficiente soporte (usando `verificar_atribucion` como se comentó en 3.3, o midiendo el score de los fragmentos: si el top hit tiene score muy bajo, probablemente era ruido), entonces sí mandar el fallback “*No tengo información sobre ese tema*”. Es decir, **fallback a RAG primero, y sólo si la respuesta parece no satisfactoria, entonces fallback al usuario**. Esta doble capa asegurará que no se descarten oportunidades de responder. Cabe la posibilidad de marcar en métricas estas recuperaciones (ej. contar cuántos `intent:n/a` luego lograron respuesta vía RAG) para monitorear.
- **5.4. Incorporar similitud semántica para mejorar la clasificación:** Actualmente se usa alias exactos y solapamiento de tokens. Una mejora sería agregar una etapa de **vectorización** de la consulta y comparación con embeddings de las preguntas frecuentes/documentos (posiblemente ya existen embeddings en Qdrant). De hecho, el orquestador tiene una función `_similarity_rescue` que parece usar embeddings precalculados (carga `SIM_VECTORS` al inicio [GitHub](#) y compara la nueva pregunta contra un set de frases de regresión) para rescatar intent misclasificados [GitHub](#). Esto ya indica beneficio: por ejemplo, podría estar detectando que cierta pregunta es muy similar a otra conocida con intent “agenda” y corrigiendo la etiqueta [GitHub](#). Ampliando esa idea, se podría usar la propia **búsqueda en la base como clasificador**: si la consulta tiene un embedding que retorna un fragmento con alta similitud en la colección de trámites, es señal de que debe ser intent “tramite/documento”. Esto requiere experimentación para definir umbrales. Una integración posible: *antes* de declarar “n/a” definitivo, calcular la similitud de la consulta con cada conjunto de documentos (tal vez mediante centroides o simplemente viendo en qué colección aparece su vector top-1 score más alto). Si, digamos, el vector de la pregunta devuelve un resultado en `tramites` con score coseno > X, entonces asumir intent “tramite” aunque las palabras no coincidieron. Esto complementaría el método actual basado en tokens, cubriendo sinónimos o formulaciones distintas. **Desventaja:** Computar embeddings de la consulta para cada mensaje podría duplicar trabajo, dado que luego igualmente se computa para la búsqueda principal. Sin embargo, Qdrant o el pipeline de RAG ya hacen embed de la consulta una vez para buscar fragmentos [GitHub](#); quizás se pueda reutilizar ese vector para también clasificar. O usar un modelo de embedding ligero separado (por ejemplo, MiniLM) sólo para intención. Dado que Qdrant aloja vectores, una idea es aprovechar la misma búsqueda: *si* la mejor coincidencia global en Qdrant pertenece a un documento/trámite conocido, usar esa info para clasificar. Por ejemplo, si el chunk más relevante tiene tag “tramite”, marcarlo como tal. Esto ya sucede indirectamente cuando el motor RAG busca, pero en la secuencia actual la búsqueda se hace **después** de la clasificación. Habría que cerrarlo en un bucle de realimentación: cuando `intent_classifier` duda, podría llamar a `rag.search_in_qdrant` en modo amplio con `topk=1` *para ver qué encuentra, y usar esa pista. Realizar esta consulta extra sólo en caso de “n/a” del primer paso sería eficiente (no para todas las queries)*. **Conclusión:** Integrar la **inteligencia semántica** de los embeddings en la decisión de intent reduciría los falsos `_n/a` debidos a vocabulario.
- **5.5. Enriquecer el catálogo de alias y frases entrenadas:** Otra vía práctica (aunque menos elegante) es analizar qué preguntas han fallado y agregarlas como alias de los items correspondientes. Por ejemplo, añadir “principios medioambientales” como alias del fragmento de *Principios del reglamento ambiental*, o incluir “permiso de aterrizaje” (sin “de”) como alias general. El sistema ya tiene un JSON de intent de regresión (`intent_regresion.json`) para pruebas [GitHub](#), lo cual sugiere que pueden incorporar nuevas frases para asegurar que clasifiquen consistentemente en cada versión. **Ventaja:** Es una solución rápida: actualizar los archivos RAG-*.json con alias adicionales mejora inmediatamente la detección, porque el IntentEngine normaliza y busca esos alias [GitHub](#). **Desventaja:** Es reactivo y escalable hasta cierto punto. Siempre habrá nuevas formulaciones que se escapen, por lo que depender únicamente de curar alias es frágil. Aun así, mantener un registro de las consultas “n/a” más frecuentes en producción y alimentarlas como alias o entradas de FAQ explícitas es una buena práctica de mejora continua. Por ejemplo, si muchos usuarios preguntan “¿Cuál es el **mail** de licencia de conducir?” y eso no estaba textual, se le puede añadir (de hecho, en los tests se ve un ejemplo similar con “cual es el mail de la licencia de conducir” [GitHub](#) asegurándose que llame a RAG).
- **5.6. Refinar la lógica de smalltalk vs. pregunta informativa:** Asegurarse de que el fastpath de smalltalk (saludos, despedidas) no interfiere con preguntas reales. El regex de saludo por ejemplo detecta “hola”, “qué tal” [GitHub](#). Si un usuario combinara saludo con pregunta (ej. “Hola, ¿tienen información sobre permisos?”), es crucial que el bot no se quede solo con “hola” y responda un saludo cortés pero ignore la pregunta. El diseño actual parece manejarlo – sólo hace fastpath if el input es muy corto (<=3 palabras) o coincide con esas frases aisladas [GitHub](#). Aun así, conviene verificar con pruebas. Si se hallara que algunas preguntas con “gracias” o “hola” estaban siendo clasificadas como smalltalk (`intent: "faq" + sub_intent saludo/agradecimiento`) perdiendo el resto, habría que ajustarlo (posible solución: si la entrada contiene también signos de interrogación o palabras clave de trámite, priorizar eso sobre el saludo). Este ajuste fina garantiza que la intención informativa no sea enmascarada por cortesías.

En resumen, **la mejora de la clasificación** pasa por hacerla más tolerante a variaciones y por evitar abandonar las consultas borderline. Bajar la barrera de “n/a” y tratar más casos como preguntas de conocimiento llevará a que el bot intente responder más seguido, lo que

es deseable. Al mismo tiempo, se necesita una **estrategia de verificación** para esos casos recuperados, de modo que si realmente no hay nada relevante en la base, no se produzca una respuesta inventada. Una estrategia ya mencionada es la verificación de atribución o incluso un paso adicional donde si tras fallback-RAG el modelo produce algo claramente fuera de contexto, se reemplace por “*Lo siento, no encontré información sobre X*”. Esto es más manejable que pedirle al usuario reformular de entrada.

Implementando estas mejoras, se espera ver: menos clasificaciones “n/a” (porque algunas pasarán a tramites/documentos con confianza baja), menos fallbacks conversacionales, y en general un flujo más **permisivo hacia RAG**. El bot así errará del lado de *intentar responder* en vez de *desistir pronto*, alineándose con la filosofía de asistentes modernos.

6. Integración con métricas existentes y medición del impacto de las mejoras

Actualmente MunBoT expone métricas Prometheus útiles [GitHub](#), entre ellas: número total de peticiones (`munbot_requests_total`, etiquetado por intent), conteo de fallbacks (`munbot_fallbacks_total`), conteo de escalamientos a humano (`munbot_human_escalations_total`), errores por microservicio, y latencia de RAG (`rag_latency_seconds` histogram). Para incorporar las mejoras propuestas y evaluar su efectividad, se debe:

- Utilizar las métricas clave como indicadores de mejora:** El principal objetivo es **reducir la tasa de fallbacks** y la necesidad de escalamiento a humano, al brindar respuestas útiles donde antes no las había. Con las modificaciones, se espera que `munbot_fallbacks_total` incremente más lentamente respecto a `requests_total`. Se puede monitorizar el ratio (Prometheus query: `munbot_fallbacks_total/munbot_requests_total`) antes y después. Si baja significativamente, será evidencia de éxito (el objetivo podría ser bajar del X% actual a algo mucho menor). Asimismo, una baja en `human_escalations_total` indicará que menos conversaciones llegan al punto de 3 fallbacks seguidos sin solución. Cada escalamiento implica que el bot falló repetidamente, por lo que reducir esos eventos es crítico. Estas métricas ya están definidas, sólo requerirán observar su evolución tras cada cambio.
- Incorporar nuevos tags de métricas si es útil:** Por ejemplo, podría ser interesante contabilizar cuántas veces se activa el *fallback a RAG en intent n/a*. Esto podría registrarse con un Counter personalizado, ej. `munbot_recovered_queries_total` con labels {via="fallback_rag"} o similar, cada vez que una pregunta inicialmente “n/a” termina teniendo respuesta gracias al nuevo flujo. No es imprescindible, pero ayuda a demostrar cuántas respuestas adicionales está dando el bot debido a la mejora. Igualmente, se podría instrumentar un counter para cuando el bot usa el **nuevo camino resumido** para preguntas genéricas (la modificación de `needs_specific`). Por ejemplo, `munbot_auto_summary_total` cada vez que en lugar de preguntar “qué info específica”, el bot entregó una respuesta general. Estas métricas custom permitirían cuantificar el impacto directo de las nuevas funciones.
- Monitorear latencia del pipeline RAG:** Con más consultas yendo a RAG (incluyendo las antes filtradas por “n/a”), la métrica `rag_latency_seconds` podría aumentar ligeramente en promedio. Habrá que vigilar los percentiles P50/P90 de ese histograma. Si se detecta una subida significativa, habrá que optimizar: quizás ajustando el número de fragmentos `top_k` dinámicamente (por ej., usar `top_k` más bajo en consultas recuperadas de fallback para no sobrecargar), o asegurarse de no hacer embeddings redundantes como se mencionó. No obstante, una pequeña penalización de latencia es esperable dado que antes algunas consultas terminaban en <1s con un fallback rápido, y ahora esas mismas harán búsqueda y generación. Esto es un trade-off aceptable siempre que esté dentro de límites. **Medir** este efecto es importante para calibrar: si la latencia P90 se vuelve demasiado alta, se podría considerar recortar ciertos pasos (por ej., si implementamos verificación de atribución con LLM, medir si añade 1s promedio y decidir si lo mantenemos o lo activamos sólo en respuestas largas).
- Métricas de intención y respuesta para calidad:** El metric `munbot_requests_total` con label por intent puede revelar cambios en distribución. Por ejemplo, después de mejoras quizás el porcentaje de intents “n/a” registrados baje y aumenten “documento” o “tramite”. Esto confirmaría que el clasificador está etiquetando más correctamente. También conviene observar el label `intent="faq"`: antes podía englobar tanto FAQs reales como preguntas informativas genéricas (por diseño original, muchas cosas caían en “faq” por defecto). Tras los cambios, idealmente más de esas pasarán a “documento”/“tramite”. Una disminución del porcentaje de `faq` intents junto con un aumento en `documento/tramite` intents indicaría un **enrutamiento más preciso** a RAG.
- Feedback directo del usuario:** Aunque no está instrumentado en Prometheus, el sistema tiene una mecánica de feedback en la conversación (pregunta “¿Te fue útil mi respuesta?” y espera “sí/no”). Ese feedback se refleja en la lógica de *fallbackcount* y *eventualmente escalamiento*. *Sería valioso aprovecharlo cuantitativamente: por ejemplo, cuántos “no”_ se registran por 100 preguntas*. Actualmente, cada “no” incrementa `fallback_count` y eventualmente escalamiento, pero se podría exponer un Counter como `munbot_feedback_negative_total`. Si con las mejoras las respuestas son más acertadas, el usuario debería decir “Sí” con mayor frecuencia. Una disminución en el conteo de respuestas negativas (o en la proporción de conversaciones que llegan a escalamiento) sería un indicador fuerte de éxito en la calidad estilo ChatGPT. Implementar esta métrica extra (si no existe ya) ayudaría a cerrar el circuito de evaluación.
- Estrategia de despliegue y medición A/B:** Para medir impacto de cada mejora aislada, lo ideal es introducir cambios gradualmente. Por ejemplo, primero activar el *fallback a RAG en n/a* y observar métricas una semana; luego modificar lo de *needs_specific* y observar nuevamente. Si se despliegan todas las mejoras a la vez, igualmente se debería realizar un seguimiento cercano comparando con una línea base anterior. En ausencia de infraestructura de A/B testing, valdrá basarse en comparaciones temporales (pre vs post) con cuidado de que el volumen de consultas y tipos sean comparables.
- Correlacionar con logs de casos específicos:** Más allá de los números agregados, conviene revisar logs de conversaciones reales después de cada cambio. Por ejemplo, verificar manualmente que ahora ante “*permiso de aterrizaje*” el bot ofreció info en la primera respuesta, o que “*principios medioambientales del municipio*” devolvió efectivamente los principios (quizá listando los 5 principios del reglamento ambiental) en lugar de preguntar de nuevo. Estos casos de uso específicos deben probarse en entorno de staging y también monitorearse en producción inicialmente, asegurando que el comportamiento deseado concuerda con las expectativas. Si se dispone de transcripts anonimizados, se pueden buscar aquellas consultas que antes tenían malas respuestas y ver la mejora cualitativa en la nueva versión.

En cuanto a **integración con el sistema de métricas**, las mejoras discutidas no requieren cambios drásticos: simplemente, usar los counters e histogramas existentes para guiar las decisiones. Asegurarse de documentar qué significan los nuevos valores tras el cambio (por ejemplo, si baja la tasa de fallbacks pero sube levemente la latencia media, justificarlo en informes internos). Se podría añadir entradas en `METRICS.md` para cualquier métrica nueva introducida (como las sugeridas de recuperaciones o feedback), manteniendo la transparencia.

Para concluir, **cada mejora propuesta debe venir acompañada de un seguimiento en métricas**: esto permite no sólo verificar la efectividad, sino detectar rápidamente efectos adversos (p.ej., si cierto cambio accidentalmente aumentó errores en microservicios, reflejado en `mcp_microservice_errors_total`). Con el riguroso monitoreo ya implementado, MunBoT podrá iterar hacia respuestas más útiles sin perder de vista los KPI de performance. El objetivo final es que las métricas post-mejora demuestren un **asistente más eficiente (menos fallbacks/escalaciones) y aún así rápido**, acercándose así a la experiencia que brinda ChatGPT pero usando únicamente recursos locales.

CONFIGURACIÓN DEMO

Qué significa “listo para demo”

- Alcance cerrado**: define 4–6 casos que vas a mostrar (FAQ, Trámite, Documento, Agendar, Reclamo, Smalltalk) y optimiza solo eso.
- Comportamiento determinista**: misma entrada → misma salida.
- Datos sembrados**: colecciones RAG cargadas y probadas; ejemplos reales con párrafos que el bot pueda citar.
- Degradación elegante**: si algo falla, cae a una respuesta clara (no silencios, no errores técnicos).

Perfiles recomendados (elige uno según show)

A) “SAFE DEMO” (máxima estabilidad)

```
AGENT_MODE=0
AGENT_CANARY_RATIO=0
AGENT_MAX_TOOL_CALLS=0
LLM_TEMPERATURE=0.1
RAG_CATEGORY_AWARE=1
RAG_MIN_SCORE=0.35
TRACE_SAMPLING=0.0
HUMAN_ESCALATION_ENABLED=0
FALLBACK_MESSAGE="No tengo esa info aquí, pero puedo derivarte o tomar tus datos."
```

B) “SHOWCASE” (para lucir herramientas)

```
AGENT_MODE=1
AGENT_CANARY_RATIO=0
AGENT_MAX_TOOL_CALLS=1
LLM_TEMPERATURE=0.15
RAG_CATEGORY_AWARE=1
RAG_MIN_SCORE=0.35
TRACE_SAMPLING=0.1
HUMAN_ESCALATION_ENABLED=0
```

Tip: ten listo un switch rápido (variables en `.env` o `docker compose --env-file`) para cambiar de SAFE→SHOWCASE si el ambiente está inestable.

Pre-flight de 10 minutos (antes de entrar)

- Salud**: `GET /health` de todos los servicios (orchestrator, llm_docs, scheduler, complaints, qdrant).
- Qdrant**: colección con puntos (>0) y consulta de ejemplo del **mismo** tema que mostrarás.
- Warm-up**: dispara 1 FAQ, 1 Trámite, 1 Documento; revisa que devuelve fuentes y que el texto no “inventa” plazos.
- Transaccionales**:
 - Agendar: desde “Quiero agendar hora” hasta confirmación (sin ejecutar commit si no quieres).
 - Reclamo: revisa validación RUT/email y confirmación final.
- Latencia**: <1.5 s en las 3 consultas informativas.

Golden paths sugeridos (copiar y usar)

- FAQ**: “¿Cuál es el horario de atención del municipio?”
- Trámite**: “Requisitos y pasos para la licencia de conducir clase B.”
- Documento**: “¿Qué dice la ordenanza sobre ruidos molestos y desde cuándo rige?”
- Agenda**: “Quiero agendar una hora con inspección.”
- Reclamo**: “Quiero poner un reclamo por bache frente a mi casa.”

- **Smalltalk:** “Gracias / Hola / ¿Quién eres?”

Controles de riesgo (quita estrés al vivo)

- **Determinismo:** baja el `temperature` , limita `AGENT_MAX_TOOL_CALLS=1` .
- **Sin canary:** `AGENT_CANARY_RATIO=0` .
- **Umbral RAG:** si `top1 < 0.35` → responde “Información no disponible en las fuentes” (mejor eso que alucinar).
- **Copia de seguridad:** ten una respuesta “amable + derivación” lista si un flujo transaccional se cae.
- **Offline razonable:** modelos/embeddings y colecciones cargadas localmente; evita dependencias externas frágiles.

Definición de “OK para demo”

- 95% de éxito en tus 5–6 preguntas guionadas.
- 0 errores visibles (500/tracebacks) en pantalla.
- Latencia p95 < 1.5 s para FAQ/Trámite/Documento.
- Confirmación **siempre** antes de commits (agenda/reclamo).
- Logs y métricas activos por si necesitas diagnosticar (pero `TRACE_SAMPLING` bajo).